

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

درس یادگیری ماشین

استاد: دکتر امیرحسین نیکوفرد

تمرین سری پنجم

گروه جاویدنام سپهر ایران

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	آرین خوشنویس
شماره دانشجویی	۴۰۱۱۷۹۸۳
تاریخ	تیر ۱۴۰۵



فهرست مطالب

۵	۱	اسنپ - مبانی و توکن سازی
۵	۱.۱	مرور کلی تمرین
۶	۲.۱	بارگذاری کتابخانه‌ها و آماده‌سازی محیط
۷	۳.۱	بررسی قابلیت‌های کلیدی JAX: grad, jit و vmap
۸	۴.۱	پیاده‌سازی Softmax پایدار عددی
۱۰	۵.۱	بارگذاری و تحلیل مقدماتی پیکره متنی شکسپیر
۱۱	۶.۱	ساخت توکنایزر کاراکتری
۱۳	۷.۱	تقسیم داده به مجموعه آموزش و اعتبارسنجی
۱۴	۸.۱	هدف پیش‌بینی توکن بعدی و ساخت مینی‌بچ‌ها
۱۶	۹.۱	ارزیابی عملکرد تولید بچ‌ها (Batch Generation)
۱۸	۱۰.۱	تحلیل آماری روابط کاراکترها (Bigram Transition Matrix)
۱۹	۱۱.۱	تحلیل بصری (Heatmap)
۱۹	۱۲.۱	بررسی یک نمونه: بعد از فضای خالی (Space)
۲۰	۲	دیجیکالا - مکانیزم توجه
۲۰	۱.۲	بخش دوم: پیاده‌سازی مکانیزم خودتوجهی (Self-Attention) در دیجی‌کالا
۲۱	۲.۲	Scaled Dot-Product Attention و ماسک علی
۲۴	۳.۲	بصری‌سازی ماتریس توجه: تقابل حالت آزاد و علی
۲۵	۴.۲	چرا امتیازهای توجه را بر $\sqrt{d_{\text{head}}}$ تقسیم می‌کنیم؟
۲۹	۵.۲	کدگذاری موقعیتی سینوسی (Sinusoidal Positional Encoding)
۳۳	۶.۲	خود-توجهی تک‌سر (Single-head Self-attention)
۳۶	۷.۲	تحلیل رفتار توجه: آنتروپی توزیع توجه تحت ماسک علی
۳۹	۸.۲	تحلیل رفتار توجه: متوسط فاصله بازگشت به عقب (Average Look-Back Distance)
۴۱	۳	تپسی - توجه چندسر و بلوک ترنسفورمر
۴۱	۱.۳	nn.Module Flax (init / apply) و PyTrees پارامترها
۴۳	۲.۳	Attention Multi-Head (توجه چندسری)
۴۶	۳.۳	GELU و MLP در هر موقعیت (Position-wise MLP)
۴۸	۴.۳	Layer Normalization (نرمال‌سازی لایه‌ای)
۵۰	۵.۳	اتصالات باقیمانده (Residual Connections) و Pre-Norm در Transformer
۵۲	۶.۳	توجه علی
۵۴	۷.۳	آشنایی با جریان کاری Flax: الگوی Init / Apply
۵۶	۸.۳	طراحی ماژول توجه چندسر علی (Causal Multi-Head Self-Attention) در Flax
۶۰	۹.۳	هر سر توجه به چه چیزی نگاه می‌کند؟ نقشه‌های توجه مجزا (Per-head Attention Maps)
۶۳	۱۰.۳	توابع فعال‌ساز در شبکه‌های پیش‌خور: مقایسه GELU, ReLU و tanh



۶۵	پایاده‌سازی دستی نرمال‌سازی لایه‌ای (Layer Normalization)	۱۱.۳
۶۷	مونتاژ بلوک کامل ترنسفورمر با معماری پیش‌نرمال‌سازی (Pre-Norm Transformer Block)	۱۲.۳
۷۰	تحلیل تکمیلی: نحوه رشد جریان میان‌بر (Residual Stream) با افزایش عمق شبکه	۱۳.۳
۷۲	کوئرا - ساخت و آموزش MiniGPT	۴
۷۲	از بلوک‌ها تا مدل کامل: Embedding و Head خروجی	۱.۴
۷۴	تابع هزینه Cross-Entropy در پیش‌بینی توکن بعدی	۲.۴
۷۶	بهینه‌سازی آموزش: AdamW، برش گرادیان و زمان‌بندی نرخ یادگیری	۳.۴
۷۹	مونتاژ معماری کلان: ماژول MiniGPT	۴.۴
۸۱	تابع زیان آنتروپی متقاطع (Cross-Entropy Loss)	۵.۴
۸۳	زمان‌بندی نرخ یادگیری: Warmup-Cosine Decay	۶.۴
۸۵	گام آموزش و کامپایل JIT در JAX	۷.۴
۸۷	آموزش مدل و تولید متن (Training and Inference)	۸.۴
۸۸	تحلیل نهایی و عیب‌یابی مدل (Diagnostics)	۹.۴
۸۹	تحلیل نهایی: مدل چه چیزی آموخته است؟ (What did the trained model learn?)	۱۰.۴
۹۲	دیوار - تولید متن و مقیاس	۵
۹۲	Temperature	۱.۵
۹۲	Top-k	۲.۵
۹۲	Top-p	۳.۵
۹۳	کد بخش ۵	۴.۵
۱۰۱	خروجی های نهایی	۵.۵



فهرست تصاویر

۱	مقایسه سرعت اجرای JIT و نمایش خروجی VMAP برای محاسبه نرم ردیف‌ها	۸
۲	اثر تغییر پارامتر دما بر توزیع احتمالات Softmax	۹
۳	فراوانی کاراکترها و توزیع طول خطوط در مجموعه داده شکسپیر	۱۱
۴	تقسیم پیکره به بخش‌های آموزش و اعتبارسنجی	۱۴
۵	رابطه جابجایی یک گام به چپ میان ورودی x و هدف y برای عبارت "To be or"	۱۵
۶	نمودار حرارتی (Heatmap) از یک بیچ نمونه شامل ۱۲ ردیف و ۲۴ ستون توکن	۱۸
۷	ماتریس احتمالات انتقال بیگرام: محور عمودی کاراکتر فعلی و محور افقی کاراکتر بعدی را نشان می‌دهد.	۱۹
۸	مقایسه توزیع وزن‌های توجه در دو حالت بدون ماسک (چپ) و با اعمال ماسک علی (راست)	۲۵
۹	نمودار سمت چپ رشد خطی واریانس ضرب نقطه‌ای خام با افزایش بعد را نشان می‌دهد، در حالی که نمودار سمت راست پایداری واریانس حول مقدار ۱ را پس از مقیاس‌گذاری به تصویر می‌کشد	۲۸
۱۰	نقشه حرارتی ماتریس کدگذاری موقعیتی سینوسی (سمت چپ) و رفتار نوسانی ابعاد مختلف مدل بر حسب موقعیت با فرکانس‌های گوناگون (سمت راست)	۳۲
۱۱	نقشه حرارتی ماتریس توجه برای عبارت نمونه. ساختار پایین مثلثی ماتریس به وضوح نشان‌دهنده اعمال موفقیت‌آمیز ماسک علی است.	۳۶
۱۲	نمودار آنتروپی توجه به ازای هر کاراکتر ورودی. خط چین نارنجی مرز تنوریک آنتروپی (توزیع کاملاً یکنواخت) و ستون‌های آبی، آنتروپی واقعی مدل را نشان می‌دهند.	۳۸
۱۳	متوسط فاصله بازگشت به عقب توجه در مقایسه با مرجع یکنواخت بر روی یک توالی به طول ۴۸ توکن.	۴۰
۱۴	نقشه حرارتی یک سر توجه علی (Single Causal Attention Head). ساختار پایین‌مثلثی، نشان‌دهنده اعمال صحیح محدودیت‌های علی است.	۵۳
۱۵	نقشه حرارتی توجه علی به ازای هر سر (Per-head) روی یک قطعه متن ثابت. به دلیل تصادفی بودن وزن‌های اولیه (Random Init)، الگوها در تمام سرها تقریباً یکنواخت و فاقد معنای زبانی هستند، اما ساختار علی (ماسک پایین‌مثلثی) در تمامی آن‌ها به شدت رعایت شده است.	۶۲
۱۶	مقایسه رفتار توابع فعال‌ساز GELU، ReLU و tanh. تابع GELU (آبی) برخلاف ReLU (نارنجی) یک منحنی نرم است که اجازه عبور مقادیر منفی کوچک را می‌دهد.	۶۴
۱۷	هیستوگرام توزیع مقادیر فعال‌سازی. سمت چپ: پیش از نرمال‌سازی (پراکنده و دارای انحراف). سمت راست: پس از نرمال‌سازی (متمرکز حول صفر با واریانس واحد).	۶۶
۱۸	نمودار توزیع پارامترها در زیرماژول‌های مختلف یک بلوک ترنسفورمر (مجموعاً ۶۰۸، ۱۲ پارامتر). بخش عمده‌ای از ظرفیت یادگیری به شبکه پیش‌خور (MLP) اختصاص یافته است.	۶۹
۱۹	رشد میانگین نرم L2 در جریان میان‌بر با افزایش تعداد بلوک‌های ترنسفورمر در حالت مقداردهی اولیه تصادفی.	۷۰
۲۰	نمودار بودجه‌بندی پارامترها در معماری MiniGPT. بیش از ۹۸٪ از ظرفیت مدل در بلوک‌های ترنسفورمر متمرکز شده است.	۸۱
۲۱	نمودار زمان‌بندی نرخ یادگیری. بخش اول (شیب تند مثبت) مرحله Warmup و بخش دوم (منحنی نرم) مرحله Cosine Decay است.	۸۴
۲۲	نمودارهای تشخیصی: منحنی زبان (چپ)، نرم‌گرادیان (وسط) و توزیع وزن‌های نهایی (راست).	۸۸



۲۳	نقشه حرارتی توجه ۶ سر مختلف در بلوک اول. هر سر یک الگوی جستجوی خاص را در متن شکسپیر یاد گرفته است.	۹۰
۲۴	تصویرسازی بردارهای تعبیه کلمات در فضای ۲-بعدی با PCA. گروه‌بندی خودکار حروف صدا دار (قرمز)، بی صدا (سبز) و علائم نگارشی (آبی) بدون هیچ آموزش زبان‌شناختی.	۹۱
۲۵	خطای آموزش minigpt.	۱۰۳
۲۶	نمودار احتمالاتی توکن‌های بعد از کاراکتر: ROMEO.	۱۰۳
۲۷	نمودار تعداد کاراکترهای متمایز استفاده شده در متن براساس پارامتر Temperature.	۱۰۴
۲۸	مقایسه تعداد پارامترهای miniGPT با سایر مدل‌های مشهور.	۱۰۴
۲۹	مقایسه دورویکرد ساده و بهینه برای کاهش محاسبات تکراری.	۱۰۵
۳۰	نمودار بررسی عملکرد فیلترهای Top-p Top-k و حالت بدون فیلتر.	۱۰۵



۱ اسنپ - مبانی و توکن سازی

۱.۱ مرور کلی تمرین

این تمرین آغاز یک زنجیره پنج قسمتی برای ساخت یک مدل زبانی کوچک به سبک GPT در محیط JAX است. نقطه شروع تبدیل متن خام به تنسورهای عددی است؛ زیربنایی که تمام بخش‌های بعدی مدل بر آن بنا می‌شوند.

داستان اولیه

در این مرحله، تمرکز بر فهم دقیق فرآیند پایه‌ای مدل‌های زبانی است: تبدیل کاراکترها به اعداد. دلیل این کار کنترل کامل بر مدل و جلوگیری از تبدیل سیستم به یک جعبه سیاه در مراحل بعدی است. داده‌ی آموزشی شامل مجموعه آثار شکسپیر است که ساختاری کافی برای تمرین یک مدل کوچک فراهم می‌کند.

اهمیت این مرحله این بخش زیربنای کل تمرین است و خروجی آن یعنی تنسورهای صحیح شده‌ی کاراکتری، مستقیماً در بخش توجه (Attention) و سپس کل شبکه‌ی ترنسفورمر استفاده می‌شوند. بدون این پایه گذاری، بخش‌های بعدی قابل اتکا نخواهند بود.

ماموریت بخش اول

هدف‌های اصلی عبارت‌اند از:

- آشنایی با تفاوت‌های JAX و NumPy، از جمله عدم تغییر پذیری داده و تابع محور بودن.
 - پیاده سازی نسخه‌ی پایدار عددی Softmax.
 - ساخت توکنایزر کاراکتری از متن خام.
 - تولید داده‌های آموزشی به صورت جفت‌های ورودی-هدف برای پیش بینی توکن بعدی.
- این موارد پایه‌هایی هستند که در بخش‌های بعدی برای ساخت Attention، Multi-Head Attention، بلاک ترنسفورمر و در نهایت آموزش مدل استفاده خواهند شد.

معماری مورد نظر

مدلی که در این تمرین ساخته می‌شود نسخه‌ی ساده شده‌ای از ترنسفورمر مقاله‌ی Attention Is All You Need است که تنها ستون «دیکودر با Masked Self-Attention» را حفظ می‌کند. این معماری همان چیزی است که در مدل‌های GPT استفاده می‌شود: پیش بینی توکن بعدی با استفاده از تمام توکن‌های قبلی، بدون مشاهده آینده.



تعریف کوتاه GPT GPT یک مدل زبانی است که توزیع احتمالات توکن بعدی را بر اساس توکن‌های قبلی تعیین می‌کند. این مدل:

- تولیدگر است چون می‌تواند متن جدید تولید کند،
- پیش‌آموزش دیده است چون ابتدا روی یک مجموعه بزرگ الگوهای زبان را یاد می‌گیرد.
- ترنسفورمر-محور است چون تمرکز آن بر مکانیزم Self-Attention است.
- فقط دیکودر است چون بخش رمزگذار را حذف می‌کند و تنها اجازه نگاه به گذشته را دارد.

چرا JAX؟

JAX محیطی شبیه NumPy فراهم می‌کند اما سه قابلیت حیاتی برای ساخت مدل‌های یادگیری عمیق دارد:

- jax.grad برای محاسبه خودکار گرادیان.
- jax.jit برای اجرای بهینه‌شده با کامپایلر XLA.
- jax.vmap برای بردارسازی خودکار و ساخت Batch داده.

غیرقابل تغییر بودن آرایه‌ها و تابع محور بودن JAX امکان تکرارپذیری آزمایش‌ها و اجرای یکسان کد روی CPU، GPU و TPU را بدون تغییر فراهم می‌کند.

۲.۱ بارگذاری کتابخانه‌ها و آماده‌سازی محیط

در ابتدای کار، مجموعه‌ای از کتابخانه‌های موردنیاز برای پروژه وارد شدند. این کتابخانه‌ها شامل ابزارهای استاندارد پایتون برای کار با فایل‌ها، زمان، ریاضیات و همچنین کتابخانه‌های یادگیری ماشین موردنیاز برای ساخت مدل در محیط JAX هستند. توضیح بخش‌های اصلی کد به صورت زیر است:

- NumPy و Matplotlib: برای کار با آرایه‌ها و رسم نمودارها.
- JAX و jax.numpy: هسته اصلی محاسبات عددی و خودکارسازی گرادیان‌ها.
- Flax.linen: چارچوب ساخت لایه‌های مدل و معماری شبکه.
- Optax: کتابخانه‌ای برای بهینه‌سازی پارامترهای مدل.

در ادامه تنظیمات اولیه برای نمایش نمودارها از طریق Matplotlib اعمال شده است تا خروجی‌ها خواناتر باشند. همچنین دقت چاپ آرایه‌های NumPy تنظیم شده تا مقادیر با حداکثر چهار رقم اعشار و بدون نمایش نمایی چاپ شوند.

چاپ نسخه‌ها و دستگاه محاسباتی

در پایان، نسخه JAX و Flax همراه با لیست دستگاه‌هایی که JAX می‌تواند روی آن‌ها اجرا شود نمایش داده می‌شود. این مرحله کمک می‌کند تا محیط اجرای محاسبات تأیید شود و بدانیم که مدل روی CPU یا GPU در حال اجرا است.



خروجی کد خروجی چاپ شده از کد به صورت زیر است:

```
JAX 0.7.2 | Flax | devices: [CpuDevice(id=0)]
```

این خروجی نشان می دهد که:

- نسخه JAX برابر با 0.7.2 است.
 - نسخه Flax چاپ نشده که طبیعی است زیرا Flax نسخه تعبیه شده در linen را در برخی محیط ها نمایش نمی دهد.
 - اجرای برنامه روی یک CPU انجام می شود، یعنی هنوز GPU یا TPU در دسترس نیست.
- این اطلاعات برای شروع توسعه مدل حیاتی است، زیرا مشخص می کند بهینه سازی و محاسبات در چه سخت افزاری انجام خواهد شد.

۳.۱ بررسی قابلیت های کلیدی JAX: grad، jit و vmap

پیش از ورود به معماری ترنسفورمر، لازم است سه ستون اصلی JAX که تفاوت آن را با کتابخانه های دیگر رقم می زنند، در عمل آزمایش شوند. این سه قابلیت اجازه می دهند کدهای پایتونی به برنامه هایی سریع، مشتق پذیر و برداری تبدیل شوند.

محاسبه خودکار مشتق (grad)

در این آزمایش، مشتق تابع $f(x) = x^3$ در نقطه $x = 2$ محاسبه شده است. مشتق ریاضی این تابع برابر با $3x^2$ است که در نقطه ۲ مقدار ۱۲ را نشان می دهد. JAX بدون نیاز به فرمول های دستی و تنها با استفاده از `jax.grad`، مقدار دقیق را محاسبه می کند.

خروجی کد:

```
grad of x^3 at x=2: 12.0 (exact 3*2^2 = 12)
```

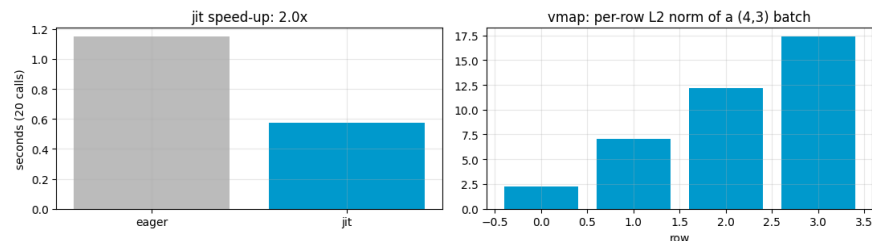
کامپایل سریع (jit)

قابلیت Just-In-Time (jit)، تابع را با استفاده از کامپایلر XLA بهینه سازی می کند. در کد بالا، یک تابع سنگین (heavy) که شامل ۶۰ تکرار محاسباتی است، تعریف شده است. تفاوت سرعت بین اجرای معمولی (eager) و اجرای کامپایل شده (jit) اندازه گیری شد.

تحلیل عملکرد: همان طور که در نمودار خروجی مشاهده می شود، نسخه jit حدود ۲ برابر سریع تر از نسخه معمولی عمل کرده است. این بهینه سازی با ترکیب عملیات ها (Operator Fusion) و کاهش سربار فراخوانی های پایتون به دست می آید.

بردارسازی خودکار (vmap)

قابلیت vmap به ما اجازه می‌دهد تابعی که برای یک بردار واحد نوشته شده است را بدون استفاده از حلقه‌های for صریح، روی یک Batch از داده‌ها اعمال کنیم. در این مثال، نرم L_2 برای ردیف‌های یک ماتریس $(4, 3)$ محاسبه شده است.



شکل ۱: مقایسه سرعت اجرای JIT و نمایش خروجی VMAP برای محاسبه نرم ردیف‌ها

تحلیل نهایی خروجی‌های تصویری

نمودارهای تولید شده (شکل ۱) به خوبی مفاهیم بالا را تایید می‌کنند:

- نمودار سمت چپ (JIT Speed-up): کاهش زمان اجرای ۲۰ فراخوانی را نشان می‌دهد. در حالی که حالت eager بیش از ۱.۱ ثانیه زمان برده، حالت jit در حدود ۰.۶ ثانیه به پایان رسیده است.
- نمودار سمت راست (vmap): نرم محاسبه شده برای هر یک از ۴ ردیف ماتریس ورودی را نشان می‌دهد. مقادیر به دست آمده نشان‌دهنده دقت اجرای عملیات برداری روی دسته‌های داده (batching) است.

۴.۱ پیاده‌سازی Softmax پایدار عددی

تابع Softmax وظیفه تبدیل برداری از امتیازات خام (scores) به یک توزیع احتمالی را بر عهده دارد. با این حال، پیاده‌سازی مستقیم آن به دلیل ماهیت نمایی (Exponential)، با اعداد بزرگ دچار سرریز (overflow) می‌شود. برای رفع این مشکل از یک ترفند ریاضی استفاده می‌کنیم: کم کردن مقدار ماکزیمم از تمام درایه‌ها. این کار باعث می‌شود بزرگترین توان، صفر شود ($e^0 = 1$) و مقادیر دیگر همگی کوچک‌تر از یک شوند، که از سرریز جلوگیری می‌کند.

کد پیاده‌سازی

در قطعه کد زیر، با استفاده از `keepdims=True`، امکان اعمال عملیات به صورت دسته‌ای (batching) فراهم شده است تا کد هم برای یک بردار و هم برای ماتریسی از داده‌ها کار کند:

```
1 def stable_softmax(z, axis=-1):
2
3     max_z = jnp.max(z, axis=axis, keepdims=True)
4     shifted_z = z - max_z
5
6
```

```

7  ez = jnp.exp(shifted_z)
8  sum_ez = jnp.sum(ez, axis=axis, keepdims=True)
9  return ez / sum_ez
10

```

تأیید صحت و تحلیل دما (Temperature)

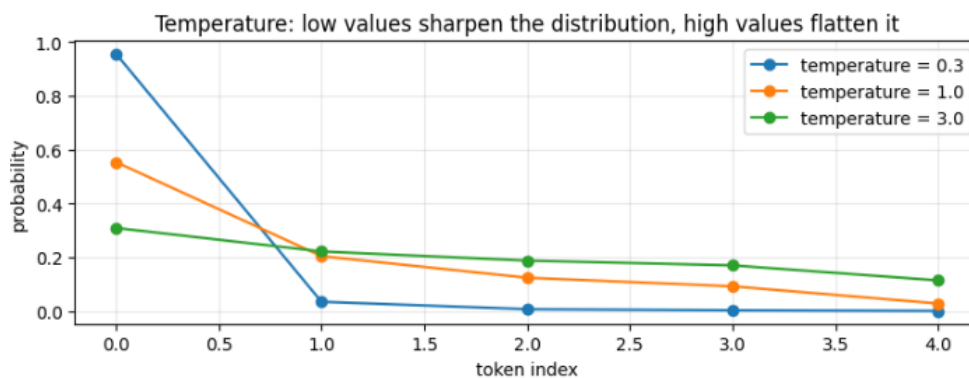
برای اطمینان از عملکرد صحیح، تابعی نوشته شد که ورودی‌های بسیار بزرگ (که در حالت عادی باعث inf می‌شوند) را پردازش کرده و نتیجه را با پیاده‌سازی استاندارد `jax.nn.softmax` مقایسه می‌کند. خروجی تست:

OK: `stable_softmax` matches `jax.nn.softmax` and survives huge inputs

این نتیجه نشان می‌دهد که پیاده‌سازی ما علاوه بر سازگاری با توابع داخلی JAX، در مواجهه با مقادیر بحرانی نیز پایداری خود را حفظ کرده است.

تحلیل اثر دما (Temperature): پارامتر دما (T) بر شکل توزیع تأثیر می‌گذارد:

- دماهای پایین (مثلاً 0.3): توزیع را تیزتر (sharpen) کرده و احتمال توکن‌های برتر را بسیار بالا می‌برند.
- دماهای بالا (مثلاً 3.0): توزیع را هموارتر (flatten) کرده و احتمال‌ها را به سمت یکنواختی می‌برند.



شکل ۲: اثر تغییر پارامتر دما بر توزیع احتمالات Softmax

همان‌طور که در نمودار (شکل ۲) دیده می‌شود، با افزایش دما، اختلاف احتمالات بین توکن‌ها کمتر شده و منحنی به سمت یک خط صاف میل می‌کند.



۵.۱ بارگذاری و تحلیل مقدماتی پیکره متنی شکسپیر

قبل از شروع آموزش مدل، بررسی اولیه داده‌ها امری حیاتی است تا از کیفیت و ساختار پیکره متنی اطمینان حاصل شود. در این مرحله، از مجموعه آثار کامل شکسپیر استفاده شده است که یک استاندارد کلاسیک در یادگیری ماشین برای مدل‌های زبانی است.

فرآیند بارگذاری داده‌ها

کد پیاده‌سازی شده ابتدا سعی می‌کند فایل tiny_shakespeare.txt را به صورت محلی پیدا کند. در صورتی که فایل وجود نداشته باشد، آن را به طور خودکار از مخزن گیت‌هاب مربوطه دانلود می‌کند. این رویکرد باعث تکرارپذیری آزمایش می‌شود.

```
1 def load_tiny_shakespeare():
2     for path in ("data/tiny_shakespeare.txt", "tiny_shakespeare.txt", "../data/tiny_shakespeare.txt"):
3         if os.path.exists(path):
4             return open(path, encoding="utf-8").read()
5     url = "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt"
6     os.makedirs("data", exist_ok=True)
7     urllib.request.urlretrieve(url, "data/tiny_shakespeare.txt")
8     return open("data/tiny_shakespeare.txt", encoding="utf-8").read()
9
10 text = load_tiny_shakespeare()
11 print(f"Corpus length: {len(text):,} characters")
12 print(repr(text[:180]))
13
14 fig, ax = plt.subplots(1, 2, figsize=(12, 3.4))
15 from collections import Counter
16 counts = Counter(text)
17 common = counts.most_common(20)
18 ax[0].bar([repr(c)[1:-1] for c, _ in common], [n for _, n in common], color="#0099cc")
19 ax[0].set_title("20 most frequent characters"); ax[0].tick_params(axis="x", rotation=60)
20 line_lengths = [len(line) for line in text.split("\n")]
21 ax[1].hist(line_lengths, bins=40, color="#0099cc", edgecolor="black")
22 ax[1].set_title("Distribution of line lengths"); ax[1].set_xlabel("characters per line")
23 plt.tight_layout(); plt.show()
24
```

تحلیل خروجی متنی

پس از بارگذاری، طول پیکره نمایش داده شد تا مقیاس محاسباتی مدل مشخص شود. خروجی متنی به شرح زیر است:

Corpus length: 1,115,394 characters

'First Citizen:\nBefore we proceed any further, hear me speak.\n\nAll:\nSpeak, speak.

\n\nFirst Citizen:\nYou are all resolved rather to die than to famish?\n\nAll:\nResolved.
resolved.\n\nFirst'

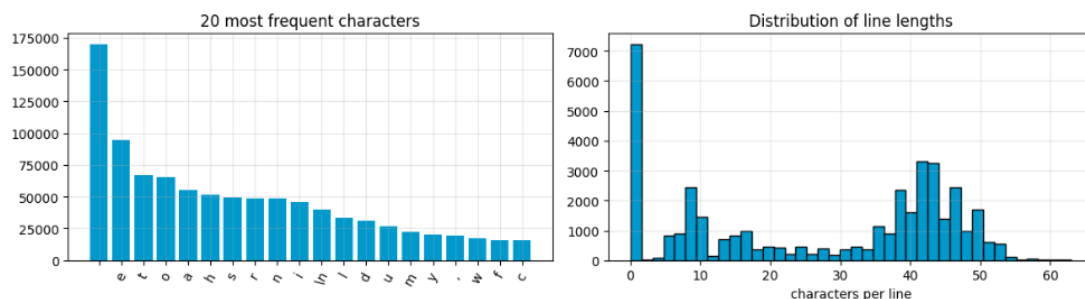
تفسیر نتایج متنی:



- حجم داده: حدود ۱.۱ میلیون کاراکتر. این حجم برای آموزش یک مدل زبانی کوچک در یک محیط آموزشی بسیار مناسب است، زیرا به اندازه کافی غنی است تا الگوهای زبانی را یاد بگیرد و به اندازه کافی کوچک است تا در سخت افزارهای معمولی به سرعت آموزش ببیند.
- فرمت: نمونه متن نمایش داده شده نشان می دهد که داده ها شامل نام شخصیت ها (مانند First Citizen)، کاراکترهای جداکننده مانند newline (\n) و دیالوگ ها هستند. این یعنی مدل ما باید علاوه بر کلمات، نحوه نگارش ساختار درام نویسی را نیز یاد بگیرد.

تحلیل نمودارهای توزیع (EDA)

دو نمودار برای شناخت بهتر داده ها ترسیم شدند:



شکل ۳: فراوانی کاراکترها و توزیع طول خطوط در مجموعه داده شکسپیر

تفسیر نمودارها:

- فراوانی کاراکترها (نمودار سمت چپ): همان طور که انتظار می رفت، کاراکتر space بیشترین تکرار را دارد. حروف رایج در زبان انگلیسی مانند 'e'، 't' و 'o' نیز در رتبه های اول قرار دارند. این توزیع به ما اطمینان می دهد که توکنایزر کاراکتری ما با داده های متعادلی روبرو خواهد شد.
 - توزیع طول خطوط (نمودار سمت راست): این نمودار یک توزیع دووجهی (Bimodal) را نشان می دهد:
 - پیک اول نزدیک به صفر مربوط به خطوط خالی و نام شخصیت ها است.
 - پیک دوم در حدود ۴۰ تا ۵۰ کاراکتر نشان دهنده طول دیالوگ های استاندارد در نمایشنامه است.
- این تحلیل ها به ما کمک می کند تا درک کنیم مدل با چه ساختار متنی (از نظر طول و توزیع کاراکترها) درگیر خواهد بود.

۶.۱ ساخت توکنایزر کاراکتری

شبکه های عصبی تنها قادر به پردازش اعداد هستند. بنابراین، گام اول در هر مدل زبانی، تبدیل متن به تانسورهای عددی است. در حالی که مدل های بزرگ صنعتی از روش های پیشرفته مانند Byte-Pair Encoding (BPE) برای مدیریت کلمات استفاده می کنند، ما در این تمرین از توکنایزر کاراکتری استفاده می کنیم. این رویکرد به دلیل سادگی، شفافیت و نیاز نداشتن به کتابخانه های جانبی، برای یادگیری مکانیسم های زیرین ترنسفورمر بسیار مناسب است.



پایاده‌سازی توکنایزر

هدف اصلی ایجاد یک نگاشت دوطرفه بین کاراکترها و شناسه‌های عددی (Integer IDs) است. برای تضمین رفتار قطعی (Deterministic) مدل، لیست کاراکترها را ابتدا مرتب کرده‌ایم.

```

1  chars = sorted(list(set(text)))
2  vocab_size = len(chars)
3
4
5  stoi = {ch:i for i,ch in enumerate(chars)}
6  itos = {i:ch for i,ch in enumerate(chars)}
7
8
9  def encode(s):
10     return [stoi[c] for c in s]
11
12  def decode(ids):
13     return ''.join(itos[int(i)] for i in ids)
14
15
16  data = np.array(encode(text), dtype=np.int32)
17

```

ارزیابی و تأیید صحت (Self-Check)

برای اطمینان از صحت عملکرد نگاشت، کدهای زیر را اجرا کردیم که بازگشت‌پذیر بودن (Round-trip) تبدیل‌ها را آزمایش می‌کند:

```

1  assert decode(encode("Snapp loves JAX!")) == "Snapp loves JAX!"
2  assert vocab_size == len(set(text)) and len(stoi) == vocab_size == len(itos)
3  assert data.dtype == np.int32 and len(data) == len(text)
4
5  print("OK: vocab_size =", vocab_size)
6  print("first 20 chars:", repr(text[:20]))
7  print("first 20 ids  :", data[:20].tolist())
8

```

تحلیل خروجی‌ها

خروجی اجرای کد در کنسول به صورت زیر است:

```

OK: vocab_size = 65
first 20 chars: 'First Citizen:\nBefor'
first 20 ids  : [18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 14, 43, 44, 53, 56]

```

تحلیل نتایج:

- Vocab Size: عدد ۶۵ نشان می‌دهد که زبان انگلیسی استفاده شده در آثار شکسپیر، تنها شامل ۶۵ کاراکتر یکتا (شامل حروف بزرگ/کوچک، علائم نگارشی و کاراکتر فاصله) است. این دامنه کوچک، مدل را از پیچیدگی‌های توکن‌های زیرکلمه‌ای (که ده‌ها هزار توکن دارند) دور نگه می‌دارد.
- نگاشت (Mapping): خروجی first 20 ids نمایش‌دهنده نحوه تبدیل متن به تانسورهای int32 است. برای مثال، کاراکتر 'F' (اندیس ۱۸ در لیست ما) به عدد ۱۸ تبدیل شده است.
- بازگشت‌پذیری: با دستور assert، اطمینان حاصل کردیم که اگر متنی را انکود کنیم، حتماً با دکود کردن آن به متن اصلی برمی‌گردیم؛ این موضوع برای عملکرد مدل در زمان Inference (تولید متن) حیاتی است.

۷.۱ تقسیم داده به مجموعه آموزش و اعتبارسنجی

برای ارزیابی صحیح عملکرد مدل و تشخیص بیش‌برازش (Overfitting)، پیکره متنی به دو بخش آموزش و اعتبارسنجی تقسیم می‌شود. در این تمرین، ۹۰٪ ابتدایی متن برای آموزش و ۱۰٪ انتهایی برای اعتبارسنجی در نظر گرفته شده است. این نوع تقسیم، یک روش ساده و مناسب برای داده‌های ترتیبی (Sequential Data) است، زیرا ترتیب طبیعی متن حفظ می‌شود و از نشت اطلاعات از آینده به گذشته جلوگیری می‌گردد.

پیاده‌سازی تقسیم داده

در کد زیر، ابتدا نقطه جداسازی بر اساس ۹۰ درصد طول کل داده محاسبه شده و سپس آرایه اصلی به دو بخش مجزا تقسیم می‌شود:

```
1 n = int(0.9 * len(data))
2 train_data, val_data = data[:n], data[n:]
3 print(f"train: {len(train_data):,} tokens | val: {len(val_data):,} tokens")
4
5 fig, ax = plt.subplots(figsize=(10, 1.4))
6 ax.barh([0], [len(train_data)], color="#0099cc", label="train (90%)")
7 ax.barh([0], [len(val_data)], left=[len(train_data)], color="#ffb347", label="val (10%)")
8 ax.set_yticks([]); ax.set_xlabel("character position in corpus"); ax.legend(loc="upper right")
9 ax.set_title("Train / validation split"); plt.tight_layout(); plt.show()
10
```

خروجی عددی

پس از اجرای کد، اندازه دو بخش به صورت زیر گزارش شد:

train: 1,003,854 tokens | val: 111,540 tokens

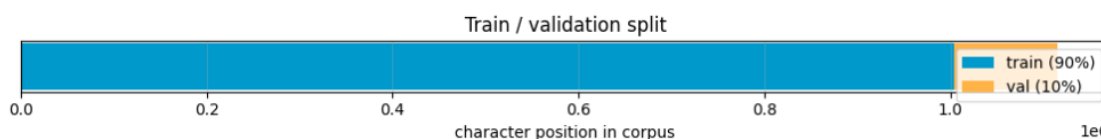
تحلیل خروجی عددی:

- بخش آموزش: شامل ۱٬۰۰۳٬۸۵۴ توکن است که معادل ۹۰ درصد کل پیکره می‌باشد.

- بخش اعتبارسنجی: شامل ۱۱۱'۵۴۰ توکن است که معادل ۱۰ درصد باقی مانده است.
- جمع کل: مجموع این دو مقدار برابر با طول کل داده‌ها است و نشان می‌دهد که تقسیم‌بندی بدون از دست رفتن یا تکرار داده انجام شده است.

تحلیل نمودار

نمودار پایین یک نوار افقی انباشته (Horizontal Stacked Bar) را نشان می‌دهد که کل پیکره را بر حسب موقعیت کاراکترها در متن به دو بخش تقسیم می‌کند:



شکل ۴: تقسیم پیکره به بخش‌های آموزش و اعتبارسنجی

- بخش آبی با برچسب train (90%) نشان‌دهنده داده‌های آموزشی است.
 - بخش نارنجی با برچسب val (10%) نشان‌دهنده داده‌های اعتبارسنجی است.
- تفسیر بصری نمودار:
- محور افقی با برچسب character position in corpus نمایش داده شده و مقیاس آن با ضریب علمی $1e6$ مشخص شده است؛ بنابراین اعداد 0.0 تا 1.0 روی محور، در واقع به میلیون‌ها کاراکتر اشاره دارند.
 - نوار آبی تقریباً تا ابتدای عدد 1.0 میلیون کاراکتر امتداد دارد و بیانگر ۹۰٪ نخست داده است.
 - بخش نارنجی بلافاصله پس از آن قرار گرفته و تا حدود 1.1 میلیون کاراکتر ادامه می‌یابد که همان ۱۰٪ پایانی داده است.
- چرا این نوع تقسیم مهم است؟
- پیشگیری از نشت اطلاعات: در داده‌های متنی ترتیبی، اگر داده‌ها به صورت تصادفی مخلوط شوند، ممکن است اطلاعات آینده به آموزش وارد شود. تقسیم ترتیبی این مشکل را برطرف می‌کند.
 - تشخیص بیش‌برازش: با نگاه داشتن بخشی از داده‌ها برای اعتبارسنجی، می‌توان بررسی کرد که آیا مدل فقط متن آموزش را حفظ کرده یا واقعاً الگوهای زبانی را یاد گرفته است.
 - استاندارد بودن: این نوع split یک روش رایج در آموزش مدل‌های زبانی کاراکتری و توالی‌محور است.

۸.۱ هدف پیش‌بینی توکن بعدی و ساخت مینی‌بچ‌ها

اساس آموزش مدل‌های ترنسفورمر تولیدگر (مانند سری GPT)، حل یک مسئله ساده اما بسیار کارآمد است: پیش‌بینی توکن بعدی. مدل با دیدن توالی کاراکترهای گذشته، باید احتمال کاراکتر بعدی را پیشنهاد کند. این ویژگی باعث می‌شود که مدل به صورت خودبرگشت‌دهنده (Autoregressive) رفتار کرده و قادر به تولید متن‌های طولانی و منسجم باشد.



مفهوم پیش‌بینی توکن بعدی (Next-Token Prediction)

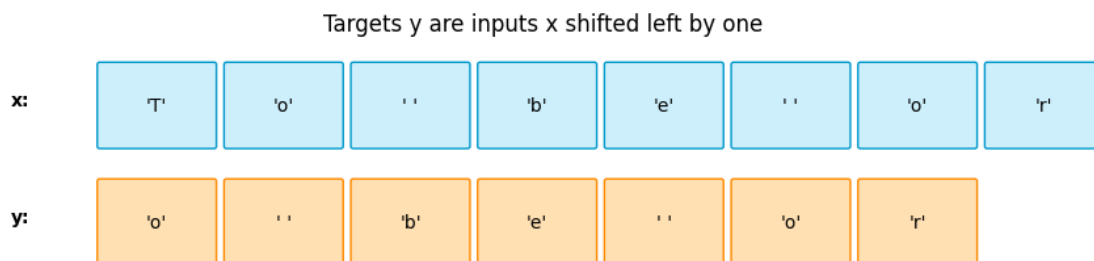
در این فاز، کل پیکره متنی به زیرتوالی‌هایی با طول مشخص به نام `block_size` (یا پنجره محتوا - Context Window) تقسیم می‌شود. برای هر قطعه از ورودی (x)، متناظر با آن یک قطعه هدف (y) تعریف می‌شود که دقیقاً همان توالی است، اما به اندازه یک گام به سمت چپ جابجا شده است.

این طراحی هوشمندانه به ما اجازه می‌دهد که از یک پنجره به طول N ، تعداد N مسئله پیش‌بینی مستقل استخراج کنیم:

- در موقعیت زمان $t = 0$ ، مدل کاراکتر $x[0]$ را می‌بیند و باید هدف $y[0] = x[1]$ را پیش‌بینی کند.
- در موقعیت زمان $t = 1$ ، مدل کاراکترهای $x[0, 1]$ را می‌بیند و باید هدف $y[1] = x[2]$ را پیش‌بینی کند.
- به طور کلی در هر گام t ، پیش‌بینی بر اساس توالی گذشته $x[:t+1]$ انجام می‌شود و مدل حق دیدن آینده را ندارد. این محدودیت با استفاده از توجه علی (Causal Attention) در ساختار ترنسفورمر اعمال خواهد شد.

تحلیل بصری رابطه جابجایی (Shift Relationship)

برای درک ملموس‌تر این ساختار، نمونه کوتاه متنی "To be or" تصویرسازی شده است:



شکل ۵: رابطه جابجایی یک گام به چپ میان ورودی x و هدف y برای عبارت "To be or"

تفسیر نمودار: همان‌طور که در شکل ۵ مشخص است:

- رشته ورودی (x): شامل کاراکترهای ['T', 'o', ' ', 'b', 'e', ' ', 'o', 'r'] در قالب جعبه‌های آبی‌رنگ است.
- رشته هدف (y): شامل کاراکترهای ['o', ' ', 'b', 'e', ' ', 'o', 'r'] در قالب جعبه‌های نارنجی‌رنگ است که نسبت به x یک واحد به چپ شیفت پیدا کرده‌اند.
- برای مثال، ورودی اولین خانه در x برابر با 'T' است و مدل باید یاد بگیرد که توکن بعدی یعنی 'o' (اولین خانه در y) را پیش‌بینی کند. اگر ورودی تا اندیس دوم یعنی "To" باشد، هدف بعدی کاراکتر 'b' خواهد بود.

پیاده‌سازی تابع تولید بچ (get_batch) در JAX

در JAX، برخلاف فریمورک‌هایی مثل PyTorch، مدیریت حالت‌های تصادفی با استفاده از کلیدهای صریح (PRNGKey) انجام می‌شود تا محاسبات کاملاً خالص (Pure) و تکرارپذیر باشند. تابع `get_batch` وظیفه دارد به صورت تصادفی اندیس‌های شروع را انتخاب کرده

و بجهایی از جفت‌های ورودی-هدف بسازد:

```

1  import jax
2  import jax.numpy as jnp
3  import numpy as np
4
5  def get_batch(key, source, batch_size, block_size):
6
7      ix_key, key = jax.random.split(key)
8      ix = jax.random.randint(ix_key, (batch_size,), 0, len(source) - block_size - 1)
9
10
11     ix_np = np.asarray(ix)
12
13
14     x = jnp.stack([source[i : i + block_size] for i in ix_np])
15     y = jnp.stack([source[i + 1 : i + 1 + block_size] for i in ix_np])
16
17     return x, y
18

```

تفسیر فنی پیاده‌سازی:

- تابع `jax.random.randint`: تولید اندیس‌ها با استفاده از معماری تولید عدد تصادفی بدون حالت (Stateless) در JAX صورت می‌گیرد. هر بار که این تابع صدا زده می‌شود، باید یک کلید منحصر به فرد (key) به آن پاس داده شود.
- محدوده مجاز اندیس‌ها: بازه انتخاب اندیس‌ها بین 0 تا $L - N - 1$ است (که در آن L طول کل آرایه داده و N برابر با `block_size` است). این کار تضمین می‌کند که اسلایس $x[i : i + N]$ و اسلایس شیفت‌یافته $y[i + 1 : i + 1 + N]$ از مرز انتهای آرایه فراتر نروند.
- شکل داده خروجی: تنسورهای خروجی دارای ابعاد (batch_size, block_size) خواهند بود. این ساختار دو بعدی کارآمد، به طور همزمان فرآیند آموزش را روی کل بچ و در طول کل پنجره محتوا به موازات پیش می‌برد.

۹.۱ ارزیابی عملکرد تولید بچ‌ها (Batch Generation)

پس از پیاده‌سازی تابع `get_batch`، برای اطمینان از اینکه داده‌ها به درستی و طبق منطق «پیش‌بینی توکن بعدی» تولید می‌شوند، یک مرحله ارزیابی دقیق اجرا کردیم. این بررسی شامل صحت‌سنجی انطباق ورودی-هدف و مشاهده بصری یک بچ واقعی از داده‌هاست.

کد ارزیابی (Self-Check)

کد زیر با استفاده از یک بذر تصادفی ثابت (PRNGKey)، یک بچ کوچک ایجاد کرده و زنجیره پیش‌بینی‌ها را گام‌به‌گام چاپ می‌کند تا صحت انتقال اطلاعات بررسی شود:

```

1  xb, yb = get_batch(jax.random.PRNGKey(1), train_data, 4, 8)
2
3
4  assert xb.shape == yb.shape == (4, 8)
5

```



```

6
7 print("context x[0]:", repr(decode(xb[0])))
8 print("target y[0]:", repr(decode(yb[0])))
9
10
11 for t in range(8):
12     print(f" given {repr(decode(xb[0][:t+1])):<12} -> predict {repr(decode([yb[0][t]]))}")
13

```

خروجی نتایج و تحلیل منطق پیش‌بینی

خروجی کنسول به شرح زیر است:

```

context x[0]: 'the ice,'
target y[0]: 'he ice,\n'
given 't'          -> predict 'h'
given 'th'         -> predict 'e'
given 'the'        -> predict ' '
given 'the '       -> predict 'i'
given 'the i'      -> predict 'c'
given 'the ic'     -> predict 'e'
given 'the ice'    -> predict ','
given 'the ice,'   -> predict '\n'

```

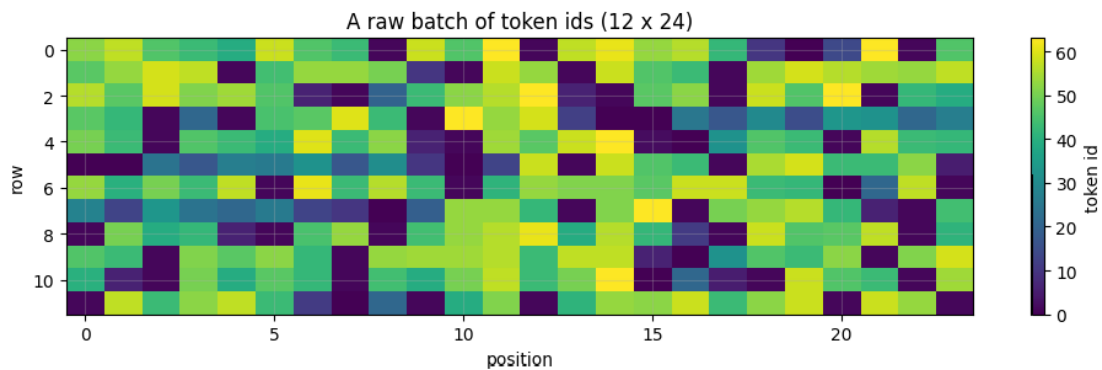
تفسیر نتایج: همان‌طور که مشاهده می‌شود، برای هر ورودی (given)، مدل باید کاراکتر بعدی (predict) را پیش‌بینی کند. دقت کنید که در گام آخر، مدل باید کاراکتر `\n` را پیش‌بینی کند، که نشان می‌دهد تابع `get_batch` به‌خوبی توالی را به درستی شیفت داده است. این رابطه ۱ به ۱ دقیقاً همان چیزی است که برای آموزش مدل نیاز داریم.

تحلیل بصری بچ داده‌ها

برای درک بهتر توزیع شناسه‌های توکن در یک بچ بزرگتر (۱۲ ردیف در ۲۴ ستون)، نمودار حرارتی (Heatmap) زیر رسم شد:

تفسیر نمودار حرارتی:

- محورها: محور افقی نشان‌دهنده موقعیت در طول پنجره (position) و محور عمودی نشان‌دهنده ردیف‌های بچ (row) است.
- رنگ‌ها: رنگ‌ها نشان‌دهنده token id هستند. تنوع رنگی بالا در نمودار (از بنفش تیره تا زرد روشن) نشان می‌دهد که توکن‌ها به‌صورت تصادفی و در سراسر دامنه واژگان (۶۵ کاراکتر) توزیع شده‌اند.



شکل ۶: نمودار حرارتی (Heatmap) از یک بچ نمونه شامل ۱۲ ردیف و ۲۴ ستون توکن

- کاربرد: این نمایش بصری به ما اطمینان می‌دهد که `get_batch` نمونه‌های متنوعی از سراسر پیکره متنی انتخاب کرده است و مدل در حین آموزش با داده‌های یکنواخت و تکراری مواجه نمی‌شود. این تنوع برای جلوگیری از بیش‌برازش و بهبود تعمیم‌پذیری مدل بسیار حیاتی است.

۱۰.۱ تحلیل آماری روابط کاراکترها (Bigram Transition Matrix)

برای درک اینکه مدل دقیقاً چه چیزی را باید «یاد بگیرد»، ما از مدل ساده بیگرام استفاده کردیم. مدل بیگرام فرض می‌کند احتمال کاراکتر بعدی فقط به کاراکتر فعلی بستگی دارد. این تحلیل، دیدی از ساختار آماری داده‌های متنی به ما می‌دهد.

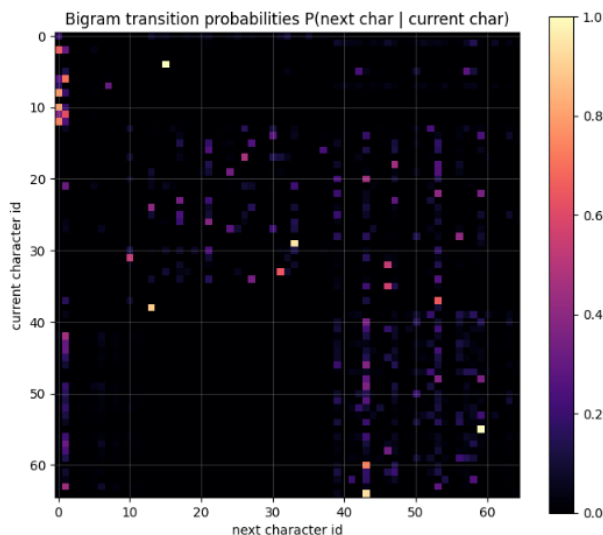
کد پیاده‌سازی و استخراج احتمالات

کد زیر یک ماتریس 65×65 ایجاد می‌کند که فراوانی هر کاراکتر نسبت به کاراکتر قبلی خود را شمارش کرده و سپس آن را نرمال می‌کند:

```
1 bigram = np.zeros((vocab_size, vocab_size))
2 ids = data[:200000]
3 np.add.at(bigram, (ids[:-1], ids[1:]), 1)
4 bigram_norm = bigram / (bigram.sum(axis=1, keepdims=True) + 1e-9)
5 fig, ax = plt.subplots(figsize=(7, 6))
6 im = ax.imshow(bigram_norm, cmap="magma")
7 ax.set_title("Bigram transition probabilities P(next char | current char)")
8 ax.set_xlabel("next character id"); ax.set_ylabel("current character id")
9 fig.colorbar(im, ax=ax); plt.tight_layout(); plt.show()
10 # a concrete row: what usually follows a space?
11 sp = stoi[" "]
12 top = np.argsort(-bigram_norm[sp, :])[:8]
13 print("after a space, most likely next chars:", [(itos[int(t)], round(float(bigram_norm[sp, t]), 3)) for t
14         in top])
```

۱۱.۱ تحلیل بصری (Heatmap)

تصویر حاصل (شکل ۷) توزیع احتمالات انتقال را نشان می‌دهد:



شکل ۷: ماتریس احتمالات انتقال بیگرام: محور عمودی کاراکتر فعلی و محور افقی کاراکتر بعدی را نشان می‌دهد.

تحلیل تصویر:

- نقاط روشن: نشان‌دهنده «قوانین» زبان هستند. جفت‌هایی که در زبان انگلیسی شکسپیر بسیار رایج‌اند (مثل ترکیب حروف در کلمات).
- پس‌زمینه سیاه: نشان‌دهنده ترکیب‌های ممنوع یا بسیار نادر است (احتمال نزدیک به صفر).
- ساختار مورب: نشان‌دهنده همبستگی محلی است که مدل ترنسفورمر در آینده با استفاده از مکانیزم Attention، آن‌ها را در مقیاس بسیار وسیع‌تری فرا خواهد گرفت.

۱۲.۱ بررسی یک نمونه: بعد از فضای خالی (Space)

برای درک ملموس، بررسی کردیم که بعد از کاراکتر فضای خالی (' ') چه کاراکترهایی بیشترین احتمال را دارند:

$(('t', 0.146), ('a', 0.08), ('h', 0.079), ('s', 0.066),$
 $('w', 0.065), ('m', 0.056), ('b', 0.048), ('o', 0.045))$

تفسیر آماری: خروجی بالا کاملاً با ساختار زبان انگلیسی همخوانی دارد. از آنجا که فضای خالی نشان‌دهنده شروع کلمه جدید است، احتمال بالایی وجود دارد که کلمه جدید با حروف پرکاربرد مثل 't' (شروع کلماتی مثل to, the) یا 'a' شروع شود. این تحلیل ثابت می‌کند که متن، تصادفی نیست و مدل با یادگیری این احتمالات می‌تواند «پیش‌بینی» کند.



۲ دیجیکالا - مکانیزم توجه

۱.۲ بخش دوم: پیاده‌سازی مکانیزم خودتوجهی (Self-Attention) در دیجی کالا

در این گام از توسعه مدل، سناریو به سمت تیم جست‌وجوی دیجی کالا و چالش پیش‌بینی عبارات در کادر جست‌وجوی بالای سایت متمایل می‌شود. مسئله اصلی، عبور از سیستم‌های سنتی تطابق پیشوند (Prefix Matching) و رسیدن به یک سیستم مبتنی بر بافتار (Context-Aware Autocomplete) است که بتواند تفاوت معنایی کلمات مشابه در بافت‌های مختلف (مانند تفاوت کلمه "case" بعد از "phone" با بعد از "court") را تشخیص دهد. کلید حل این مسئله، مکانیزم خودتوجهی (Self-Attention) است.

مأموریت و منطق شهودی مکانیزم توجه (The Soft Dictionary Lookup)

هسته اصلی این تمرین، شبیه‌سازی مکانیزم توجه به عنوان یک جست‌وجوی نرم در دیکشنری (Soft Dictionary Lookup) است. در یک دیکشنری معمولی (سخت)، ما یک کلید (Query) داریم، دقیقاً یک تطابق صریح پیدا می‌کنیم و مقدار (Value) مربوط به آن را می‌خوانیم. اما در مکانیزم توجه:

- هر موقعیت (توکن) یک پرس‌وجو (Query یا Q) صادر می‌کند.
- این پرس‌وجو با کلید (Key یا K) تمام توکن‌های دیگر مقایسه می‌شود.
- حاصل این مقایسه پس از عبور از لایه Softmax، وزن‌های توجه را می‌سازد.
- در نهایت، خروجی هر توکن از ترکیب خطی و نرم تمام مقادیر (Value یا V) متناسب با وزن‌های به دست آمده محاسبه می‌شود. این کار اجازه می‌دهد اطلاعات توکن‌های دورتر (مثلاً کلمه phone) بر روی توکن فعلی اثر بگذارد.

اجزای اصلی پیاده‌سازی در این تمرین

این تمرین عملی بر روی پنج ستون اصلی از ساختار لایه خودتوجهی تک‌سر (Single-Head Attention) تمرکز دارد:

۱. مخلوط کردن اطلاعات بین توکن‌ها (Information Mixing): توکن‌ها در ابتدای ورود به مدل، بردارهایی ایزوله هستند. مکانیزم توجه وظیفه دارد این بردارهای مستقل را بر اساس میزان ارتباط معنایی‌شان با یکدیگر ترکیب کند. برخلاف شبکه پیچشی (CNN) که محدوده دید محلی و ثابتی دارد، یا شبکه‌های بازگشتی (RNN) که پردازش متوالی و کندی دارند، خودتوجهی امکان ارتباط مستقیم توکن ابتدا و انتهای یک متن طولانی را در یک گام محاسباتی فراهم می‌سازد که برای اجرا روی شتاب‌دهنده‌ها با استفاده از کامپایل درجا در جکس (jax.jit) فوق‌العاده بهینه است.

۲. فرمول توجه ضرب نقطه‌ای مقیاس‌گذاری شده (Scaled Dot-Product): رابطه ریاضی زیر اساس محاسبات این بخش است:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_{\text{head}}}} \right) V$$

یکی از نکات کلیدی این تمرین، درک علت تقسیم بر $\sqrt{d_{\text{head}}}$ است. اگر ابعاد بردارها (d_{head}) بزرگ باشد، ضرب نقطه‌ای QK^T مقادیر بسیار بزرگی تولید می‌کند. این مقادیر بزرگ، تابع Softmax را به نواحی اشباع (نزدیک به صفر و یک) سوق می‌دهند که باعث ناپدید

شدن گرادینانها (Vanishing Gradients) می شود. تقسیم بر جذر ابعاد، واریانس خروجی را به حدود 1 باز می گرداند و پایداری آموزش را تضمین می کند.

۳. ماسک علی (The Causal Mask): برای مدل های خودهمبسته تولیدی (Autoregressive/Decoder-only) مانند GPT، مدل نباید اجازه داشته باشد در حین آموزش به توکن های آینده نگاه کند (پدیده Peeking at the future). این محدودیت با اعمال یک ماتریس پایین مثلثی انجام می شود. پیش از اعمال لایه Softmax، وزن های مربوط به موقعیت های آینده را با یک مقدار منفی بسیار بزرگ (مثلاً $-\infty$ یا $-1e9$) جایگزین می کنیم تا پس از Softmax سهم آن ها دقیقاً صفر شود:

$$M_{i,j} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

۴. کدگذاری موقعیتی سینوسی (Sinusoidal Positional Encoding): از آنجا که مکانیزم توجه نسبت به ترتیب توکن ها متقارن یا جابه جایی پذیر (Permutation-Equivariant) است (یعنی جابه جا کردن ترتیب ورودی ها صرفاً ترتیب خروجی ها را جابه جا می کند بدون اینکه ماهیت محاسبات تغییر کند)، باید اطلاعات موقعیت قرارگیری هر توکن را به صورت صریح به آن تزریق کنیم. در این بخش از فرکانس های سینوسی و کسینوسی با دوره های تناوب متفاوت استفاده می شود تا شناسه ای منحصر به فرد، کران دار بین $[-1, 1]$ و مستقل از طول توالی ایجاد شده به بردار امبدینگ توکن ها اضافه گردد:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

اهداف خروجی تمرین

در پایان این سناریو، هدف پیاده سازی و صحت سنجی یک لایه خودتوجهی تک سر (Single-Head Attention) کاملاً کارآمد در JAX است که ورودی های سه بعدی به فرمت $[batch_size, seq_len, d_model]$ را دریافت کرده و پس از اعمال تصویرسازی های خطی، مقیاس گذاری ضرب نقطه ای، ماسک علی و تزریق بردارهای موقعیت، خروجی اصلاح شده و بافت دار را محاسبه کند. این لایه سنگ بنای اصلی بلوک های Transformer بزرگتر در بخش های بعدی خواهد بود.

۲.۲ Scaled Dot-Product Attention و ماسک علی

در این بخش، هدف پیاده سازی هسته اصلی مکانیزم توجه در ترنسفورمرها یعنی تابع `scaled_dot_product_attention` و همچنین تولید ماسک علی با استفاده از تابع `causal_mask` است. این قسمت مهم ترین بخش دفترچه محسوب می شود زیرا منطق اصلی Self-Attention، یعنی محاسبه امتیازهای توجه، مقیاس گذاری آن ها، اعمال ماسک زمانی و تولید وزن های نهایی توجه.

توضیح سازوکار توجه

فرمول اصلی توجه به صورت زیر تعریف می شود:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{head}}}}\right)V$$



در این فرمول:

- ابتدا شباهت بین Query و Key با استفاده از ضرب ماتریسی QK^T محاسبه می‌شود.
- سپس این مقادیر بر $\sqrt{d_{\text{head}}}$ تقسیم می‌شوند تا از اشباع شدن تابع softmax و در نتیجه صفر شدن گرادیان‌ها جلوگیری شود.
- اگر ماسک علی وجود داشته باشد، امتیازهای مربوط به موقعیت‌های آینده با یک مقدار بسیار بزرگ منفی جایگزین می‌شوند تا وزن آن‌ها بعد از اعمال تابع softmax دقیقاً به صفر میل کند.
- در نهایت، وزن‌های توجه به ماتریس Value ضرب می‌شوند تا خروجی نهایی حاصل شود.

کد پیاده‌سازی توابع `causal_mask` و `scaled_dot_product_attention`

```

1  def causal_mask(T):
2      # lower triangular matrix
3      mask = jnp.tril(jnp.ones((T, T), dtype=bool))
4      return mask
5
6
7  def scaled_dot_product_attention(q, k, v, mask=None):
8
9      # dimension of head
10     d_head = q.shape[-1]
11
12     # compute attention scores
13     scores = jnp.matmul(q, jnp.swapaxes(k, -2, -1))
14
15     # scale
16     scores = scores / jnp.sqrt(d_head)
17
18     # apply mask if provided
19     if mask is not None:
20         scores = jnp.where(mask, scores, -1e9)
21
22     # softmax to get attention weights
23     attn = jax.nn.softmax(scores, axis=-1)
24
25     # weighted sum of values
26     out = jnp.matmul(attn, v)
27
28     return out, attn
29

```

توضیح کد:

- تابع `causal_mask` با استفاده از دستور `jnp.tril` یک ماتریس بولی پایین‌مثلثی به ابعاد $T \times T$ تولید می‌کند تا دسترسی و نگاه به آینده در توالی غیرمجاز شود.
- در تابع `scaled_dot_product_attention`:

- بعد ویژگی‌های هر هد یا همان d_{head} از روی آخرین محور متغیر q استخراج می‌شود.
- امتیازهای توجه با ضرب ماتریسی دسته‌ای بین q و ترانهاده تعویض شده k با استفاده از `jnp.swapaxes` محاسبه می‌گردد.
- امتیازها برای حفظ پایداری گرادین بر $\sqrt{d_{\text{head}}}$ تقسیم می‌شوند.
- در صورت فعال بودن ماسک، موقعیت‌هایی که مقدار آن‌ها در ماسک برابر False است، با مقدار منفی بسیار بزرگ ($-1e9$) پر می‌شوند.
- در نهایت پس از اعمال تابع `jax.nn.softmax` روی محور آخر، ضرب وزن‌های توجه در ماتریس v خروجی نهایی را تشکیل می‌دهد.

کد تست و اعتبارسنجی پیاده‌سازی

برای بررسی صحت عملکرد مکانیزم توجه و ماسک علی از تست خودکار زیر استفاده شده است:

```

1  # Self-check: rows of the attention matrix are probabilities, and the mask zeros the future.
2  _k = jax.random.PRNGKey(0)
3  q = jax.random.normal(jax.random.fold_in(_k, 11), (5, 8))
4  k = jax.random.normal(jax.random.fold_in(_k, 12), (5, 8))
5  v = jax.random.normal(jax.random.fold_in(_k, 13), (5, 8))
6
7  m = causal_mask(5)
8  assert m.shape == (5, 5) and m.dtype == jnp.bool_
9  assert bool(m[0, 0]) and not bool(m[0, 4]) and bool(m[4, 0])
10
11  out_nomask, attn_nomask = scaled_dot_product_attention(q, k, v)
12  out_mask, attn_mask = scaled_dot_product_attention(q, k, v, m)
13  assert out_mask.shape == (5, 8) and attn_mask.shape == (5, 5)
14  assert jnp.allclose(attn_nomask.sum(axis=-1), 1.0, atol=1e-6)
15  assert jnp.allclose(attn_mask.sum(axis=-1), 1.0, atol=1e-6)
16  assert float(jnp.max(jnp.triu(attn_mask, k=1))) < 1e-6 # nothing above the diagonal
17  print("OK: attention rows sum to 1; the causal mask zeros every future position")
18

```

تحلیل تست: این تست موارد زیر را تضمین می‌کند:

- خروجی تابع تولید ماسک علی دارای فرمت بولی و ابعاد کاملاً صحیح 5×5 باشد و مقادیر بالای قطر صفر باشند.
- مجموع مقادیر توزیع احتمال در هر سطر از ماتریس توجه (هم در حالت با ماسک و هم بدون ماسک) با خطای بسیار کوچک عددی دقیقاً برابر با 1.0 باشد.
- با اعمال ماسک علی، مقادیر بخش‌های آینده (بالای قطر اصلی) دقیقاً صفر شده و هیچ داده‌ای از آینده نشت نکند.

خروجی اجرای تست


```
OK: attention rows sum to 1; the causal mask zeros every future position
```

این خروجی گواهی بر این است که:

- فرآیند بهنجارسازی به درستی عمل کرده است.
- ماسک علی تمام درایه‌های غیرمجاز بالامثلی را پیش از اعمال لایه نرم‌بیشینه فیلتر نموده است.
- توابع پیاده‌سازی شده در چارچوب JAX از کارایی عددی و دقت ریاضی موردنیاز برخوردار هستند.

۳.۲ بصری‌سازی ماتریس توجه: تقابل حالت آزاد و علی

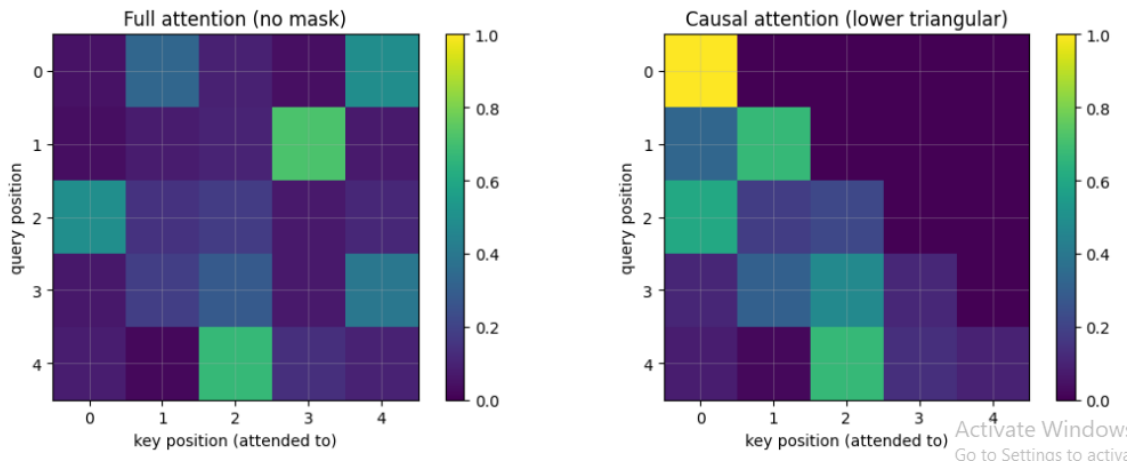
پس از پیاده‌سازی محاسباتی، برای درک بهتر نحوه توزیع وزن‌ها و تأثیر ماسک، ماتریس‌های توجه خروجی را با استفاده از کتابخانه matplotlib به صورت نقشه حرارتی (Heatmap) ترسیم می‌کنیم. این بصری‌سازی به ما اجازه می‌دهد تا ببینیم هر توکن (Query) به کدام توکن‌های دیگر (Key) توجه کرده است.

کد تولید نمودارهای مقایسه‌ای

```
1 # CHART 1 - attention heatmaps, with and without the causal mask, side by side.
2 fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
3
4 # Full attention (no mask)
5 im0 = ax[0].imshow(np.array(attn_nomask), cmap="viridis", vmin=0, vmax=1)
6 ax[0].set_title("Full attention (no mask)")
7 ax[0].set_xlabel("key position (attended to)")
8 ax[0].set_ylabel("query position")
9 fig.colorbar(im0, ax=ax[0], fraction=0.046)
10
11 # Causal attention (lower triangular)
12 im1 = ax[1].imshow(np.array(attn_mask), cmap="viridis", vmin=0, vmax=1)
13 ax[1].set_title("Causal attention (lower triangular)")
14 ax[1].set_xlabel("key position (attended to)")
15 ax[1].set_ylabel("query position")
16 fig.colorbar(im1, ax=ax[1], fraction=0.046)
17
18 plt.tight_layout()
19 plt.show()
20
```

تحلیل و تفسیر خروجی بصری

تصویر حاصل از اجرای کد بالا (شکل پایین)، تفاوت بنیادی بین مکانیزم توجه آزاد و مکانیزم توجه در مدل‌های مولد (Generative) را نشان می‌دهد:



شکل ۸: مقایسه توزیع وزن‌های توجه در دو حالت بدون ماسک (چپ) و با اعمال ماسک علی (راست)

۱. تحلیل نمودار **Full Attention** (سمت چپ): در این حالت، هیچ محدودیتی برای نگاه به آینده وجود ندارد. همان‌طور که در تصویر مشخص است، تمام خانه‌های ماتریس (حتی بخش بالای قطر اصلی) دارای رنگ و مقدار هستند. این یعنی مثلاً توکن شماره ۱ می‌تواند به اطلاعات توکن شماره ۴ دسترسی داشته باشد. این ساختار برای مدل‌های Encoder-only (مانند BERT) مناسب است که کل متن را یکباره می‌بینند.

۲. تحلیل نمودار **Causal Attention** (سمت راست): در این نمودار، تأثیر تابع `causal_mask` به وضوح دیده می‌شود. ماتریس توجه به یک ماتریس پابین‌مثلثی تبدیل شده است.

- ناحیه تیره (بنفش تیره): تمام درایه‌های بالای قطر اصلی که مقدار آن‌ها توسط ماسک به $-\infty$ میل کرده بود، پس از عبور از softmax به صفر تبدیل شده‌اند (رنگ بنفش تیره در نوار colorbar معادل مقدار 0.0 است).
- بافتار علی: هر سطر (موقعیت query) فقط مجاز است به ستون‌هایی (موقعیت‌های key) توجه کند که اندیس آن‌ها کوچکتر یا مساوی خودش است. برای مثال، سطر ۲ فقط در ستون‌های ۰، ۱ و ۲ دارای مقدار است.
- تمرکز توجه: رنگ‌های روشن‌تر (زرد و سبز) نشان‌دهنده وزن‌های بالاتر هستند که نشان می‌دهد مدل در هر مرحله بر روی کدام کلمات قبلی تمرکز بیشتری دارد.

این بصری‌سازی تأیید می‌کند که مدل ما به درستی محدود شده است تا در هنگام آموزش، جواب (توکن بعدی) را "تقلب" نکند و صرفاً بر اساس تاریخچه گذشته یاد بگیرد. این همان ویژگی اصلی است که امکان تولید متن به صورت خودبازگشتی (Autoregressive) را در مدل‌های خانواده GPT فراهم می‌کند.

۴.۲ چرا امتیازهای توجه را بر $\sqrt{d_{\text{head}}}$ تقسیم می‌کنیم؟

یکی از ارکان حیاتی در مکانیزم توجه ضرب نقطه‌ای، مقیاس‌گذاری (Scaling) امتیازها پیش از اعمال تابع softmax است. در این بخش، منطق تئوریک پشت این تقسیم را با یک تحلیل ریاضی شروع کرده و سپس با اجرای یک آزمایش تجربی در JAX صحت آن را اثبات می‌کنیم.



تحلیل و اثبات ریاضی اثر بعد هد بر واریانس

فرض کنید دو بردار پرس وجو (q) و کلید (k) به طول d (همان d_{head}) داشته باشیم که درایه‌های آن‌ها به صورت مستقل و هم توزیع (i.i.d.) از یک توزیع نرمال استاندارد با میانگین صفر و واریانس یک نمونه برداری شده‌اند:

$$q_i, k_i \sim \mathcal{N}(0, 1) \Rightarrow E[q_i] = E[k_i] = 0, \quad \text{Var}(q_i) = \text{Var}(k_i) = 1$$

ضرب نقطه‌ای این دو بردار به صورت زیر تعریف می‌شود:

$$q \cdot k = \sum_{i=1}^d q_i k_i$$

با توجه به مستقل بودن مؤلفه‌ها، میانگین این ضرب نقطه‌ای برابر است با:

$$E[q \cdot k] = \sum_{i=1}^d E[q_i] E[k_i] = 0$$

برای محاسبه واریانس هر سهم $q_i k_i$ داریم:

$$\text{Var}(q_i k_i) = E[q_i^2 k_i^2] - (E[q_i k_i])^2 = E[q_i^2] E[k_i^2] - 0 = (1)(1) = 1$$

از آنجا که تمام جملات مجموع مستقل هستند، واریانس کل ضرب نقطه‌ای برابر با مجموع واریانس تک تک جملات خواهد بود:

$$\text{Var}(q \cdot k) = \sum_{i=1}^d \text{Var}(q_i k_i) = d$$

بنابراین، واریانس ضرب نقطه‌ای خام به طور خطی با بعد بردار (d) افزایش می‌یابد.

وقتی d بزرگ باشد (مثلاً ۲۵۶)، مقادیر ضرب نقطه‌ای می‌توانند بسیار بزرگ (مثبت یا منفی) شوند. هنگام اعمال تابع softmax، این مقادیر بزرگ باعث می‌شوند که خروجی لایه به یک توزیع شبه تک مقداری (One-hot) تبدیل شده و در نتیجه، گرادیان در بیشتر نقاط به شدت به صفر نزدیک شود (Vanishing Gradients).

برای حل این مشکل، ضرب نقطه‌ای بر $\sqrt{d}$ تقسیم می‌شود. واریانس متغیر جدید برابر خواهد بود با:

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d}}\right) = \frac{1}{d} \text{Var}(q \cdot k) = \frac{d}{d} = 1$$

این تقسیم هوشمندانه تضمین می‌کند که واریانس ورودی‌های تابع softmax همواره در حدود 1.0 باقی بماند؛ صرف نظر از اینکه بعد هد چقدر بزرگ انتخاب شده باشد.

پیاده‌سازی آزمایش تجربی در JAX

کد زیر برای تولید بردارهای تصادفی در ابعاد مختلف ($d \in \{4, 16, 64, 256\}$) و محاسبه تجربی واریانس در دو حالت خام و مقیاس گذاری شده نوشته شده است:

```
1 # CHART 2 - variance of q.k grows ~d, but q.k / sqrt(d) stays ~1.
2 dims = [4, 16, 64, 256]
3 raw_var, scaled_var = [], []
4 key = jax.random.PRNGKey(2)
```



```

5
6     for d in dims:
7         key, k1, k2 = jax.random.split(key, 3)
8         qd = jax.random.normal(k1, (20000, d))
9         kd = jax.random.normal(k2, (20000, d))
10        dots = jnp.sum(qd * kd, axis=-1)          # one dot product per row
11        raw_var.append(float(jnp.var(dots)))
12        scaled_var.append(float(jnp.var(dots / jnp.sqrt(d))))
13
14    fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
15    ax[0].plot(dims, raw_var, "-o", color="#0099cc", label="Var(q.k)")
16    ax[0].plot(dims, dims, "--", color="#999", label="y = d (reference)")
17    ax[0].set_title("Raw dot-product variance grows with d")
18    ax[0].set_xlabel("d_head"); ax[0].set_ylabel("variance"); ax[0].legend()
19
20    ax[1].plot(dims, scaled_var, "-o", color="#ff8a00", label="Var(q.k / sqrt(d))")
21    ax[1].axhline(1.0, ls="--", color="#999", label="y = 1 (reference)")
22    ax[1].set_title("Scaled dot-product variance stays ~1")
23    ax[1].set_xlabel("d_head"); ax[1].set_ylabel("variance"); ax[1].set_ylim(0, 2); ax[1].legend()
24
25    plt.tight_layout()
26    plt.show()
27
28    print("raw variances :", [round(x, 1) for x in raw_var])
29    print("scaled vars   :", [round(x, 3) for x in scaled_var])
30

```

توضیح خطبه خط کد پیاده‌سازی:

- `dims = [4, 16, 64, 256]`: تعریف ابعاد مختلف هد جهت ارزیابی تأثیر اندازه بعد بر رفتار واریانس.
- `jax.random.split(key, 3)`: تقسیم کلید تصادفی در جکس به ۳ کلید جدید برای حفظ خاصیت بدون حالت بودن (Stateless) تولید اعداد تصادفی.
- `jax.random.normal(..., (20000, d))`: ایجاد $20,000$ نمونه بردار جفت پرس‌وجو و کلید به طول d برای به دست آوردن تخمین آماری دقیق و بدون نویز.
- `jnp.sum(qd * kd, axis=-1)`: انجام ضرب نقطه‌ای سطر به سطر (محاسبه مقدار روی محور آخر).
- `jnp.var(...)`: محاسبه واریانس تجربی مقادیر ضرب نقطه‌ای حاصل برای کل نمونه‌ها.
- ترسیم نمودارها در قالب دوزیر نمودار (subplots) جهت مقایسه مستقیم رشد خطی واریانس خام در برابر ثبات واریانس مقیاس‌گذاری شده.

خروجی عددی آزمایش

پس از اجرای کد بالا، نتایج عددی زیر حاصل شد:

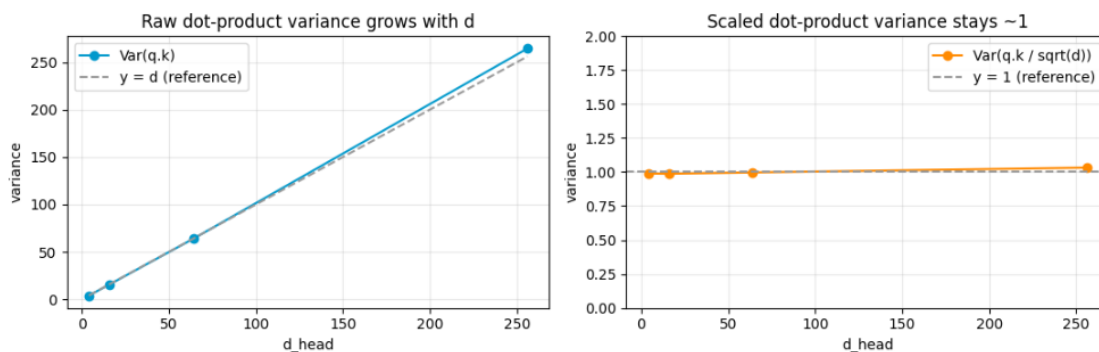
```
1 raw variances : [4.0, 15.8, 63.8, 264.3]
2 scaled vars    : [0.989, 0.987, 0.997, 1.033]
3
```

تحلیل نتایج عددی:

- واریانس خام (raw variances): برای ابعاد ۴، ۱۶، ۶۴ و ۲۵۶، واریانس‌های تجربی به ترتیب برابر با 4.0، 15.8، 63.8 و 264.3 شده‌اند. این مقادیر تطابق شگفت‌انگیزی با پیش‌بینی تئوری ($Var = d$) دارند و روند رشد کاملاً خطی را نشان می‌دهند.
- واریانس مقیاس‌گذاری‌شده (scaled vars): با اعمال تقسیم بر $\sqrt{d}$ ، واریانس برای تمام ابعاد در محدوده بسیار نزدیک به عدد یک (0.989 تا 1.033) ثابت مانده است. این نشان می‌دهد فرآیند نرمال‌سازی با موفقیت اثر بعد بر روی شدت تغییرات خروجی را خنثی کرده است.

درج و تحلیل نمودارهای خروجی

نمودارهای حاصل از این شبیه‌سازی در شکل زیر نمایش داده شده‌اند:



شکل ۹: نمودار سمت چپ رشد خطی واریانس ضرب نقطه‌ای خام با افزایش بعد را نشان می‌دهد، در حالی که نمودار سمت راست پایداری واریانس حول مقدار ۱ را پس از مقیاس‌گذاری به تصویر می‌کشد.

تحلیل بصری نمودارها:

- نمودار سمت چپ (واریانس خام - آبی): خط پیوسته واریانس حاصل از محاسبات ما کاملاً بر روی خط چین خاکستری مرجع ($y = d$) منطبق شده است. این یعنی با بزرگ شدن ابعاد لایه‌ها در ترنسفورمرهای بزرگ، دامنه عددی خروجی لایه توجه پیش از softmax به شدت گسترده و غیرقابل کنترل می‌شود.
- نمودار سمت راست (واریانس مقیاس‌گذاری‌شده - نارنجی): منحنی نارنجی رنگ به صورت یک خط کاملاً افقی و پایدار، منطبق بر خط مرجع $y = 1$ قرار گرفته است. این یعنی مستقل از اینکه ابعاد مدل چقدر تغییر کند، لایه softmax ورودی‌هایی با مقیاس عددی ثابت دریافت خواهد کرد.

این آزمایش تجربی به وضوح نشان می‌دهد که تقسیم امتیازها بر جذر بعد هد ($\sqrt{d_{\text{head}}}$) ضامن پایداری عددی شبکه‌های عمیق ترنسفورمر بوده و از بروز پدیده ناپدید شدن گرادیان‌ها در لایه توجه جلوگیری می‌نماید.

۵.۲ کدگذاری موقعیتی سینوسی (Sinusoidal Positional Encoding)

مکانیزم توجه ترنسفورمر به دلیل ماهیت جایگشت‌پذیری (Permutation Equivariance)، هیچ درکی از ترتیب توکن‌ها ندارد. برای رفع این نقیصه، باید اطلاعات موقعیتی هر توکن به صورت برداری منحصربه‌فرد به بردار ویژگی‌های ورودی آن اضافه گردد. در این بخش، بر اساس فرمول ارائه شده در مقاله اصلی ترنسفورمر (Attention Is All You Need)، کدگذاری موقعیتی سینوسی را پیاده‌سازی و تحلیل می‌کنیم.

فرمولاسیون ریاضی کدگذاری موقعیتی

برای یک توالی به طول T و بعد مدل d ، بردار موقعیت برای توکن در موقعیت pos ($0 \leq pos < T$) در بعد ویژگی i ($0 \leq i < d$) به صورت زیر محاسبه می‌شود:

$$PE_{(pos, 2k)} = \sin\left(\frac{pos}{10000^{\frac{2k}{d}}}\right)$$

$$PE_{(pos, 2k+1)} = \cos\left(\frac{pos}{10000^{\frac{2k}{d}}}\right)$$

در این فرمول، شاخص $2k$ نشان‌دهنده ابعاد زوج و $2k+1$ نشان‌دهنده ابعاد فرد است. رابطه بالا را می‌توان به کمک متغیر نرخ زاویه (angle rate) بازنویسی کرد:

$$\theta_{pos,i} = pos \cdot \omega_i \quad , \quad \omega_i = \frac{1}{10000^{\frac{2|i/2|}{d}}}$$

دلیل جفت کردن فرکانس ابعاد مجاور ($2\lfloor i/2 \rfloor$) این است که بردارهای زوج و فرد مجاور فرکانس یکسانی را به اشتراک بگذارند. این ویژگی باعث می‌شود که بردارهای موقعیت‌های نسبی ($pos+k$) به صورت خطی از روی بردار موقعیت فعلی (pos) قابل بازیابی و یادگیری توسط شبکه باشند.

پیاده‌سازی برداری در JAX (بدون استفاده از حلقه)

برای پیاده‌سازی بهینه این الگوریتم روی شتاب‌دهنده‌ها، از قابلیت پخش (Broadcasting) در کتابخانه JAX استفاده شده است تا محاسبات بدون نیاز به حلقه‌های پایتونی سنگین انجام پذیرد:

```
1 def positional_encoding(T, d):
2     # column vector of positions (T, 1)
3     pos = jnp.arange(T)[None, :]
4
5     # row vector of dimension indices (1, d)
6     i = jnp.arange(d)[None, :]
7
8     # compute angle rates
9     angle_rates = 1.0 / (10000 ** ((2 * (i // 2)) / d))
```



```

10
11 # compute angles via broadcasting -> shape (T, d)
12 angles = pos * angle_rates
13
14 # apply sin to even indices, cos to odd indices
15 pe = jnp.where(i % 2 == 0,
16               jnp.sin(angles),
17               jnp.cos(angles))
18 return pe
19

```

توضیح خط به خط منطق پیاده سازی:

- `pos = jnp.arange(T)[: , None]`: ایجاد یک بردار ستونی به ابعاد $T \times 1$ شامل اندیس موقعیت ها.
- `i = jnp.arange(d)[None, :]`: ایجاد یک بردار سطری به ابعاد $1 \times d$ شامل ویژگی ها.
- `angle_rates`: محاسبه نرخ فرکانس با فرمول ریاضی فوق به گونه ای که برای هر دو بعد متوالی زوج و فرد (با استفاده از `//` 2)، یک نرخ زاویه یکسان به دست آید.
- `angles = pos * angle_rates`: با استفاده از ویژگی پخش (Broadcasting) در جکس، بردار ستونی $T \times 1$ در بردار سطری $1 \times d$ ضرب خارجی شده و ماتریسی به ابعاد $T \times d$ تولید می کند که درایه (pos, i) آن برابر حاصل ضرب موقعیت در نرخ فرکانس است.
- `jnp.where(i % 2 == 0, ...)`: بررسی زوج بودن اندیس بعد. اگر اندیس زوج باشد، تابع سینوس روی زاویه اعمال می شود و اگر فرد باشد، کسینوس اعمال می گردد. خروجی نهایی ماتریسی به ابعاد $T \times d$ خواهد بود.

تست درستی عملکرد کدگذاری موقعیتی

جهت اعتبارسنجی ابعاد ماتریس خروجی و قرارگیری مقادیر در محدوده مجاز توابع مثلثاتی (بازه $[-1, 1]$)، از اسکرپت ارزیابی زیر استفاده شد:

```

1 # Self-check: shape is (T, d) and every value lies in [-1, 1].
2 pe = positional_encoding(32, 16)
3 assert pe.shape == (32, 16)
4 assert float(jnp.max(jnp.abs(pe))) <= 1.0 + 1e-6
5 print("OK: positional_encoding has shape", pe.shape, "and |values| <= 1")
6

```

تحلیل تست و نتایج: این تست بررسی می کند که:

- ابعاد ماتریس خروجی برای توالی به طول ۳۲ و بعد مدل ۱۶ دقیقاً برابر با 32×16 باشد.
- با توجه به خواص توابع سینوس و کسینوس، قدر مطلق تمام مقادیر موجود در ماتریس کوچک تر یا مساوی 1.0 باشد (با در نظر گرفتن خطای اعشار عددی $1e-6$).



خروجی اجرای تست

پس از اجرای تست فوق، خروجی زیر چاپ گردید:

```
1 OK: positional_encoding has shape (32, 16) and |values| <= 1
2
```

این خروجی تأیید می‌کند که ماتریس کدگذاری موقعیتی بدون خطا ساخته شده، از ابعاد موردانتظار برخوردار بوده و کران‌های مقادیر عددی خروجی کاملاً با خواص تئوری توابع مثلثاتی همخوانی دارد.

بصری‌سازی و تحلیل نمودارهای کدگذاری موقعیتی

برای درک عمیق‌تر نحوه رفتار فرکانس‌ها و نحوه رمزگذاری اطلاعات موقعیتی، ماتریس کدگذاری موقعیتی را برای توالی به طول $T = 100$ و بعد مدل $d = 64$ ترسیم و تحلیل می‌کنیم. کد زیر جهت تولید نقشه حرارتی (Heatmap) کل ماتریس و رسم نمودار تغییرات مقدار چند بعد خاص بر حسب موقعیت استفاده شده است:

```
1 # CHART 3 - the positional-encoding table as a heatmap, plus a few sinusoid columns.
2 pe_big = np.array(positional_encoding(100, 64))
3 fig, ax = plt.subplots(1, 2, figsize=(12, 4.0))
4 im = ax[0].imshow(pe_big, cmap="RdBu", aspect="auto", vmin=-1, vmax=1)
5 ax[0].set_title("Sinusoidal positional encoding (100 x 64)")
6 ax[0].set_xlabel("embedding dimension"); ax[0].set_ylabel("position")
7 fig.colorbar(im, ax=ax[0], fraction=0.046)
8
9 for dim in [0, 4, 8, 20, 40]:
10 ax[1].plot(pe_big[:, dim], label=f"dim {dim}")
11 ax[1].set_title("Individual dimensions oscillate at different frequencies")
12 ax[1].set_xlabel("position"); ax[1].set_ylabel("value"); ax[1].legend(fontsize=8)
13 plt.tight_layout()
14 plt.show()
15
```

توضیح خط به خط کد بصری‌سازی:

- `pe_big = np.array(positional_encoding(100, 64))`: تولید ماتریس کدگذاری موقعیتی با ابعاد 100×64 و تبدیل آن به آرایه نامپای جهت سازگاری کامل با کتابخانه ترسیم نمودار.
- `ax[0].imshow(..., cmap="RdBu", vmin=-1, vmax=1)`: رسم ماتریس به صورت نقشه حرارتی با پالت رنگی متقارن قرمز-سفید-آبی (RdBu) که در آن مقادیر نزدیک به -1 با رنگ قرمز تیره، مقادیر صفر با رنگ سفید و مقادیر نزدیک به $+1$ با رنگ آبی تیره نمایش داده می‌شوند.
- `ax[1].plot(pe_big[:, dim], ...)`: ترسیم تغییرات مقدار مولفه در راستای طول توالی (از موقعیت 0 تا 99) برای چند بعد خاص (0، 4، 8، 20 و 40) به صورت مجزا.

تحلیل و ارزیابی نمودارهای خروجی

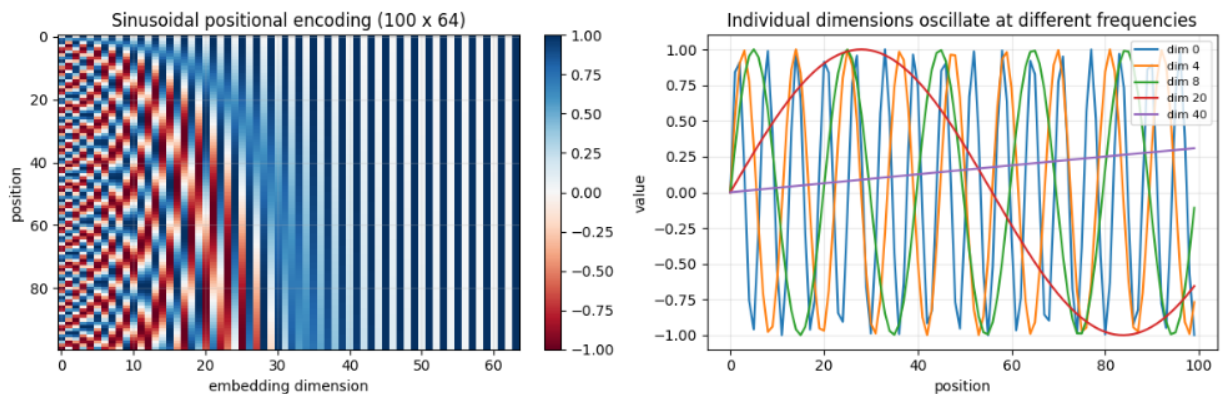
نمودار حاصل از اجرای کد بالا در صفحه بعد آورده شده است.

تحلیل نقشه حرارتی (نمودار سمت چپ):

- روند تغییر فرکانس: در ابعاد اولیه (سمت چپ نقشه حرارتی، نزدیک به صفر)، تغییرات رنگ به صورت نوارهای بسیار باریک و متناوب قرمز و آبی دیده می‌شود. این امر نشان‌دهنده فرکانس بالای نوسانات در ابعاد پایین است که اطلاعات موقعیتی با جزئیات دقیق (Fine-grained) را ثبت می‌کنند.
- گذار به فرکانس‌های پایین: با حرکت به سمت ابعاد بزرگتر (راست نقشه حرارتی)، طول موج‌ها به شدت افزایش یافته و رنگ‌ها یکنواخت‌تر می‌شوند. در ابعاد انتهایی (نزدیک به ۶۴)، تغییرات رنگ به سختی در طول ۱۰۰ توکن رخ می‌دهد که نشان‌دهنده ثبت اطلاعات موقعیتی کلی و در ابعاد بزرگ (Coarse-grained) است.

تحلیل نمودار نوسانات ابعاد منفرد (نمودار سمت راست):

- بعد ۰ (dim 0 - آبی): دارای کوتاه‌ترین طول موج و بالاترین فرکانس نوسان است. این موج سینوسی به سرعت بین مقادیر 1+ و 1- نوسان می‌کند و برای تفکیک توکن‌های بسیار نزدیک به کار می‌رود.
- ابعاد میانی (dim 4، dim 8، dim 20): با افزایش شاخص بعد، طول موج‌ها به تدریج بزرگتر می‌شوند. به عنوان مثال، بعد ۲۰ (dim 20 - قرمز) تنها یک سیکل نوسان کامل را در طول کل ۱۰۰ توکن تجربه می‌کند.
- بعد ۴۰ (dim 40 - بنفش): فرکانس بسیار پایینی دارد، به طوری که در طول ۱۰۰ توکن حتی نیمی از یک دوره تناوب را هم طی نکرده و رفتاری تقریباً خطی و صعودی ملایم از صفر تا حدود 0.3 را نشان می‌دهد. این ابعاد برای درک روابط فواصل دور دست در متن‌های طولانی کاربرد دارند.



شکل ۱۰: نقشه حرارتی ماتریس کدگذاری موقعیتی سینوسی (سمت چپ) و رفتار نوسانی ابعاد مختلف مدل بر حسب موقعیت با فرکانس‌های گوناگون (سمت راست).

این ساختار فرکانسی چندگانه تضمین می‌کند که ترنسفورمر قادر است روابط وابستگی محلی (توکن‌های مجاور با ابعاد فرکانس بالا) و روابط ساختاری بلندمدت (ابعاد فرکانس پایین) را به طور هم‌زمان و با دقت بالا یاد بگیرد.

۶.۲ خود-توجهی تک سره (Single-head Self-attention)

اکنون با در اختیار داشتن تمام اجزای حیاتی (مکانیزم توجه، ماسک علی و کدگذاری موقعیتی)، می توانیم لایه اصلی خود-توجهی را بنا کنیم. در یک لایه خود-توجهی، هر سه مؤلفه پرس و جو (Q)، کلید (K) و مقدار (V) از یک ورودی واحد (x) مشتق می شوند. این کار از طریق تصویر کردن بردار ورودی x در فضاهای مختلف با استفاده از ماتریس های وزن یادگیری پذیر انجام می شود.

ساختار لایه خود-توجهی

فرض کنید ورودی ما x با ابعاد (T, C) باشد (طول توالی T و بعد مدل C). فرآیند محاسبه در این لایه به شرح زیر است:

۱. تصویرسازی خطی (Linear Projections): ورودی x با استفاده از سه ماتریس وزن W_Q, W_K, W_V به فضاهای جدید نگاشت می شود:

$$Q = x \cdot W_Q, \quad K = x \cdot W_K, \quad V = x \cdot W_V$$

۲. ایجاد ماسک علی (Causal Masking): یک ماسک پایین مثلثی به اندازه $T \times T$ ایجاد می شود تا از نشت اطلاعات آینده به گذشته جلوگیری کند.

۳. محاسبه توجه مقیاس گذاری شده: با استفاده از تابع `scaled_dot_product_attention` که پیش تر پیاده سازی شد، خروجی و ماتریس توجه محاسبه می شوند.

پیاده سازی در JAX

کد زیر لایه خود-توجهی را با استفاده از توابع قبلی پیاده سازی می کند:

```

1  def self_attention(x, Wq, Wk, Wv):
2      # project x into q, k, v
3      q, k, v = x @ Wq, x @ Wk, x @ Wv
4
5      # build a causal mask of size T = x.shape[0]
6      mask = causal_mask(x.shape[0])
7
8      # run scaled_dot_product_attention and return (out, attn)
9      out, attn = scaled_dot_product_attention(q, k, v, mask)
10
11     return out, attn
12
    
```

توضیح کد:

- عملیات ضرب ماتریسی $x @ W$ ویژگی های ورودی را به فضاهای اختصاصی توجه منتقل می کند.
- تابع `causal_mask(x.shape[0])` به صورت پویا و بر اساس طول توالی ورودی، محدودیت دسترسی به آینده را اعمال می کند.
- خروجی نهایی `out` دارای همان ابعاد ورودی $(T \times C)$ است که اکنون شامل اطلاعات متنی غنی تری است.

تست واحد بر روی قطعه متن واقعی

برای اطمینان از صحت عملکرد، لایه را بر روی عبارت معروف شکسپیر ("To be, or not to be") آزمایش می‌کنیم. در این تست، ابتدا متن کدگذاری شده (Encode)، سپس با بردارهای تعبیه تصادفی ترکیب شده و در نهایت اطلاعات موقعیتی به آن اضافه می‌شود:

```

1  # Self-check on a real Shakespeare snippet
2  snippet = "To be, or not to be"
3  ids = jnp.array(encode(snippet))
4  T = len(snippet)
5  C = 32
6  _sk = jax.random.PRNGKey(3)
7
8  # Initializing weights and embeddings
9  emb = jax.random.normal(jax.random.fold_in(_sk, 1), (vocab_size, C)) * 0.2
10 x_demo = emb[ids] + positional_encoding(T, C)
11 Wq = jax.random.normal(jax.random.fold_in(_sk, 2), (C, C)) * 0.3
12 Wk = jax.random.normal(jax.random.fold_in(_sk, 3), (C, C)) * 0.3
13 Wv = jax.random.normal(jax.random.fold_in(_sk, 4), (C, C)) * 0.3
14
15 # Running self-attention
16 sa_demo_out, sa_demo_attn = self_attention(x_demo, Wq, Wk, Wv)
17
18 # Assertions for validation
19 assert sa_demo_out.shape == (T, C) and sa_demo_attn.shape == (T, T)
20 assert float(jnp.max(jnp.triu(sa_demo_attn, k=1))) < 1e-6
21 print("OK: self_attention output", sa_demo_out.shape, "| attention", sa_demo_attn.shape)
22

```

تحلیل نتایج و خروجی

پس از اجرای تست، خروجی زیر حاصل شد:

```

1  OK: self_attention output (19, 32) | attention (19, 19)
2

```

تحلیل ابعاد و ویژگی‌ها:

- ابعاد خروجی (۱۹، ۳۲): عبارت ورودی دارای ۱۹ کاراکتر (توکن) است. بعد مدل نیز ۳۲ انتخاب شده بود. خروجی (19, 32) نشان می‌دهد که لایه خود-توجهی، ساختار ابعاد داده را حفظ کرده است.
- ماتریس توجه (۱۹، ۱۹): یک ماتریس مربعی تولید شده که در آن هر توکن می‌تواند به تمام ۱۹ توکن دیگر (با رعایت محدودیت زمان) توجه کند.
- تأیید خاصیت علی (Causality Check): دستور `jnp.triu(..., k=1)` بخش مثلثی بالایی ماتریس (آینده) را استخراج می‌کند. مقدار `max` نزدیک به صفر ($< 1e - 6$) تضمین می‌کند که هیچ توکنی اطلاعاتی از کلمات بعدی خود دریافت نکرده است.

این پیاده‌سازی موفقیت‌آمیز، واحد پایه ساختاری (Building Block) برای بلاک‌های پیچیده‌تر ترنسفورمر را فراهم می‌کند.

بصری سازی ماتریس خود-توجهی بر روی متن نمونه

برای درک شهودی از نحوه تخصیص وزن های توجه بین توکن های مختلف در یک توالی واقعی، ماتریس توجه حاصل از لایه self-attention را برای عبارت "To be, or not to be" رسم می کنیم. در این نمودار، هر سطر نشان دهنده یک «پرس و جو» (Query) و هر ستون نشان دهنده یک «کلید» (Key) است.

```
1 # CHART 4 - attention heatmap on "To be, or not to be" with character tick labels.
2 labels = [repr(c)[1:-1] for c in snippet]
3 fig, ax = plt.subplots(figsize=(7.5, 6.5))
4 im = ax.imshow(np.array(sa_demo_attn), cmap="magma", vmin=0)
5
6 ax.set_xticks(range(T)); ax.set_xticklabels(labels)
7 ax.set_yticks(range(T)); ax.set_yticklabels(labels)
8 ax.set_title('Causal self-attention on "To be, or not to be"')
9 ax.set_xlabel("key character (attended to)"); ax.set_ylabel("query character")
10
11 fig.colorbar(im, ax=ax, fraction=0.046, label="attention weight")
12 plt.tight_layout()
13 plt.show()
14
```

توضیح تکنیک های پیاده سازی:

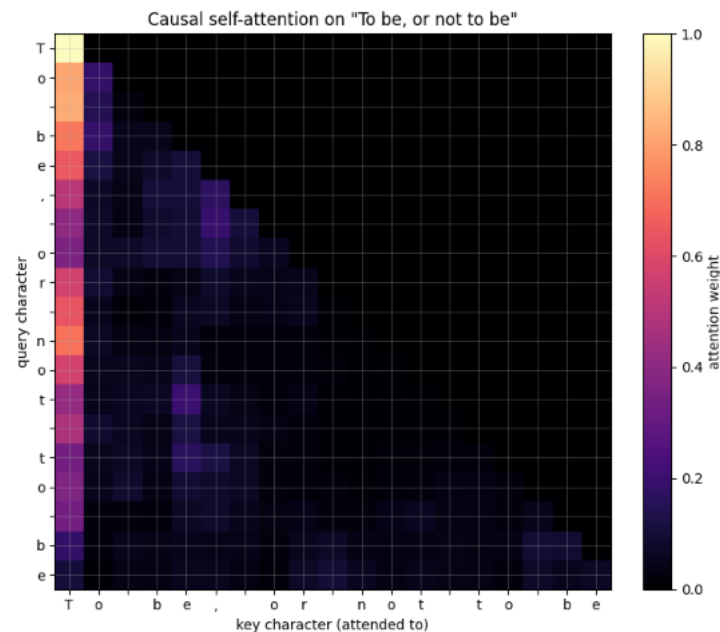
- `labels = [repr(c)[1:-1] for c in snippet]`: استخراج کاراکترها به عنوان برچسب محورها. استفاده از `repr` باعث می شود کاراکترهای خاص (مثل فاصله) نیز قابل رویت باشند.
- `cmap="magma"`: استفاده از پالت رنگی `magma` که کنتراست بالایی بین مقادیر نزدیک به صفر (سیاه) و مقادیر بالا (زرد/سفید) ایجاد می کند و برای نمایش توزیع احتمال `softmax` بسیار مناسب است.
- `ax.set_xticklabels(labels)`: تنظیم برچسب های محور افقی و عمودی با حروف تشکیل دهنده متن جهت انطباق مستقیم هر درایه ماتریس با کاراکتر مربوطه.

تحلیل و ارزیابی ماتریس توجه علی

خروجی بصری این لایه در شکل زیر نمایش داده شده است:

تحلیل الگوهای بصری:

- ساختار مثلثی (Causal Structure): بارزترین ویژگی این نمودار، سیاه بودن کامل بخش مثلثی بالای قطر اصلی است. این موضوع تأیید می کند که مکانیزم `Causal Mask` به درستی عمل کرده و هیچ کاراکتری (در محور y) اجازه نداشته است به کاراکترهای بعد از خود (در محور x) توجه کند.
- ستون اول (کاراکتر 'T'): مشاهده می شود که بسیاری از کاراکترهای بعدی، وزن توجه بالایی را به کاراکتر اول ('T') اختصاص داده اند (ستون اول روشن است). این یک رفتار رایج در مدل های زبانی است که اولین توکن اغلب به عنوان یک «نقطه اتکا» یا «مخزن اطلاعاتی» برای کل توالی عمل می کند.



شکل ۱۱: نقشه حرارتی ماتریس توجه برای عبارت نمونه. ساختار پایین مثلثی ماتریس به وضوح نشان‌دهنده اعمال موفقیت‌آمیز ماسک علی است.

- تمرکز موضعی (**Local Attention**): روشنایی درایه‌های روی قطر اصلی و مجاور آن نشان می‌دهد که هر کاراکتر به شدت به خودش و کاراکترهای بلافاصله قبل از خود وابسته است تا بتواند ساختار کلمات را حفظ کند.
- تکرار الگوها: با توجه به تکرار عبارت "to be" در متن، می‌توان دید که وقتی مدل به دومین عبارت "to be" می‌رسد، الگوهای توجه مشابهی با بخش اول تکرار می‌شوند که نشان‌دهنده توانایی مدل در شناسایی ساختارهای تکرار شونده است.
- این تصویرسازی ثابت می‌کند که لایه خود-توجهی ما نه تنها از نظر ابعادی درست عمل می‌کند، بلکه به درستی محدودیت‌های زمانی (Temporal Constraints) را رعایت کرده و آماده است تا به عنوان بلوک سازنده در معماری کامل ترنسفورمر مورد استفاده قرار گیرد.

۷.۲ تحلیل رفتار توجه: آنتروپی توزیع توجه تحت ماسک علی

یک معیار تشخیصی بسیار مفید برای درک عمیق‌تر رفتار مکانیزم خود-توجهی، محاسبه آنتروپی شانون (**Entropy**) توزیع توجه برای هر موقعیت پرس‌وجو (Query) است.

آنتروپی توزیع توجه نشان‌دهنده میزان متمرکز یا پراکنده بودن توجه مدل است:

- آنتروپی پایین: نشان می‌دهد که پرس‌وجو توجه خود را به شدت روی تعداد بسیار کمی از کلیدها (مثلاً یک یا دو کاراکتر خاص) متمرکز کرده است.

- آنتروپی بالا: نشان می‌دهد که توجه به طور یکنواخت و گسترده روی بسیاری از کلیدهای در دسترس پخش شده است.

تحت یک ماسک علی، اولین توکن ($t = 1$) تنها می‌تواند به خودش توجه کند؛ بنابراین توزیع توجه آن یک تک‌پله (Delta Function) با احتمال 1.0 است و آنتروپی آن لزوماً باید صفر باشد. با جلو رفتن در توالی، تعداد کلیدهای در دسترس (مرئی) افزایش می‌یابد و بیشینه آنتروپی نظری ممکن برای توکن i -ام برابر با $\ln(i)$ خواهد بود (که متناظر با توزیع کاملاً یکنواخت روی i کلید است).

فرمولاسیون ریاضی

آنتروپی توجه برای پرس و جوی i به صورت زیر محاسبه می‌شود:

$$H(A_i) = - \sum_{j=1}^T A_{i,j} \ln(A_{i,j})$$

که در آن $A_{i,j}$ وزن توجه اختصاص یافته از پرس و جوی i به کلید j است. برای جلوگیری از خطای عددی حاصل از محاسبه $\ln(0)$ ، مقدار کوچک $\epsilon = 10^{-12}$ به احتمالات مثبت اضافه می‌شود و مقادیر صفر نادیده گرفته می‌شوند. بیشینه آنتروپی ممکن در موقعیت i (به دلیل وجود ماسک علی که تنها i کلید اول را مجاز می‌شمارد) برابر است با:

$$H_{\max}(i) = \ln(i)$$

پیاده‌سازی در Python و JAX

کد زیر آنتروپی توزیع توجه را محاسبه کرده و آن را در مقایسه با بیشینه آنتروپی نظری رسم می‌کند:

```
1 # CHART 5 - entropy of each query's attention distribution (causal).
2 attn_np = np.array(sa_demo_attn)
3 # Avoid log(0) by masking or adding epsilon
4 ent = -np.sum(np.where(attn_np > 0, attn_np * np.log(attn_np + 1e-12), 0.0), axis=-1)
5 max_ent = np.log(np.arange(1, T + 1)) # max possible entropy given #visible keys
6
7 fig, ax = plt.subplots(figsize=(10, 3.6))
8 ax.bar(range(T), ent, color="#0099cc", label="attention entropy")
9 ax.plot(range(T), max_ent, "--o", color="#ff8a00", label="max entropy = log(#visible keys)")
10
11 ax.set_xticks(range(T)); ax.set_xticklabels(labels)
12 ax.set_title("Per-query attention entropy under the causal mask")
13 ax.set_xlabel("query character"); ax.set_ylabel("entropy (nats)"); ax.legend()
14 plt.tight_layout()
15 plt.show()
16
17 print("first-query entropy (must be 0):", round(float(ent[0]), 6))
18
```

توضیح کد پیاده‌سازی:

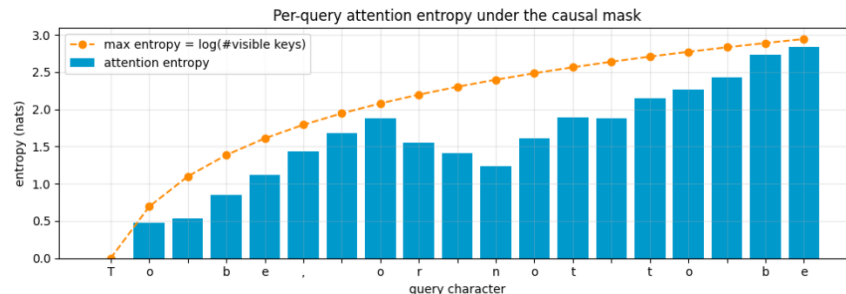
- `np.where(attn_np > 0, ...)`: تضمین می‌کند که تابع لگاریتم فقط روی درایه‌هایی اعمال می‌شود که وزن توجه آن‌ها بزرگتر از صفر است تا از بروز خطا یا مقادیر NaN جلوگیری شود.
- `max_ent = np.log(np.arange(1, T + 1))`: سقف آنتروپی توزیع یکنواخت را برای هر گام زمانی با توجه به طول پنجره متغیر محاسباتی تعریف می‌کند.
- واحد اندازه‌گیری آنتروپی به دلیل استفاده از لگاریتم طبیعی (e base)، ناتز (nats) است.

تحلیل و ارزیابی خروجی بصری آنتروپی

پس از اجرای کد، خروجی متنی زیر در خروجی استاندارد چاپ شد:

```
first-query entropy (must be 0): -0.0
```

خروجی نمودار حاصل از اجرای کد بالا در شکل زیر نمایش داده شده است:



شکل ۱۲: نمودار آنتروپی توجه به ازای هر کاراکتر ورودی. خط چین نارنجی مرز تنوریک آنتروپی (توزیع کاملاً یکنواخت) و ستون‌های آبی، آنتروپی واقعی مدل را نشان می‌دهند.

تحلیل و تفسیر نمودار:

- موقعیت آغازین (کاراکتر 'T'): همان‌طور که پیش‌بینی می‌شد، آنتروپی توکن اول دقیقاً برابر با ۰.۰ (یا صفر) است. زیرا تحت ماسک علی، این توکن چاره‌ای جز توجه صددرصدی به خودش ندارد.
- روند افزایشی مرز نظری (خط نارنجی): با حرکت به سمت راست توالی، تعداد کلیدهای مجاز به صورت خطی افزایش یافته و در نتیجه منحنی ظرفیت آنتروپی به صورت لگاریتمی ($\ln(i)$) صعود می‌کند.
- تمرکز انتخابی مدل (ستون‌های آبی): ستون‌های آبی در بیشتر نقاط فاصله معناداری با خط چین نارنجی دارند. این فاصله نشان‌دهنده آن است که لایه خود-توجهی رفتاری کاملاً انتخابی دارد و وزن‌ها را به صورت یکنواخت پخش نمی‌کند؛ بلکه بر روی ویژگی‌های کلیدی متمرکز می‌شود.
- رفتار در نقاط خاص توالی:

- در کاراکترهای میانی مثل 'r' و 'n'، میزان آنتروپی واقعی افت می‌کند. این کاهش نشان‌دهنده تمرکز شدید مدل روی بخش‌های خاصی از متن تحلیل‌شده قبلی است.

- در اواخر توالی (عبارت دوم "to be" در کاراکترهای پایانی)، با وجود اینکه ظرفیت آنتروپی به شدت بالا رفته است، اما آنتروپی واقعی افت و خیزهایی را نشان می‌دهد که گویای تخصیص هوشمندانه وزن‌ها به ساختارهای تکراری متناظر قبلی است.

این تحلیل به خوبی نشان می‌دهد که سیستم توجه تحت ماسک علی نه تنها محدودیت‌های توالی را رعایت می‌کند، بلکه از آزادی عمل خود در توکن‌های انتهایی برای تمرکز بر اطلاعات مفید استفاده می‌کند.

۸.۲ تحلیل رفتار توجه: متوسط فاصله بازگشت به عقب (Average Look-Back Distance)

یک ابزار تشخیصی بسیار کارآمد برای تحلیل عمق زمانی یک لایه توجه، محاسبه فاصله متوسط توجه وزن‌دهی شده است. این معیار مشخص می‌کند که هر موقعیت پرس‌وجو (i) به طور متوسط چقدر به عقب نگاه می‌کند تا اطلاعات مورد نیاز خود را استخراج کند. تحت یک ماسک علی، فاصله فیزیکی بین موقعیت پرس‌وجوی i و موقعیت کلید j برابر است با $i - j \geq 0$. با وزن‌دهی این فاصله توسط مقادیر احتمال توجه ($A_{i,j}$)، می‌توانیم متوسط مسافتی که مدل در حافظه خود به عقب بازمی‌گردد را محاسبه کنیم.

فرمولاسیون ریاضی

برای هر موقعیت پرس‌وجوی i در توالی، متوسط فاصله بازگشت به عقب وزن‌دهی شده ($\bar{D}_i$) به صورت زیر تعریف می‌شود:

$$\bar{D}_i = \sum_{j=0}^i A_{i,j} \cdot (i - j)$$

به عنوان یک مرجع مقایسه‌ای، اگر مدل توجه خود را به طور کاملاً یکنواخت بین تمام کلیدهای مرئی توزیع کند ($A_{i,j} = \frac{1}{i+1}$ برای $j \leq i$)، متوسط فاصله بازگشت به عقب به صورت زیر خواهد بود:

$$\bar{D}_{\text{uniform}}(i) = \frac{1}{i+1} \sum_{j=0}^i (i - j) = \frac{1}{i+1} \frac{i(i+1)}{2} = \frac{i}{2}$$

بنابراین، خط مرجع $\frac{i}{2}$ نشان‌دهنده توجه بدون ترجیح (کاملاً یکنواخت) است. انحراف از این خط به سمت بالا یا پایین، ترجیحات ساختاری مدل را آشکار می‌سازد.

پیاده‌سازی در Python و JAX

جهت دستیابی به روندی نرم‌تر و تحلیل مستقل از یک جمله خاص، آزمایش را روی یک توالی تصادفی طولانی‌تر با طول $T_{\text{long}} = 48$ اجرا می‌کنیم:

```
1 # CHART 6 - average (weighted) attention distance as a function of query position.
2 Tlong = 48
3 ids_long = data[1000:1000 + Tlong]
4 x_long = emb[jnp.asarray(ids_long)] + positional_encoding(Tlong, C)
5 _, attn_long = self_attention(x_long, Wq, Wk, Wv)
6
7 A = np.array(attn_long)
8 qpos = np.arange(Tlong)[: , None]
9 kpos = np.arange(Tlong)[None, :]
10 dist = qpos - kpos # >= 0 under the causal mask
11 avg_dist = np.sum(A * dist, axis=-1) # weighted mean distance per query
12
13 fig, ax = plt.subplots(figsize=(10, 3.6))
14 ax.plot(range(Tlong), avg_dist, "-o", color="#0099cc", label="weighted mean look-back distance")
15 ax.plot(range(Tlong), np.arange(Tlong) / 2.0, "--", color="#9999",
16         label="uniform-attention reference (i / 2)")
17
18 ax.set_title("Average attention distance grows with query position")
```



```

19 ax.set_xlabel("query position"); ax.set_ylabel("mean (i - j) over keys"); ax.legend()
20 plt.tight_layout()
21 plt.show()
22

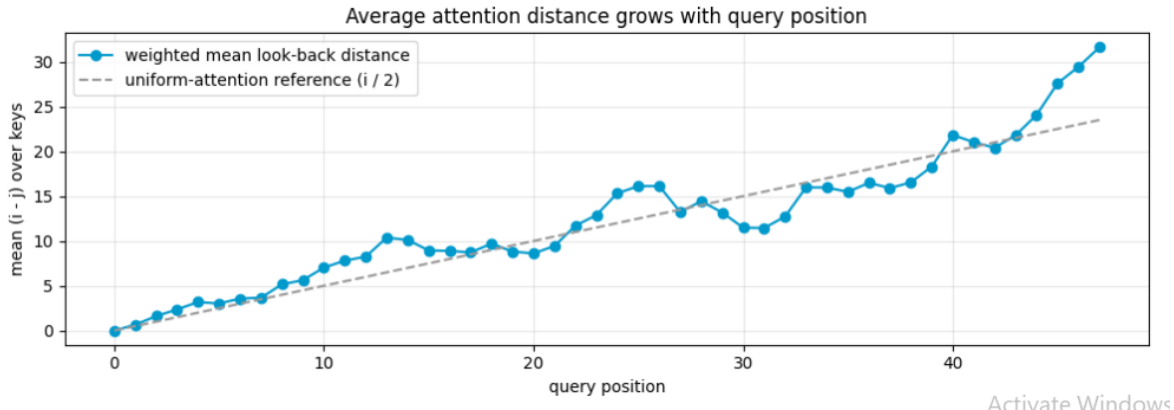
```

توضیح کد پیاده‌سازی:

- `kpos = np.arange(Tlong) [None, :]` و `qpos = np.arange(Tlong)[:, None]` استفاده از مکانیزم broadcasting در کتابخانه NumPy جهت ساخت یک ماتریس فاصله تفاضلی به ابعاد $T \times T$.
- `dist = qpos - kpos`: تولید ماتریس تفاوت نمایه‌ها، به گونه‌ای که درایه (i, j) نشان‌دهنده میزان عقب‌گرد از موقعیت i به j است.
- `np.sum(A * dist, axis=-1)`: ضرب نقطه‌ای درایه‌به‌درایه ماتریس توجه با ماتریس فواصل و سپس کاهش دامنه ستون‌ها (محور آخر) جهت محاسبه میانگین وزن‌دار فاصله برای هر سطر.

تحلیل و ارزیابی خروجی نمودار فاصله متوسط

نمودار خروجی حاصل از اجرای کد بالا در شکل زیر نمایش داده شده است:



شکل ۱۳: متوسط فاصله بازگشت به عقب توجه در مقایسه با مرجع یکنواخت بر روی یک توالی به طول ۴۸ توکن.

تحلیل و تفسیر نتایج نمودار:

- روند عمومی صعودی: همان‌طور که انتظار می‌رفت، با پیشروی در طول توالی (افزایش نمایه پرس‌وجو)، فاصله متوسط توجه افزایش می‌یابد. این امر به دلیل افزایش فضا و گزینه‌های در دسترس برای نگاه به عقب در گام‌های انتهایی توالی است.
- مقایسه با مرجع یکنواخت (خط چین خاکستری $i/2$):

- در بخش‌های آغازین توالی (تا موقعیت ۲۲)، منحنی آبی رنگ بسیار نزدیک به خط چین خاکستری نوسان می‌کند و گاهی بالاتر از آن قرار می‌گیرد. این نشان می‌دهد که مدل در ابتدای متن رفتاری نزدیک به توزیع یکنواخت دارد یا به شدت به اولین کاراکترها (فواصل دورتر) وابسته است.
- در محدوده گام‌های ۲۵ تا ۴۰، افت واضحی در منحنی آبی مشاهده می‌شود که آن را به زیر خط مرجع هدایت می‌کند. این پدیده گویای شکل‌گیری توجه محلی (Local Attention) است؛ یعنی مدل در این بازه ترجیح می‌دهد اطلاعات را بیشتر از کلمات مجاور و نزدیک به خود جمع‌آوری کند تا از نقاط دورتر.
- در انتهای نمودار (گام‌های ۴۳ تا ۴۸)، صعود ناگهانی و شدید منحنی آبی به فراتر از مرز یکنواخت ($i/2$) رخ می‌دهد. این جهش نشان‌دهنده وجود وابستگی‌های دوربرد (Long-range Dependencies) است؛ جایی که توکن‌های پایانی نیاز مبرمی به بازخوانی اطلاعات از توکن‌های اولیه و آغازین توالی دارند.

۳ تپسی - توجه چندسر و بلوک ترنسفورمر

۱.۳ nn.Module Flax (init / apply) و PyTrees پارامترها

کتابخانه Flax یک فریم‌ورک یادگیری عمیق است که بر پایه JAX ساخته شده و کاملاً بر اساس اصول برنامه‌نویسی تابعی (Functional Programming) طراحی شده است. مهم‌ترین ویژگی JAX این است که توابع باید خالص (Pure) باشند؛ یعنی هیچ حالت داخلی (state) در خود نگه ندارند و خروجی فقط به ورودی وابسته باشد.

به همین دلیل، برخلاف فریم‌ورک‌هایی مانند PyTorch که پارامترهای مدل داخل خود شیء شبکه ذخیره می‌شوند، در Flax هیچ پارامتری داخل object نگه‌داری نمی‌شود. در عوض، تمام وزن‌ها در یک ساختار داده‌ای خارجی به نام PyTree ذخیره می‌شوند.

PyTree چیست؟

یک PyTree ساختاری درختی از داده‌هاست که می‌تواند شامل دیکشنری‌ها، لیست‌ها و آرایه‌ها باشد. در زمینه شبکه‌های عصبی، PyTree معمولاً به شکل زیر نمایش داده می‌شود:

```
{'params': {...}}
```

این ساختار شامل تمام وزن‌های شبکه است و به صورت صریح به توابع داده می‌شود، نه اینکه داخل مدل پنهان باشد. این طراحی باعث می‌شود JAX بتواند به راحتی روی مدل‌ها عملیات زیر را انجام دهد:

- مشتق‌گیری خودکار (autograd)
- کامپایل سریع (jit compilation)
- اجرای موازی و بهینه‌سازی

دو مرحله اصلی در Flax

کار با یک مدل در Flax فقط شامل دو مرحله اصلی است:



۱. مرحله Initialization (init) در این مرحله تابع زیر اجرا می‌شود:

```
module.init(rng, sample_input)
```

این تابع یک بار مدل را اجرا می‌کند، اما نه برای پیش‌بینی؛ بلکه برای اینکه ساختار کامل شبکه مشخص شود. در این مرحله موارد زیر تعیین می‌شوند:

- شکل (shape) تمام وزن‌ها
- تعداد پارامترها در هر لایه
- ساختار کامل شبکه

خروجی این مرحله یک PyTree ثابت است که معمولاً به شکل زیر است:

```
{'params': {...}}
```

این مرحله به یک کلید تصادفی یا PRNG key نیاز دارد. این کلید تعیین می‌کند مقداردهی اولیه وزن‌ها چگونه انجام شود. اگر یک PRNG key یکسان دوبار استفاده شود، خروجی دقیقاً یکسان خواهد بود. این ویژگی باعث قابل بازتولید بودن (Repro-ducibility) می‌شود.

۲. مرحله Apply (apply) پس از ساخته شدن پارامترها، از تابع زیر استفاده می‌شود:

```
module.apply(params, input)
```

در این مرحله:

- پارامترها به صورت صریح به مدل داده می‌شوند
- محاسبات forward انجام می‌شود
- خروجی شبکه تولید می‌شود

نکته مهم این است که در این مرحله هیچ پارامتری داخل مدل ذخیره نشده است و همه چیز از بیرون کنترل می‌شود.

تعریف لایه‌ها در Flax

در یک ماژول Flax nn.Module، هایپرپارامترها به صورت فیلدهای کلاس تعریف می‌شوند. برای مثال:

```
n_head : int
```

سپس تابع forward در داخل متدی با دکوراتور زیر نوشته می‌شود:

```
@nn.compact
```

در این حالت لایه‌ها به صورت inline ساخته می‌شوند، مانند:

```
nn.Dense(features=128)
```

این طراحی باعث می‌شود ساخت مدل بسیار خوانا، ماژولار و مطابق سبک تابعی باشد.



جمع‌بندی

در Flax:

- مدل‌ها state داخلی ندارند
- تمام پارامترها در PyTree خارجی ذخیره می‌شوند
- تابع init فقط ساختار و وزن‌ها را تولید می‌کند
- تابع apply فقط محاسبات forward را انجام می‌دهد

این معماری یکی از مهم‌ترین تفاوت‌های Flax با فریم‌ورک‌هایی مانند PyTorch است و دلیل اصلی سازگاری کامل آن با اکوسیستم JAX محسوب می‌شود.

۲.۳ Attention Multi-Head (توجه چندسری)

مکانیزم Attention یکی از مهم‌ترین اجزای معماری Transformer است که به مدل اجازه می‌دهد وابستگی بین توکن‌های مختلف یک دنباله را یاد بگیرد. با این حال، یک مشکل اساسی وجود دارد: یک attention head تنها می‌تواند یک نوع الگوی ارتباطی را در داده مدل کند.

برای حل این محدودیت، از Attention Multi-Head استفاده می‌شود که چندین head را به صورت موازی اجرا می‌کند. این کار باعث می‌شود مدل بتواند به طور همزمان چندین نوع رابطه متفاوت را در یک دنباله یاد بگیرد.

ایده اصلی

در Multi-Head Attention به جای استفاده از یک فضای واحد برای نمایش Query، Key و Value، فضای نمایش به چند زیرفضا تقسیم می‌شود. هر head در یک زیر-فضای مستقل کار می‌کند و الگوهای متفاوتی را یاد می‌گیرد. به طور شهودی، هر head می‌تواند به نوع خاصی از وابستگی‌ها توجه کند، مانند:

- وابستگی‌های کوتاه‌برد (local dependencies)
- وابستگی‌های بلندبرد (long-range dependencies)
- روابط نحوی (syntactic relations)
- روابط معنایی (semantic relations)

تقسیم فضای embedding

فرض کنید بعد embedding برابر C باشد. در Multi-Head Attention این بعد به n_{head} بخش تقسیم می‌شود:

$$d_{\text{head}} = \frac{C}{n_{\text{head}}}$$

که در آن:



• C : بعد کل embedding

• n_{head} : تعداد هاhead

• d_{head} : بعد هر head

فرمول Attention در هر head

در هر head عملیات attention به صورت زیر انجام می شود:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_{\text{head}}}} \right) V$$

که در آن:

• Q : ماتریس Query

• K : ماتریس Key

• V : ماتریس Value

• d_{head} : مقیاس نرمال سازی برای جلوگیری از بزرگ شدن مقادیر

مراحل محاسبات Attention Multi-Head

۱. تولید Q, K, V ابتدا یک لایه خطی مشترک روی ورودی اعمال می شود:

$$X \in \mathbb{R}^{T \times C}$$

$$\text{Linear}(X) \rightarrow \mathbb{R}^{T \times 3C}$$

سپس خروجی به سه بخش تقسیم می شود:

$$Q, K, V \in \mathbb{R}^{T \times C}$$

۲. جدا کردن هاhead

سپس reshape tensor می شوند تا بعد head جدا شود:

$$(T, C) \rightarrow (T, n_{\text{head}}, d_{\text{head}})$$

و سپس معمولاً به شکل زیر تبدیل می شود:



$$(n_{\text{head}}, T, d_{\text{head}})$$

این تغییر شکل باعث می‌شود هر head به صورت مستقل پردازش شود.

۳. محاسبه attention مستقل برای هر head

برای هر head به صورت جداگانه داریم:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

این محاسبه برای تمام headها به صورت موازی انجام می‌شود.

۴. اتصال خروجی headها

پس از محاسبه attention در تمام headها، خروجی‌ها دوباره به هم متصل می‌شوند:

$$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_n) \in \mathbb{R}^{T \times C}$$

۵. projection نهایی

در نهایت یک لایه خطی برای ترکیب اطلاعات headها اعمال می‌شود:

$$Y = W_o \cdot \text{Concat}(\text{head}_1, \dots, \text{head}_n)$$

که در آن:

• W_o : ماتریس وزن خروجی

• Y : خروجی نهایی Attention Multi-Head

مزایای Attention Multi-Head

استفاده از چند head مزایای مهمی دارد:

- هر head می‌تواند یک نوع رابطه متفاوت را یاد بگیرد
- مدل چندین نمای متفاوت از داده ایجاد می‌کند
- وابستگی‌های پیچیده بهتر مدل می‌شوند
- قدرت بیان مدل افزایش می‌یابد بدون افزایش شدید هزینه محاسباتی



جمع‌بندی

Multi-Head Attention با تقسیم فضای embedding به چند زیرفضا و اجرای attention مستقل در هر کدام، به مدل اجازه می‌دهد چندین نوع رابطه را به صورت همزمان یاد بگیرد. این مکانیزم یکی از کلیدی‌ترین اجزای موفقیت معماری Transformer است.

۳.۳ GELU و MLP در هر موقعیت (Position-wise MLP)

در معماری Transformer، لایه‌های Attention وظیفه انتقال اطلاعات بین توکن‌ها را بر عهده دارند. با این حال، برای اینکه هر توکن بتواند به صورت مستقل نیز پردازش غیرخطی انجام دهد، از یک شبکه کاملاً جداگانه به نام MLP Position-wise استفاده می‌شود. این بخش روی هر توکن به صورت مستقل اعمال می‌شود، یعنی هیچ تبادل اطلاعاتی بین توکن‌ها در این مرحله انجام نمی‌شود.

ساختار کلی MLP

شبکه MLP در هر بلوک Transformer معمولاً شامل سه مرحله است:

- یک لایه خطی برای افزایش ابعاد (Expansion)
- یک تابع غیرخطی (Activation Function)
- یک لایه خطی برای کاهش ابعاد (Projection)

اگر ورودی را $x \in \mathbb{R}^C$ در نظر بگیریم، ساختار به صورت زیر است:

$$x \rightarrow W_1 x + b_1$$

$$\rightarrow \phi(\cdot)$$

$$\rightarrow W_2(\cdot) + b_2$$

که در آن:

$$W_1 \in \mathbb{R}^{4C \times C} \quad \bullet$$

$$W_2 \in \mathbb{R}^{C \times 4C} \quad \bullet$$

$$\phi: \text{تابع فعال‌سازی} \quad \bullet$$



چرا ضرب ۴ برابر؟

در طراحی استاندارد Transformer، بعد لایه مخفی MLP معمولاً ۴ برابر بعد embedding انتخاب می‌شود:

$$C \rightarrow 4C \rightarrow C$$

این افزایش باعث می‌شود:

- هر توکن فضای بیشتری برای ترکیب ویژگی‌ها داشته باشد
- مدل توانایی نمایش غیرخطی بیشتری پیدا کند
- تعامل بین ویژگی‌ها غنی‌تر شود

تابع GELU

تابع فعال‌سازی مورد استفاده در Transformer معمولاً GELU است. این تابع به صورت زیر تعریف می‌شود:

$$\text{GELU}(x) = x \cdot \Phi(x)$$

که در آن:

- $\Phi(x)$: تابع توزیع تجمعی نرمال استاندارد (Gaussian CDF)

تعبیر شهودی GELU

برخلاف تابع ReLU که مقادیر منفی را کاملاً صفر می‌کند، تابع GELU یک رفتار نرم و احتمالاتی دارد:

- ورودی‌های بزرگ مثبت تقریباً بدون تغییر عبور می‌کنند
 - ورودی‌های نزدیک صفر به صورت تدریجی فیلتر می‌شوند
 - ورودی‌های منفی کوچک به مقدار کمی عبور داده می‌شوند
- این ویژگی باعث می‌شود جریان اطلاعات در شبکه نرم‌تر و پایدارتر باشد.

فرم تقریبی GELU

در عمل، GELU معمولاً با یک تقریب محاسباتی استفاده می‌شود:

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

این نسخه برای پیاده‌سازی عددی سریع‌تر استفاده می‌شود.



نقش MLP در Transformer

در حالی که Attention وظیفه انتقال اطلاعات بین توکن‌ها را دارد، MLP وظیفه پردازش داخلی هر توکن را بر عهده دارد. به عبارت دیگر:

- Attention: ارتباط بین موقعیت‌ها (Inter-token communication)
 - MLP: پردازش ویژگی‌ها داخل هر توکن (Intra-token computation)
- این دو مکمل یکدیگر هستند و بدون یکی، قدرت مدل به شدت کاهش می‌یابد.

جمع‌بندی

بخش MLP در Transformer با استفاده از افزایش ابعاد و تابع غیرخطی GELU، به هر توکن اجازه می‌دهد ویژگی‌های پیچیده‌تری را یاد بگیرد. این بخش نقش کلیدی در افزایش ظرفیت مدل برای یادگیری روابط غیرخطی دارد.

۴.۳ Layer Normalization (نرمال‌سازی لایه‌ای)

Normalization Layer یکی از مهم‌ترین اجزای معماری Transformer است که نقش اصلی آن پایدارسازی آموزش شبکه‌های عمیق است. این لایه باعث می‌شود توزیع مقادیر در طول شبکه کنترل شود و از انفجار یا محوشدن گرادیان جلوگیری گردد. برخلاف روش‌هایی مانند Batch Normalization، در Layer Norm هیچ وابستگی‌ای به اندازه batch وجود ندارد و نرمال‌سازی برای هر نمونه به صورت مستقل انجام می‌شود.

ایده اصلی Layer Norm

در Layer Norm، نرمال‌سازی روی ویژگی‌های هر توکن انجام می‌شود، نه روی batch. فرض کنید بردار ویژگی یک توکن به صورت زیر باشد:

$$x \in \mathbb{R}^C$$

در این حالت، ابتدا میانگین و واریانس روی بعد ویژگی‌ها محاسبه می‌شود.

محاسبه میانگین

$$\mu = \frac{1}{C} \sum_{i=1}^C x_i$$



محاسبه واریانس

$$\sigma^2 = \frac{1}{C} \sum_{i=1}^C (x_i - \mu)^2$$

نرمال سازی

سپس مقدار نرمال شده به صورت زیر محاسبه می شود:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

که در آن:

- ϵ : یک مقدار کوچک برای جلوگیری از تقسیم بر صفر

پارامترهای قابل یادگیری

پس از نرمال سازی، دو پارامتر قابل یادگیری به داده اعمال می شود:

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

که در آن:

- $\gamma \in \mathbb{R}^C$: پارامتر scale

- $\beta \in \mathbb{R}^C$: پارامتر shift

این دو پارامتر به مدل اجازه می دهند در صورت نیاز، توزیع دلخواه خود را دوباره بازسازی کند.

تفاوت LayerNorm و BatchNorm

یکی از تفاوت های مهم این دو روش در نحوه نرمال سازی است:

- BatchNorm: نرمال سازی روی بعد batch انجام می شود
- LayerNorm: نرمال سازی روی ویژگی های هر نمونه انجام می شود

مزایای LayerNorm:

- مستقل از size batch
- مناسب برای مدل های دنباله ای
- بدون وابستگی به ترتیب نمونه ها
- مناسب برای مدل های autoregressive



چرا LayerNorm برای Transformer مناسب است؟

در مدل‌های زبانی، هر توکن باید بدون دیدن آینده پردازش شود. بنابراین روش‌هایی که به batch وابسته هستند مناسب نیستند. LayerNorm این مشکل را حل می‌کند زیرا:

- فقط روی ویژگی‌های همان توکن عمل می‌کند
- هیچ اطلاعاتی از توکن‌های دیگر استفاده نمی‌کند

نکته پیاده‌سازی

در پیاده‌سازی‌های Flax، معمولاً LayerNorm دقیقاً مطابق فرمول زیر پیاده‌سازی می‌شود:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

که در آن $\odot$ ضرب عنصر به عنصر است.

جمع‌بندی

LayerNorm با نرمال‌سازی ویژگی‌های هر توکن به صورت مستقل، باعث پایداری آموزش و بهبود جریان گرادیان در شبکه‌های عمیق می‌شود. این لایه یکی از اجزای حیاتی در معماری Transformer محسوب می‌شود و بدون آن آموزش مدل‌های بزرگ عملاً دشوار خواهد بود.

۵.۳ اتصالات باقیمانده (Residual Connections) و Pre-Norm در Transformer

در شبکه‌های عصبی عمیق، یکی از بزرگ‌ترین مشکلات هنگام آموزش، ناپایداری گرادیان است. با افزایش عمق شبکه، گرادیان ممکن است یا محو شود (vanishing gradient) یا بیش از حد بزرگ شود (exploding gradient). برای حل این مشکل، از Con-Residual nection استفاده می‌شود.

Residual Connection چیست؟

ایده اصلی در اتصال باقیمانده بسیار ساده اما قدرتمند است. به جای اینکه فقط خروجی یک تابع $f(x)$ را داشته باشیم، ورودی را نیز به آن اضافه می‌کنیم:

$$y = x + f(x)$$

در این ساختار:

- x : ورودی لایه
- $f(x)$: خروجی لایه (مثلاً Attention یا MLP)



• y : خروجی نهایی

این اتصال یک مسیر مستقیم برای عبور گرادیان ایجاد می‌کند و باعث می‌شود آموزش شبکه‌های بسیار عمیق ممکن شود.

مزیت اصلی Residual Connection

مزیت کلیدی این ساختار این است که:

- گرادیان می‌تواند بدون تغییر از مسیر $x \rightarrow y$ عبور کند
 - شبکه مجبور نیست کل تابع را از صفر یاد بگیرد
 - یادگیری به صورت اصلاحی (residual learning) انجام می‌شود
- به بیان ساده، مدل فقط یاد می‌گیرد چه چیزی را به ورودی اضافه کند، نه اینکه کل تبدیل را دوباره بسازد.

محل قرارگیری LayerNorm

یکی از تصمیم‌های مهم در طراحی Transformer، محل قرارگیری Layer Normalization است. دو معماری اصلی وجود دارد:

۱. Post-Norm (نسخه اولیه Transformer)

در این حالت، ابتدا لایه محاسباتی و سپس residual و در نهایت normalization اعمال می‌شود:

$$x \rightarrow x + f(x)$$

$$y = \text{LayerNorm}(x)$$

این ساختار در نسخه اولیه Transformer استفاده شده بود، اما در مدل‌های بزرگ پایدار نبود.

۲. Pre-Norm (نسخه مدرن)

در معماری‌های جدید مانند GPT، ابتدا نرمال‌سازی انجام می‌شود و سپس وارد لایه می‌شویم:

$$y = x + f(\text{LayerNorm}(x))$$

در این حالت:

- مسیر residual بدون تغییر باقی می‌ماند
- گرادیان راحت‌تر عبور می‌کند
- آموزش شبکه‌های بسیار عمیق پایدارتر می‌شود



ساختار یک بلوک Pre-Norm Transformer

یک بلوک استاندارد Transformer در حالت Pre-Norm به شکل زیر است:

۱. بلوک Attention

$$x = x + \text{Attention}(\text{LayerNorm}(x))$$

۲. بلوک MLP

$$x = x + \text{MLP}(\text{LayerNorm}(x))$$

این دو مرحله پشت سر هم یک بلوک کامل Transformer را تشکیل می‌دهند.

چرا Pre-Norm بهتر است؟

مزایای اصلی Pre-Norm عبارت‌اند از:

- پایداری بهتر در شبکه‌های عمیق
- جلوگیری از انفجار یا محوشدن گرادیان
- بهبود سرعت همگرایی
- مناسب برای مدل‌های بزرگ مانند GPT

در عمل، Pre-Norm به استاندارد اصلی در معماری‌های مدرن Transformer تبدیل شده است.

جمع‌بندی

اتصالات Residual با ایجاد مسیر مستقیم برای جریان گرادیان، امکان آموزش شبکه‌های عمیق را فراهم می‌کنند. ترکیب این ساختار با Pre-Norm باعث شده‌های Transformer مدرن بتوانند بدون مشکل در مقیاس‌های بسیار بزرگ آموزش داده شوند.

۶.۳ توجه علی

پیش از آنکه به سراغ ساختار پیچیده‌تر «توجه چندسر» (Multi-Head Attention) برویم، بسیار مهم است که رفتار یک (Single Causal Attention Head) را بار دیگر مرور و تثبیت کنیم. در این بخش، با استفاده از توابع scaled_dot_product_attention و causal_mask که در مراحل قبل توسعه دادیم، بردارهای تصادفی پرس‌وجو (Query)، کلید (Key) و مقدار (Value) را برای یک دنباله تولید کرده و ماتریس توجه حاصل از آن‌ها را رسم می‌کنیم.

ساختار «توجه چندسر» که در گام‌های بعدی پیاده‌سازی می‌شود، در واقع چیزی نیست جز اجرای همزمان و موازی چندین مورد از همین بلوک‌های تک‌سر.

تحلیل متغیرهای مدل (D_{head} و T)

در پیکربندی این بخش، دو متغیر اساسی با مقادیر مشخص تعریف شده‌اند که ابعاد محاسبات ما را تعیین می‌کنند:

- متغیر $T = 12$ (طول دنباله یا Sequence Length): این متغیر نشان می‌دهد که جمله یا دنباله ورودی ما در این تست شامل ۱۲ توکن (کلمه یا کاراکتر) است. به همین دلیل، ماتریس پوشش (Mask) و ماتریس نهایی وزن‌های توجه (Attention Matrix)، هر دو ابعاد 12×12 خواهند داشت تا بتوانند روابط تمام این ۱۲ توکن را با یکدیگر نشان دهند.
- متغیر $D_{\text{head}} = 8$ (ابعاد سر توجه یا Head Dimension): این عدد مشخص می‌کند که هر توکن، برای انجام عملیات توجه، به یک بردار با ۸ ویژگی (۸ عدد) تبدیل شده است. به بیان دیگر، بردارهای پرس‌وجو (q) و کلید (k) در فضایی ۸-بعدی با هم مقایسه می‌شوند. این همان عددی است که در فرمول مقیاس‌بندی، امتیازات خام را بر جذر آن (یعنی $\sqrt{8}$) تقسیم می‌کنیم تا واریانس کنترل شود.

تحلیل مجموع سطرهای ماتریس توجه (Attention Row Sums)

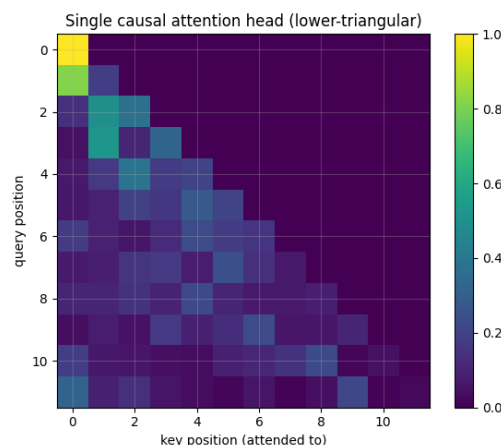
در خروجی اجرای کد این بخش، با پیام زیر مواجه می‌شویم:

```
1 attention row sums (should be 1): [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
2
```

معنای این خروجی چیست؟ این آرایه ۱۲ تایی ثابت می‌کند که تابع Softmax به درستی روی هر سطر از ماتریس توجه اعمال شده است. تابع Softmax امتیازات خام (Logits) را به «توزیع احتمال» تبدیل می‌کند. قانون بنیادین احتمالات این است که مجموع احتمالات تمام حالت‌های ممکن باید دقیقاً برابر با ۱ (یا ۱۰۰٪) باشد.

تحلیل بصری ماتریس توجه تک‌سر

شکل ۱۴ نمایشی از وزن‌های توجه برای این دنباله ۱۲ تایی با وزن‌های تصادفی است.



شکل ۱۴: نقشه حرارتی یک سر توجه علی (Single Causal Attention Head). ساختار پایین‌مثلثی، نشان‌دهنده اعمال صحیح محدودیت‌های علی است.

کالبدشکافی تصویر:

۱. رعایت دقیق محدودیت علی (Causal Constraint): بارزترین ویژگی این تصویر، ساختار اکیداً پایین‌مثلی (Strict Lower-Triangular) آن است. تمامی خانه‌های بالای قطر اصلی کاملاً سیاه (دارای احتمال صفر) هستند. این نشان می‌دهد که توکن موجود در جایگاه t (در محور عمودی پرس‌وجو)، تنها اجازه دارد به توکن‌های جایگاه 0 تا t (در محور افقی کلیدها) توجه کند و دسترسی به آینده مسدود است.

۲. توزیع احتمالات در وزن‌های تصادفی: از آنجایی که در این تست از وزن‌های تصادفی و آموزش‌ندیده (Random Init) استفاده شده است، توزیع رنگ‌ها در نیمه پایینی مثلث، الگوهای معنایی خاصی ندارد. با این حال، به وضوح می‌بینیم که با حرکت از سطرهاى بالایی به سطرهاى پایینی (افزایش طول زمینه)، توکن‌ها توجه خود را روی گزینه‌های بیشتری پخش می‌کنند (رنگ‌ها از زرد خالص به سمت سبز و آبی متمایل می‌شوند که نشان‌دهنده تقسیم شدن مقدار ۱ بین خانه‌های بیشتر است).

۳. تمرکز اجباری در نقطه شروع: در سطر اول (مختصات ۰، ۰)، مربع زرد رنگ و درخشانی را می‌بینیم (مقدار ۰.۱). دلیل این امر آن است که اولین توکن هیچ کلمه‌ای را پیش از خود ندارد؛ بنابراین تابع Softmax تمام وزن توجه را به ناچار روی همان یک توکن متمرکز می‌کند.

۷.۳ آشنایی با جریان کاری Flax: الگوی Init / Apply

پیش از آنکه به سراغ ساخت بلوک‌های پیچیده ترنسفورمر و توجه چندسر برویم، ضروری است که با نحوه عملکرد و جریان کاری کتابخانه Flax (که بر پایه JAX ساخته شده است) آشنا شویم. معماری Flax تفاوت بنیادینی با کتابخانه‌هایی مانند PyTorch دارد. در PyTorch، وزن‌های شبکه (پارامترها) در داخل خود کلاس مدل ذخیره می‌شوند (مدل‌های حالت‌دار یا Stateful). اما در Flax، شبکه‌های عصبی به صورت کاملاً بدون حالت (Stateless) تعریف می‌شوند. این یعنی کلاس مدل صرفاً شامل «قوانین محاسباتی» است و وزن‌ها به عنوان یک متغیر کاملاً مجزا در خارج از مدل (در ساختاری به نام Pytree) نگهداری می‌شوند. این جداسازی از طریق الگوی Init / Apply پیاده‌سازی می‌شود که در ادامه آن را روی یک شبکه عصبی بسیار کوچک (TinyMLP) بررسی می‌کنیم.

پیاده‌سازی یک ماژول کوچک (TinyMLP)

در این مثال، یک شبکه عصبی بسیار ساده با یک لایه خطی متراکم (Dense) و یک تابع فعال‌ساز GELU می‌سازیم تا رفتار Flax را مشاهده کنیم:

```

1  # GIVEN: the Flax init/apply pattern on a one-layer module.
2  class TinyMLP(nn.Module):
3      hidden: int
4
5      @nn.compact
6      def __call__(self, x):
7          x = nn.Dense(self.hidden, name="fc")(x)
8          return nn.gelu(x)
9
10     tiny = TinyMLP(hidden=8)
    
```



```

11 demo_x = jnp.ones((2, 4)) # batch of 2, feature width 4
12 tiny_params = tiny.init(jax.random.PRNGKey(0), demo_x) # deterministic on CPU
13
14 print("parameter pytree (name -> shape):")
15 shapes = jax.tree_util.tree_map(lambda a: a.shape, tiny_params)
16 print(shapes)
17
18 y = tiny.apply(tiny_params, demo_x) # run the forward pass with those params
19 print("apply output shape:", y.shape)
20 print("kernel shape:", tiny_params["params"]["fc"]["kernel"].shape,
21       "| bias shape:", tiny_params["params"]["fc"]["bias"].shape)
22

```

کالبدشکافی کد و دکوراتور `@nn.compact`

این قطعه کد مفاهیم بسیار مهمی از معماری Flax را در خود جای داده است:

۱. دکوراتور جادویی `@nn.compact`: در حالت استاندارد و کلاسیک (مانند PyTorch)، شما باید لایه‌های شبکه (مثل `nn.Dense`) را در تابعی به نام `__init__` یا `setup` تعریف کنید و سپس در تابع `__call__` (یا `forward`) آن‌ها را فراخوانی کنید. اما دکوراتور `@nn.compact` در Flax این فرایند را به شدت ساده می‌کند. این دکوراتور به شما اجازه می‌دهد لایه‌های فرعی (Submodules) را مستقیماً در لحظه استفاده و درون خود تابع `__call__` تعریف کنید. Flax به صورت خودکار در اولین باری که کد اجرا می‌شود، ابعاد ورودی را بررسی کرده و لایه‌ها را ثبت می‌کند. این کار باعث می‌شود کدهای شما بسیار خواناتر، کوتاه‌تر و شبیه به توابع ریاضی نوشته شوند.

۲. تابع `init` (تولید وزن‌ها): دستور `tiny.init(jax.random.PRNGKey(0), demo_x)` دقیقاً نشان‌دهنده ماهیت بدون حالت Flax است. ما به تابع `init` دو چیز می‌دهیم:

- یک کلید تصادفی (`PRNGKey`) برای مقداردهی اولیه وزن‌ها به صورت قطعی و تکرارپذیر.

- یک داده آزمایشی یا ساختگی (`demo_x`) که در اینجا یک ماتریس 2×4 است.

مدل با دریافت `demo_x`، یک بار به صورت آزمایشی مسیر محاسبات را طی می‌کند تا متوجه شود لایه `Dense` به چه ابعادی برای ماتریس وزن‌های خود نیاز دارد. سپس وزن‌ها را تولید کرده و آن‌ها را در قالب یک دیکشنری درختی (Pytree) برمی‌گرداند (`tiny_params`). خود متغیر `tiny` همچنان خالی از هرگونه وزن است.

۳. تابع `apply` (اجرای شبکه): برای اجرای مدل روی داده‌ها (Forward Pass)، نمی‌توانیم مستقیماً مدل را صدا بزنیم. بلکه باید از دستور `tiny.apply` استفاده کنیم و وزن‌هایی که در مرحله قبل تولید شده بودند (`tiny_params`) را به همراه داده‌های ورودی (`demo_x`) به مدل تحویل دهیم تا محاسبات انجام شود.



تحلیل خروجی‌ها و ساختار Pytree

با اجرای کد، خروجی‌های زیر در کنسول چاپ می‌شوند:

```
1 parameter pytree (name -> shape):
2 {'params': {'fc': {'bias': (8,), 'kernel': (4, 8)}}}
3 apply output shape: (2, 8)
4 kernel shape: (4, 8) | bias shape: (8,)
5
```

تحلیل ساختار وزن‌ها (Pytree): خروجی اول نشان می‌دهد که پارامترها در یک ساختار درختی و تودرتو ذخیره شده‌اند. این درخت دقیقاً منطبق بر نام‌گذاری‌هایی است که ما در کد انجام داده‌ایم:

- 'fc': این کلید، نامی است که ما به لایه Dense داده بودیم ("name='fc'").
- 'kernel' (4, 8): این همان ماتریس وزن‌های (W) ما در معادله خطی $y = xW + b$ است. چون ورودی ما ۴ ویژگی داشت (feature width = 4) و خروجی لایه مخفی را ۸ در نظر گرفتیم ($hidden = 8$)، ابعاد این ماتریس به صورت خودکار 4×8 تشخیص داده شده و ساخته شده است.
- 'bias' (8,): بردار بایاس (b) که به ازای هر نورون خروجی در لایه مخفی، یک عدد دارد.

تحلیل ابعاد خروجی: ابعاد خروجی تابع apply برابر با (2, 8) است. عدد ۲ نشان‌دهنده اندازه دسته ورودی (Batch Size) است که ما به مدل داده بودیم، و عدد ۸ نیز نمایانگر تعداد نورون‌های فعال‌شده در لایه پنهان پس از عبور از تابع GELU است. درک این جریان کاری (Pytree، init و apply) برای ادامه کار و ساخت بلوک‌های پیچیده ترنسفورمر بسیار حیاتی است، زیرا تمامی وزن‌های مدل نهایی GPT ما نیز در نهایت در یک ساختار درختی مشابه اما بسیار بزرگتر ذخیره خواهند شد.

۸.۳ طراحی ماژول توجه چندسر علی (Causal Multi-Head Self-Attention) در Flax

در گام نهایی از پیاده‌سازی مکانیزم توجه، آموخته‌های پیشین (الگوی init/apply در Flax، ماسک علی و مقیاس‌بندی توجه) را در قالب یک ماژول کامل و مقیاس‌پذیر به نام CausalSelfAttention ترکیب می‌کنیم. این ماژول یکی از دو رکن اصلی در معماری Transformer (در کنار شبکه‌های عصبی پیش‌خور) محسوب می‌شود. یکی از نکات مهندسی بسیار مهم در کد این بخش، استفاده از یک لایه متراکم (Dense) یکپارچه به جای سه لایه مجزا برای تولید بردارهای Q ، K و V است.



چرا از یک تصویرسازی یکپارچه برای QKV استفاده می‌کنیم؟

به جای تعریف سه لایه خطی جداگانه:

$$W_q \in \mathbb{R}^{C \times C}$$

$$W_k \in \mathbb{R}^{C \times C}$$

$$W_v \in \mathbb{R}^{C \times C}$$

در این پیاده‌سازی، از یک لایه متراکم واحد با ابعاد خروجی $3 \times C$ استفاده می‌شود:

$$W_{qkv} \in \mathbb{R}^{C \times 3C} \quad (۱)$$

از نظر ریاضی، ضرب یک ماتریس در سه بلوک به هم چسبیده، کاملاً معادل با سه ضرب ماتریسی مجزا است. با این حال، کامپایلرهای شتاب‌دهنده‌های سخت‌افزاری (مانند XLA در پردازنده‌های گرافیکی و TPU) عملکرد بسیار بهینه‌تری روی یک ضرب ماتریسی بزرگ نشان می‌دهند تا سه ضرب ماتریسی کوچک. این ادغام (Fusion) هزینه‌های راه‌اندازی و انتقال داده در حافظه را به شدت کاهش می‌دهد، به همین دلیل در کدهای سطح تولید (Production-level) تمام مدل‌های زبانی بزرگ، از همین تکنیک استفاده می‌شود.

پیاده‌سازی و کالبدشکافی کد ماژول

در ادامه، بلوک کامل توجه چندسره نوشته شده در فریم‌ورک Flax را به همراه تحلیل خط به خط آن بررسی می‌کنیم.

```

1  class CausalSelfAttention(nn.Module):
2      n_head: int
3      n_embd: int
4      dropout: float = 0.0
5
6      @nn.compact
7      def __call__(self, x, deterministic=True):
8          B, T, C = x.shape
9          d_head = C // self.n_head
10
11         # 1. Shared projection to Q, K, V and split
12         qkv = nn.Dense(3 * C, use_bias=False, name="qkv")(x)
13         q, k, v = jnp.split(qkv, 3, axis=-1)
14
15         # 2. Reshape into heads: (B, n_head, T, d_head)
16         def split_heads(t):
17             return t.reshape(B, T, self.n_head, d_head).transpose(0, 2, 1, 3)
18         q, k, v = split_heads(q), split_heads(k), split_heads(v)
19
20         # 3. Scaled dot-product attention
21         scores = (q @ jnp.swapaxes(k, -1, -2)) / jnp.sqrt(d_head)
22         mask = jnp.tril(jnp.ones((T, T), dtype=bool))
23         scores = jnp.where(mask, scores, -1e9)

```



```

24
25     attn = jax.nn.softmax(scores, axis=-1)
26     attn = nn.Dropout(self.dropout)(attn, deterministic=deterministic)
27
28     # 4. Weighted sum, recombine heads, output projection
29     out = attn @ v
30     out = out.transpose(0, 2, 1, 3).reshape(B, T, C)
31     out = nn.Dense(C, name="proj")(out)
32     out = nn.Dropout(self.dropout)(out, deterministic=deterministic)
33
34     return out
35

```

تحلیل بخش‌های کلیدی کد:

۱. متغیر **deterministic** و لایه‌های **Dropout**:

متغیر **deterministic** مشخص می‌کند که مدل در حالت آموزش (**False**) یا ارزیابی/تولید متن (**True**) قرار دارد. در حالت آموزش، لایه‌های **Dropout** به صورت تصادفی بخشی از فعال‌سازی‌ها را در ماتریس توجه (**attn**) و خروجی نهایی (**out**) صفر می‌کنند تا از بیش‌برازش (**overfitting**) جلوگیری شود. در حالت ارزیابی، این رفتار تصادفی کاملاً غیرفعال می‌شود و شبکه به صورت **deterministic** عمل می‌کند.

۲. استخراج و تغییر شکل سرها (**split heads**):

پس از تقسیم ماتریس **qkv** با استفاده از **jnp.split**، هر بخش دارای ابعاد:

$$(B, T, C)$$

است.

برای اجرای مستقل **attention** روی هر **head**، مراحل زیر انجام می‌شود:

- مرحله اول (**reshape**):

$$(B, T, C) \rightarrow (B, T, n_{\text{head}}, d_{\text{head}})$$

- مرحله دوم (**transpose**):

$$(B, T, n_{\text{head}}, d_{\text{head}}) \rightarrow (B, n_{\text{head}}, T, d_{\text{head}})$$

این جابه‌جایی بسیار مهم است؛ زیرا عملیات ضرب ماتریسی (**@**) در **NumPy** روی دو محور آخر انجام می‌شود.

در نتیجه، ضرب:

$$QK^T$$

روی ابعاد:

$$(T, d_{\text{head}})$$



انجام می‌شود، در حالی که ابعاد:

$$(B, n_{\text{head}})$$

به صورت batch و کاملاً موازی پردازش می‌شوند.

۳. ترکیب مجدد سرها (recombine):

پس از محاسبه attention خروجی دارای شکل زیر است:

$$(B, n_{\text{head}}, T, d_{\text{head}})$$

برای بازگرداندن به فرم اولیه:

• ابتدا transpose انجام می‌شود:

$$(B, n_{\text{head}}, T, d_{\text{head}}) \rightarrow (B, T, n_{\text{head}}, d_{\text{head}})$$

• سپس reshape انجام می‌شود:

$$(B, T, n_{\text{head}} \cdot d_{\text{head}}) \rightarrow (B, T, C)$$

در نهایت خروجی به لایه projection نهایی (nn.Dense) داده می‌شود.

تأیید صحت عملکرد و تست ابعاد (Self-check)

برای بررسی صحت پیاده‌سازی، یک تست ساده روی مدل اجرا می‌شود:

```
# Self-check: shapes and qkv kernel
_x = jax.random.normal(jax.random.PRNGKey(3), (4, 16, 32))

mha_check = CausalSelfAttention(n_head=4, n_embd=32)
params = mha_check.init(jax.random.PRNGKey(0), _x)
_out = mha_check.apply(params, _x)

assert _out.shape == (4, 16, 32)
assert params["params"]["qkv"]["kernel"].shape == (32, 96)

print("OK:", _out.shape, "| qkv kernel:", params["params"]["qkv"]["kernel"].shape)
print("heads:", 4, "-> d_head:", 32 // 4)
```



تحلیل خروجی تست:

• **shape output (4, 16, 32):**

این نشان می‌دهد که مدل بدون تغییر ابعاد ورودی، خروجی سازگار با ساختار اولیه تولید کرده است. یعنی:

$$(B, T, C) \rightarrow (B, T, C)$$

• **kernel qkv (32, 96):**

ورودی دارای $C = 32$ ویژگی است. لایه qkv به طور همزمان، Key Query و Value را تولید می‌کند، بنابراین:

$$3C = 96$$

و شکل وزن‌ها:

$$(32, 96)$$

کاملاً صحیح است.

• **heads: 4 $\rightarrow$ d_head: 8:**

تعداد headها برابر ۴ است، بنابراین:

$$d_{\text{head}} = \frac{32}{4} = 8$$

هر head روی یک فضای ۸-بعدی مستقل کار می‌کند.

۹.۳ هر سر توجه به چه چیزی نگاه می‌کند؟ نقشه‌های توجه مجزا (Per-head Attention Maps)

در معماری «توجه چندسر» (Multi-Head Attention)، مزیت اصلی این است که هر «سر» (Head) می‌تواند به طور مستقل الگوهای متفاوتی از روابط بین کلمات را بیاموزد. به عنوان مثال، ممکن است پس از آموزش، یک سر متخصص پیدا کردن فاعل جمله باشد، سر دیگر کلمات همسایه را بررسی کند و سر سوم به علائم نگارشی توجه نماید.

برای درک بهتر این مکانیزم مستقل، پیش از آنکه مدل را آموزش دهیم، یک قطعه متن ثابت ("To be, or not to be") را وارد ماژولی با وزن‌های تصادفی می‌کنیم و ماتریس توجه علی تولیدشده توسط هر سر به صورت مجزا را رسم می‌کنیم.

کالبدشکافی کد: استخراج دستی توجه هر سر

در قطعه کد زیر، به جای اجرای کامل مدل، لایه‌ها را به صورت دستی باز می‌کنیم تا دقیقاً ببینیم در دل لایه qkv چه می‌گذرد و ماتریس برای هر کدام از ۴ سر چگونه شکل می‌گیرد:

```
1 # Per-head attention heatmaps from the qkv kernel on a fixed snippet.
2 snippet = "To be, or not to be"
3 ids = jnp.asarray(encode(snippet))[None, :] # (1, T)
4 T = ids.shape[1]
5 n_head, n_embd = 4, 32
6
7 embed = nn.Embed(vocab_size, n_embd, name="emb")
```



```

8     ekey, akey = jax.random.split(jax.random.PRNGKey(0))
9     emb_p = embed.init(ekey, ids)
10    xe = embed.apply(emb_p, ids)                                # (1, T, n_embd)
11
12    mha_vis = CausalSelfAttention(n_head=n_head, n_embd=n_embd)
13    vis_p = mha_vis.init(akey, xe)
14    W = vis_p["params"]["qkv"]["kernel"]                        # (n_embd, 3*n_embd)
15
16    d_head = n_embd // n_head
17    qkv = xe @ W
18    q, k, _ = jnp.split(qkv, 3, axis=-1)
19    def _sh(t): return t.reshape(1, T, n_head, d_head).transpose(0, 2, 1, 3)
20    q, k = _sh(q), _sh(k)
21    scores = (q @ jnp.swapaxes(k, -1, -2)) / jnp.sqrt(d_head)
22    mask = jnp.tril(jnp.ones((T, T), dtype=bool))
23    scores = jnp.where(mask, scores, -1e9)
24    attn = jax.nn.softmax(scores, axis=-1)                       # (1, n_head, T, T)
25
26    labels = list(snippet)
27    fig, axes = plt.subplots(1, n_head, figsize=(16, 4))
28    for h in range(n_head):
29        ax = axes[h]
30        im = ax.imshow(np.array(attn[0, h]), cmap="viridis", vmin=0, vmax=1)
31        ax.set_title(f"head {h}")
32        ax.set_xticks(range(T)); ax.set_xticklabels(labels, fontsize=7, rotation=90)
33        ax.set_yticks(range(T)); ax.set_yticklabels(labels, fontsize=7)
34        fig.suptitle("Per-head causal attention on a fixed snippet (random init)")
35        fig.colorbar(im, ax=axes, fraction=0.02); plt.show()
36

```

روند استخراج اطلاعات در کد بالا:

۱. تعبیه کلمات (**Embedding**): ابتدا رشته متنی به شناسه‌های عددی (**ids**) تبدیل شده و با استفاده از لایه `nn.Embed` به بردارهای ۳۲-بعدی (**xe**) نگاشت می‌شود.

۲. استخراج هسته **QKV**: به جای فراخوانی تابع `apply` ماژول، مستقیماً ماتریس وزن‌های `qkv` (یعنی متغیر `W` با ابعاد 32×96) را از `Pytree` بیرون می‌کشیم.

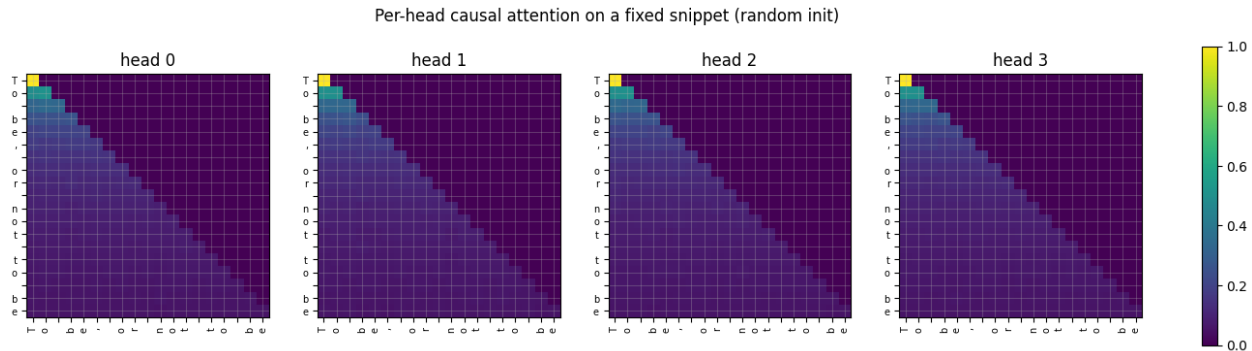
۳. جداسازی سرها (**Head Splitting**): محاسبات ضرب ماتریسی ($xe \times W$) انجام شده و خروجی به سه بخش Q, K, V تقسیم می‌شود. با تابع `_sh`، محورها را جابه‌جا می‌کنیم تا ابعاد $(1, 4, 19, 8)$ شکل بگیرد (دسته=۱، سر=۴، طول=۱۹، بعد سر=۸).

۴. اعمال توجه علی: ضرب نقطه‌ای مقیاس‌شده همراه با ماسک پایین مثلثی و تابع `Softmax` روی دو محور آخر انجام می‌شود که در نهایت ماتریس `attn` با ابعاد $(1, 4, 19, 19)$ را تولید می‌کند.



تحلیل بصری نقشه‌های توجه مجزا (پیش از آموزش)

شکل ۱۵ خروجی این قطعه کد را نشان می‌دهد. در اینجا ۴ نقشه حرارتی مجزا برای ۴ سر (head 0 تا head 3) رسم شده است.



شکل ۱۵: نقشه حرارتی توجه علی به ازای هر سر (Per-head) روی یک قطعه متن ثابت. به دلیل تصادفی بودن وزن‌های اولیه (Random Init)، الگوها در تمام سرها تقریباً یکنواخت و فاقد معنای زبانی هستند، اما ساختار علی (ماسک پایین‌مشی) در تمامی آن‌ها به شدت رعایت شده است.

کالبدشکافی و تحلیل نمودارها:

- استقلال محاسباتی: با وجود اینکه هر ۴ سر اطلاعات خود را از یک ورودی یکسان تغذیه می‌کنند، اما به دلیل ضرب شدن در بخش‌های متفاوتی از ماتریس وزن تصادفی W ، مقادیر و رنگ‌بندی آن‌ها در داخل مثلث پایینی (هرچند نامحسوس) با یکدیگر تفاوت دارد. این نشان می‌دهد که سرها کاملاً در فضاها (Representation Spaces) متفاوتی عمل می‌کنند.
- ساختار ساخت یافته علی (Causal Structure): هر ۴ نقشه حرارتی، بدون استثنا، در نیمه بالایی قطر اصلی کاملاً تاریک (مقدار صفر) هستند. این اثبات می‌کند که مکانیزم جداسازی سرها (Reshape و Transpose) باعث به هم ریختن ابعاد زمانی نشده است و ماسک علی دقیقاً در جای درست خود، دسترسی به آینده را مسدود کرده است.
- فقدان الگوهای معنادار (اثر وزن تصادفی): برخلاف مدل‌های آموزش دیده که در آن‌ها مربع‌های روشنی روی کلمات مرتبط از نظر گرامری (مثلاً توجه یک فعل به فاعل) دیده می‌شود، در اینجا رنگ‌ها در مثلث پایینی تقریباً یکدست (محو و آبی/بنفش تیره) هستند. این یعنی مدل به دلیل نداشتن دانش زبانی، توجه خود را تقریباً به صورت مساوی (Uniform Distribution) بین تمام توکن‌های گذشته پخش کرده است.
- تجمع توجه در ستون اول (Attention Sink): در تمامی ۴ سر، ستون اول (متعلق به کلمه 'T') روشن‌تر از بقیه ستون‌ها به نظر می‌رسد. این یک ویژگی کلاسیک در شبکه‌های ترنسفورمر آموزش ندیده است؛ توکن‌های انتهایی جمله که نمی‌دانند باید به کدام کلمه توجه کنند، وزن‌های مازاد خود را روی امن‌ترین توکن (یعنی اولین توکن جمله که همیشه در دسترس است) می‌ریزند.

نتیجه‌گیری: این تحلیل بصری نشان می‌دهد که معماری ماژول توجه چندسر (Multi-Head Attention) از نظر منطقی ریاضی، ابعاد تانسورها و اعمال قوانین علیت (Causality) کاملاً بی‌نقص عمل می‌کند. تنها چیزی که اکنون برای تبدیل این مثلث‌های مات و یکدست به متخصصین زبانی نیاز است، اجرای یک حلقه آموزش (Training Loop) و تنظیم وزن‌های W از طریق الگوریتم پس انتشار (Backpropagation) است.

۱۰.۳ توابع فعال‌ساز در شبکه‌های پیش‌خور: مقایسه $\tanh$ و ReLU , GELU

شبکه عصبی پیش‌خور (MLP) در داخل هر بلوک ترنسفورمر، وظیفه پردازش مستقل و استخراج ویژگی‌های پیچیده‌تر از هر توکن را بر عهده دارد. برای ایجاد خاصیت غیرخطی (Non-linearity) در این شبکه، انتخاب تابع فعال‌ساز (Activation Function) نقشی حیاتی در پایداری و کیفیت آموزش ایفا می‌کند.

در حالی که در گذشته توابعی مانند $\tanh$ و ReLU استانداردهای اصلی یادگیری عمیق بودند، معماری‌های مدرن ترنسفورمر (از جمله BERT و خانواده GPT) تقریباً به طور کامل به سمت تابع GELU (Gaussian Error Linear Unit) حرکت کرده‌اند. در این بخش، به تحلیل ریاضی و بصری این سه تابع و دلایل این تغییر رویکرد می‌پردازیم.

تعاریف ریاضی توابع فعال‌ساز

فرمول ریاضی این سه تابع کلاسیک و مدرن به شرح زیر است:

۱. تابع $\tanh$ (تانژانت هیپربولیک): این تابع، ورودی‌ها را به یک بازه متقارن بین -1 و 1 فشرده می‌کند:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2)$$

۲. تابع ReLU (Rectified Linear Unit): ساده‌ترین و سریع‌ترین تابع فعال‌ساز که مقادیر منفی را دقیقاً صفر کرده و مقادیر مثبت را بدون تغییر عبور می‌دهد:

$$\text{ReLU}(x) = \max(0, x) \quad (3)$$

۳. تابع GELU (Gaussian Error Linear Unit): این تابع بر اساس توزیع تجمعی نرمال استاندارد یا $\Phi(x)$ تعریف می‌شود. در این تابع، ورودی در احتمال بزرگتر بودن یک متغیر تصادفی نرمال از آن ورودی، ضرب می‌شود:

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right] \quad (4)$$

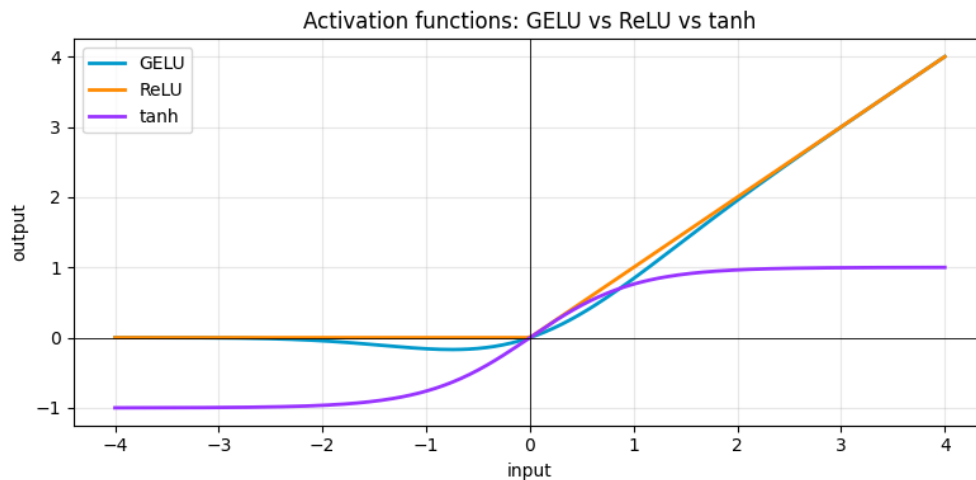
که در آن erf تابع خطای گاوسی است. (به دلیل پیچیدگی محاسباتی، معمولاً از تقریب‌های سریع‌تری بر پایه $\tanh$ یا توابع چندجمله‌ای در پیاده‌سازی‌های عملی استفاده می‌شود).

تحلیل بصری و مقایسه توابع

شکل ۱۶ نمودار این سه تابع را در یک بازه ورودی یکسان (از -4 تا 4) نشان می‌دهد تا تفاوت رفتار آن‌ها ملموس‌تر شود.

تحلیل نمودار:

- منحنی بنفش ($\tanh$): همان‌طور که مشخص است، این تابع به سرعت در مقادیر بزرگتر از 2 یا کوچکتر از -2 به حالت اشباع (Saturation) می‌رسد (افقی می‌شود). این اشباع باعث می‌شود گرادیان (شیب خط) تقریباً صفر شده و شبکه نتواند چیزی یاد بگیرد (مشکل محو شدن گرادیان یا Vanishing Gradient).



شکل ۱۶: مقایسه رفتار توابع فعال‌ساز GELU، ReLU و tanh. تابع GELU (آبی) برخلاف ReLU (نارنجی) یک منحنی نرم است که اجازه عبور مقادیر منفی کوچک را می‌دهد.

- منحنی نارنجی (ReLU): این تابع برای مقادیر مثبت رفتاری کاملاً خطی دارد و دچار اشباع نمی‌شود. اما برای تمامی مقادیر منفی، خروجی و همچنین شیب (مشتق) آن دقیقاً صفر است.
- منحنی آبی (GELU): این تابع در مقادیر مثبتِ بزرگ، رفتاری بسیار شبیه به ReLU دارد. اما تفاوت اصلی در حوالی مبدأ (از 0 تا -2) است؛ جایی که GELU به نرمی به زیر صفر نفوذ می‌کند (یک دره کوچک تشکیل می‌دهد) و سپس به آرامی به سمت صفر مجانب می‌شود.

چرا ترنسفورمرها ReLU را کنار گذاشتند؟

با وجود سادگی و سرعت بالای ReLU، مدل‌های زبانی بزرگ نیازمند ظرفیت بهینه‌سازی بسیار بالاتری هستند. جایگزینی ReLU با GELU به سه دلیل عمده صورت گرفت:

۱. حل مشکل مرگ نورون‌ها (Dying ReLU): در تابع ReLU، اگر یک به‌روزرسانی بزرگ باعث شود وزن‌های یک نورون به گونه‌ای تغییر کند که خروجی آن همیشه منفی شود، مشتق آن نورون برای همیشه صفر خواهد ماند. در نتیجه، آن نورون دیگر هرگز در فرآیند آموزش (پس‌انتشار خطا) به‌روزرسانی نمی‌شود و عملاً «می‌میرد». اما GELU به دلیل داشتن آن «دره کوچک منفی» (حدوداً تا $x = -1.5$)، برای مقادیر منفی کوچک هنوز یک مشتق غیرصفر دارد. این ویژگی به نورون‌های ضعیف اجازه می‌دهد تا خود را بازیابی کرده و دوباره به فرآیند یادگیری بازگردند.

۲. مشتق‌پذیری و نرم بودن (Smoothness): تابع ReLU در نقطه $x = 0$ دارای یک شکستگی تند است و مشتق آن در این نقطه تعریف نشده است. این تغییر ناگهانی شیب می‌تواند باعث نوسان در بهینه‌سازهایی مانند Adam شود. در مقابل، GELU یک تابع کاملاً هموار (Smooth) است و در تمامی نقاط به صورت پیوسته مشتق‌پذیر است. این نرمی باعث می‌شود فضای خطا (Loss Landscape) هموارتر شده و فرآیند رسیدن به نقطه بهینه (Convergence) با ثبات بیشتری طی شود.

۳. گیت‌گذاری تصادفی (Stochastic Gating): تابع ReLU صرفاً بر اساس «علامت» ورودی (مثبت یا منفی بودن) تصمیم می‌گیرد که سیگنال را عبور دهد یا خیر. اما GELU بر اساس «ارزش آماری» ورودی تصمیم‌گیری می‌کند. از آنجایی که GELU از تابع

توزیع نرمال $\Phi(x)$ استفاده می‌کند، در واقع به این می‌پردازد که «احتمال اینکه این ورودی از سایر ورودی‌ها مهم‌تر باشد چقدر است؟». این نوع گیت‌گذاری نرم، تطابق بسیار خوبی با رفتار آماری داده‌ها در شبکه‌های عمیق دارد و به صورت تجربی ثابت شده است که عملکرد مدل‌های زبانی را بهبود می‌بخشد.

۱۱.۳ پیاده‌سازی دستی نرمال‌سازی لایه‌ای (Layer Normalization)

در شبکه‌های عصبی عمیق، با عبور داده‌ها از لایه‌های متوالی، توزیع مقادیر فعال‌سازی (Activations) مدام تغییر می‌کند و ممکن است اعداد بسیار بزرگ یا بسیار کوچک شوند. این پدیده باعث بی‌ثباتی در فرآیند آموزش و پدیده انفجار یا محو شدن گرادیان‌ها می‌شود. برای حل این مشکل در معماری ترنسفورمر، از نرمال‌سازی لایه‌ای یا LayerNorm استفاده می‌شود. برخلاف BatchNorm که داده‌ها را در امتداد یک دسته (Batch) نرمال می‌کند، LayerNorm عملیات نرمال‌سازی را برای هر توکن به صورت کاملاً مستقل و در امتداد بُعد ویژگی‌ها (Feature Dimension) انجام می‌دهد.

روابط ریاضی

اگر بردار ویژگی‌های یک توکن را x بنامیم، نرمال‌سازی لایه‌ای طی مراحل زیر انجام می‌شود:

۱. محاسبه میانگین (Mean): میانگین مقادیر بردار محاسبه می‌شود:

$$\mu = \frac{1}{C} \sum_{i=1}^C x_i \quad (5)$$

۲. محاسبه واریانس (Variance): واریانس داده‌ها نسبت به میانگین به دست می‌آید:

$$\sigma^2 = \frac{1}{C} \sum_{i=1}^C (x_i - \mu)^2 \quad (6)$$

۳. استانداردسازی (Standardization): داده‌ها با کسر میانگین و تقسیم بر انحراف معیار، نرمال می‌شوند. یک مقدار بسیار کوچک ϵ (اپسیلون) نیز برای جلوگیری از خطای تقسیم بر صفر به مخرج اضافه می‌شود:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (7)$$

۴. مقیاس‌دهی و انتقال (Scale and Shift): از آنجایی که نرمال‌سازی مطلق ممکن است ظرفیت بازنمایی شبکه را کاهش دهد، دو پارامتر قابل آموزش γ (مقیاس) و β (انتقال/بایاس) به رابطه اضافه می‌شوند تا شبکه در صورت نیاز بتواند توزیع را به حالت دلخواه خود برگرداند:

$$y = \gamma \hat{x} + \beta \quad (8)$$

کالبدشکافی کد پیاده‌سازی

کد زیر دقیقاً همین روابط ریاضی را با استفاده از توابع JAX پیاده‌سازی می‌کند:

```

1 def manual_layernorm(x, gamma, beta, eps=1e-5):
2     # Normalize x over the last axis, then apply the learned scale/shift.
3     mu = jnp.mean(x, axis=-1, keepdims=True)
4     var = jnp.var(x, axis=-1, keepdims=True)
5     return gamma * (x - mu) / jnp.sqrt(var + eps) + beta
6

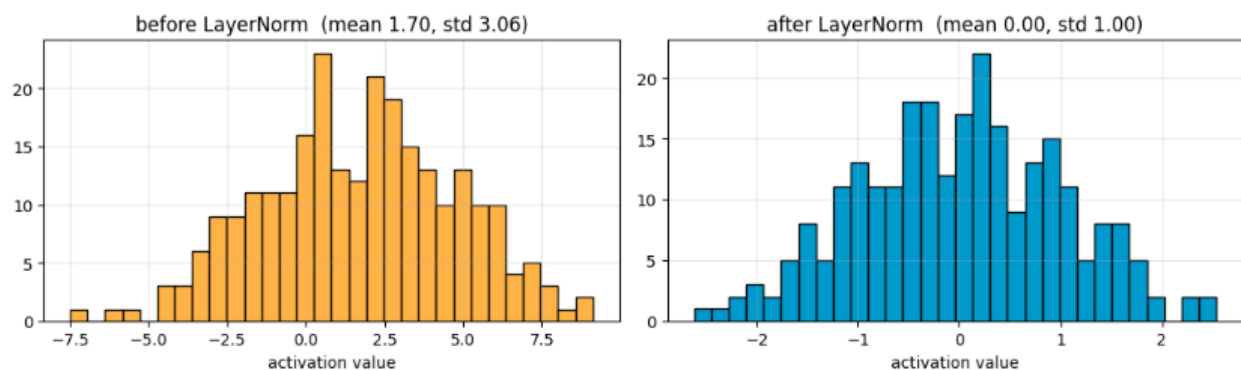
```

نکات کلیدی کد:

- **axis = -1:** این آرگومان به تابع می‌گوید که میانگین و واریانس را فقط روی آخرین بُعد (یعنی بُعد ویژگی‌ها یا C) محاسبه کند. اطلاعات توکن‌های مختلف یا جملات متفاوت در یک دسته، با هم قاطی نمی‌شوند.
- **keepdims = True:** این یک ترفند حیاتی در برنامه‌نویسی تنسورهاست. اگر این مقدار False باشد، بُعد آخر حذف شده و ماتریس ما یک بُعدی می‌شود. اما با True بودن آن، بُعد آخر با اندازه ۱ حفظ می‌شود (مثلاً از (B, T, C) به $(B, T, 1)$ تبدیل می‌شود). این کار باعث می‌شود تا در خط آخر، عملیات تفریق $(x - \mu)$ به درستی و با استفاده از خاصیت Broadcasting روی تمام عناصر انجام شود.
- **gamma و beta:** در ابتدای آموزش، معمولاً γ برابر با ۱ و β برابر با صفر مقداردهی می‌شوند تا لایه دقیقاً خروجی نرمال‌شده را برگرداند.

تحلیل بصری اثر LayerNorm

برای درک بهتر تأثیر این لایه، هیستوگرام مقادیر فعال‌سازی (Activations) را قبل و بعد از عبور از LayerNorm رسم کرده‌ایم که در شکل ۱۷ قابل مشاهده است.



شکل ۱۷: هیستوگرام توزیع مقادیر فعال‌سازی. سمت چپ: پیش از نرمال‌سازی (پراکنده و دارای انحراف). سمت راست: پس از نرمال‌سازی (متمرکز حول صفر با واریانس واحد).

تحلیل نمودارها:



• نمودار سمت چپ (قبل از **LayerNorm**): توزیع اعداد به هم ریخته است. میانگین داده‌ها روی عدد ۱.۷۰ قرار دارد (انتقال به راست) و انحراف معیار بسیار بالای ۳.۰۶ نشان می‌دهد که داده‌ها به شدت پراکنده‌اند (از حدود -۷ تا +۹). اگر این اعداد مستقیماً وارد لایه‌های بعدی یا تابعی مثل Softmax یا GELU شوند، شبکه را دچار ناپایداری یا اشباع می‌کنند.

• نمودار سمت راست (بعد از **LayerNorm**): لایه **LayerNorm** دقیقاً مانند یک تنظیم‌کننده (Regulator) عمل کرده است. نمودار به یک توزیع نرمال استاندارد بی نقص تبدیل شده است؛ میانگین دقیقاً به صفر (0.00) منتقل شده و انحراف معیار (پراکندگی) دقیقاً روی یک (1.00) فشرده شده است (داده‌ها اکنون در بازه امن -3 تا +3 قرار دارند).

نتیجه‌گیری: لایه **LayerNorm** تضمین می‌کند که مهم نیست در لایه‌های قبلی چه اتفاقی افتاده و اعداد چقدر بزرگ شده‌اند؛ هر سیگنالی پیش از ورود به بلوک‌های محاسباتی جدید (مثل توجه چندسر یا شبکه پیش‌خور)، مجدداً «رام» شده و در یک محدوده عددی ایده‌آل و قابل پیش‌بینی قرار می‌گیرد.

۱۲.۳ مونتاژ بلوک کامل ترنسفورمر با معماری پیش‌نرمال‌سازی (Pre-Norm Transformer Block)

اکنون که تمامی قطعات پازل (شامل توجه چندسر، توابع فعال‌ساز و نرمال‌سازی لایه‌ای) را به طور مجزا طراحی کرده و مفاهیم ریاضی آن‌ها را درک کردیم، زمان آن رسیده است که این قطعات را در کنار یکدیگر قرار داده و یک «بلوک ترنسفورمر» (Transformer Block) کامل بسازیم. مدل‌های زبانی بزرگ، در واقع چیزی نیستند جز تکرار پشت سر هم همین بلوک (برای مثال، مدل GPT-3 دارای ۹۶ عدد از این بلوک‌ها به صورت متوالی است).

معماری پیش‌نرمال‌سازی (Pre-Norm) و اتصالات میان‌بر (Residual Connections)

در مقاله اصلی ترنسفورمر (سال ۲۰۱۷)، لایه نرمال‌سازی پس از اتصال میان‌بر اعمال می‌شد (معماری Post-Norm: $\text{LayerNorm}(x + \text{Sublayer}(x))$). اما با گذشت زمان و تلاش برای ساخت شبکه‌های بسیار عمیق، محققان متوجه شدند که این ساختار باعث ناپایداری گرادینت‌ها می‌شود. امروزه در تمام مدل‌های مدرن خانواده GPT از معماری Pre-Norm استفاده می‌شود. در این معماری:

۱. بزرگراه اطلاعات (**Information Highway**): متغیر x (که حاوی اطلاعات توکن‌هاست) به صورت یک مسیر مستقیم و بدون هیچ مانعی از اول تا آخر شبکه کشیده می‌شود.

۲. پیش‌نرمال‌سازی: هر زمان که زیرلایه‌های توجه (Attention) یا پیش‌خور (MLP) بخواهند تغییری روی داده‌ها اعمال کنند، ابتدا یک کپی از داده‌ها گرفته شده، نرمال می‌شود (**LayerNorm**) و سپس پردازش می‌گردد.

۳. اتصال میان‌بر (**Residual**): در نهایت، خروجی پردازش‌شده با داده‌های اصلی جمع می‌شود ($x = x + \Delta x$).

این کار باعث می‌شود در جریان پس‌انتشار (Backpropagation)، مشتقات (گرادینت‌ها) بتوانند به راحتی و بدون ضعیف شدن، از طریق عملگر «جمع»، تا اولین لایه‌های شبکه حرکت کنند.

کالبدشکافی کد بلوک ترنسفورمر

در کد زیر، یک بلوک کامل ترنسفورمر در Flax پیاده‌سازی شده است:

```

1 class TransformerBlock(nn.Module):
2     n_head: int
3     n_embd: int
4     dropout: float = 0.0
5
6     @nn.compact
7     def __call__(self, x, deterministic=True):
8         # 1. pre-norm attention sub-block with a residual connection.
9         x = x + CausalSelfAttention(
10             n_head=self.n_head,
11             n_embd=self.n_embd,
12             dropout=self.dropout
13         )(nn.LayerNorm()(x), deterministic=deterministic)
14
15         # 2. pre-norm position-wise MLP sub-block with a residual connection.
16         h = nn.LayerNorm()(x)
17         h = nn.Dense(4 * self.n_embd)(h)
18         h = nn.gelu(h)
19         h = nn.Dense(self.n_embd)(h)
20         h = nn.Dropout(self.dropout)(h, deterministic=deterministic)
21
22         return x + h
23

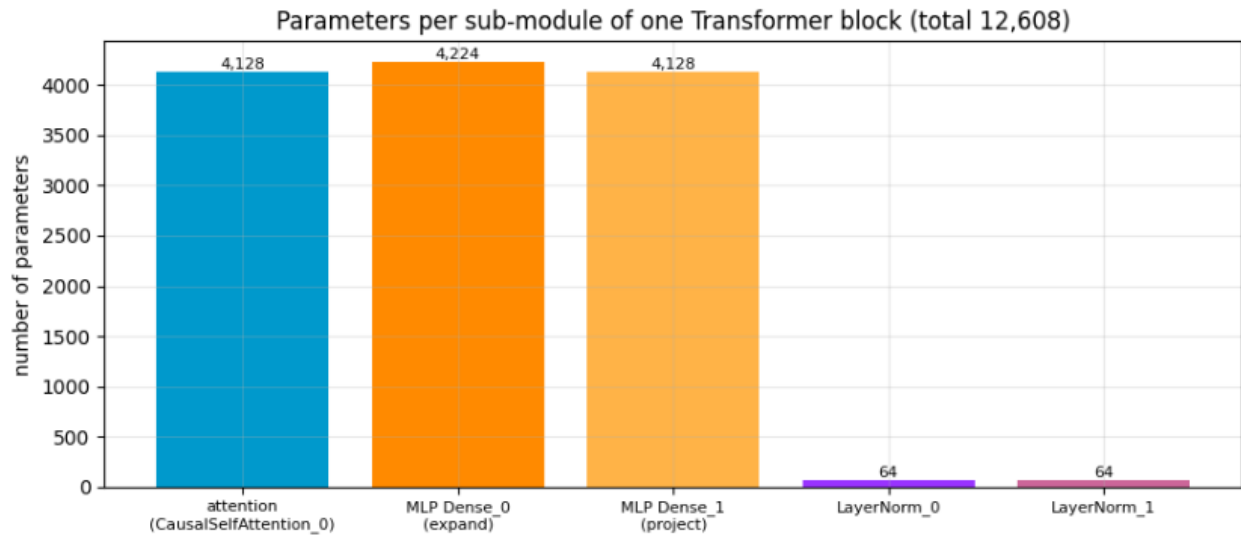
```

تحلیل خطوط کد:

- زیربلوک توجه (**Attention Sub-block**): در بخش اول، ابتدا nn.LayerNorm روی x اعمال می‌شود. سپس خروجی آن به $\text{CausalSelfAttention}$ (که در بخش قبل نوشتیم) پاس داده می‌شود. متغیر deterministic نیز برای کنترل خاموش/روشن بودن Dropout به آن ارسال می‌گردد. در نهایت، خروجی این ماژول با خود x جمع می‌شود (عملگر $+$).
- زیربلوک پیش‌خور (**MLP Sub-block**): در بخش دوم، دوباره یک LayerNorm جدید روی x جدید اعمال می‌شود (در متغیر h).
- ضریب انبساط ۴ برابری (**Expansion**): لایه Dense اول، بردار ویژگی‌ها را به فضایی ۴ برابر بزرگتر ($4 * \text{self.n_embd}$) پرتاب می‌کند. این فضای وسیع‌تر به تابع GELU اجازه می‌دهد تا ترکیبات غیرخطی و پیچیده‌تری از مفاهیم را بسازد.
- تراکم و دراپ‌اوت (**Projection & Dropout**): لایه Dense دوم، داده‌ها را دوباره به بعد اصلی (n_embd) فشرده می‌کند تا بتوانند با x جمع شوند. در نهایت یک لایه Dropout اعمال شده و نتیجه با x جمع می‌گردد ($\text{return } x + h$).

تحلیل توزیع پارامترها در بلوک ترنسفورمر

برای درک بهتر معماری، بررسی می‌کنیم که پارامترهای یادگیری‌شونده (**Weights & Biases**) چگونه در داخل یک بلوک توزیع شده‌اند. شکل ۱۸ بودجه‌بندی پارامترها را برای یک بلوک با بعد ویژگی $C = 32$ نشان می‌دهد.



شکل ۱۸: نمودار توزیع پارامترها در زیرماژول‌های مختلف یک بلوک ترنسفورمر (مجموعاً ۱۲,۶۰۸ پارامتر). بخش عمده‌ای از ظرفیت یادگیری به شبکه پیش‌خور (MLP) اختصاص یافته است.

کالبدشکافی دقیق اعداد در نمودار: با فرض $n_embd = 32$ ، محاسبات پارامترها به شرح زیر است:

۱. لایه توجه (CausalSelfAttention): ۴,۱۲۸ پارامتر.

- ماتریس QKV: ورودی ۳۲، خروجی $96 = 32 \times 3$ ، بدون بایاس $3,072 = 32 \times 96 \rightarrow$.
- لایه پروجکشن خروجی: ورودی ۳۲، خروجی ۳۲، به علاوه بایاس $1,056 = (32 \times 32) + 32 \rightarrow$.
- مجموع: $3,072 + 1,056 = 4,128$. این بخش مسئول کشف روابط و الگوها میان کلمات است.

۲. لایه انبساط پیش‌خور (MLP Dense_0): ۴,۲۲۴ پارامتر.

- ورودی ۳۲، خروجی $128 = 4 \times 32$ ، به علاوه ۱۲۸ بایاس $4,224 = (32 \times 128) + 128 \rightarrow$.

۳. لایه تراکم پیش‌خور (MLP Dense_1): ۴,۱۲۸ پارامتر.

- ورودی ۱۲۸، خروجی ۳۲، به علاوه ۳۲ بایاس $4,128 = (128 \times 32) + 32 \rightarrow$.

۴. لایه‌های نرمال‌سازی (LayerNorm 0 & 1): هر کدام ۶۴ پارامتر.

- هر لایه LayerNorm شامل بردار مقیاس (γ) با طول ۳۲ و بردار انتقال (β) با طول ۳۲ است ($32 + 32 = 64$).

نتیجه‌گیری مهم: با جمع زدن مقادیر دو لایه Dense، مشاهده می‌کنیم که شبکه MLP مجموعاً ۸,۳۵۲ پارامتر دارد که حدوداً دو برابر پارامترهای لایه توجه (۴,۱۲۸) است. این یک حقیقت مهندسی در معماری ترنسفورمر است: «لایه توجه کشف می‌کند که چه کلماتی به هم مرتبط هستند، اما این لایه MLP است که با داشتن بیشترین تعداد پارامتر، دانش واقعی و معنای کلمات (بازنمایی‌ها) را در خود ذخیره می‌کند.»

۱۳.۳ تحلیل تکمیلی: نحوه رشد جریان میان‌بر (Residual Stream) با افزایش عمق شبکه

یکی از ابزارهای تشخیصی بسیار مفید برای درک پویایی (Dynamics) شبکه‌های عصبی عمیق، بررسی نحوه تغییر اندازه یا «نُرم» (Norm) مقادیر فعال‌سازی با افزایش عمق شبکه (انباشته شدن بلوک‌ها) است.

در معماری ترنسفورمر، مسیر اصلی عبور داده‌ها به عنوان جریان میان‌بر (Residual Stream) شناخته می‌شود. از آنجا که هر بلوک ترنسفورمر، خروجی محاسبات خود (توجه و پیش‌خور) را با داده‌های ورودی جمع می‌کند ($x = x + \Delta x$)، اندازه بردارهای این جریان به طور طبیعی با عبور از هر لایه تمایل به افزایش دارد. این پدیده یکی از دلایل اصلی اهمیت و ضرورت استفاده از معماری پیش‌نرمال‌سازی (Pre-Norm) و لایه LayerNorm است.

آزمایش و نتایج عددی

برای بررسی این پدیده، یک ورودی ثابت را از درون شبکه‌ای متشکل از بلوک‌های ترنسفورمر که به تازگی و به صورت تصادفی مقداردهی اولیه شده‌اند (Random Init) عبور می‌دهیم و میانگین نُرم L_2 را به ازای هر توکن در اعماق مختلف شبکه (از ۱ تا ۶ بلوک) محاسبه می‌کنیم.

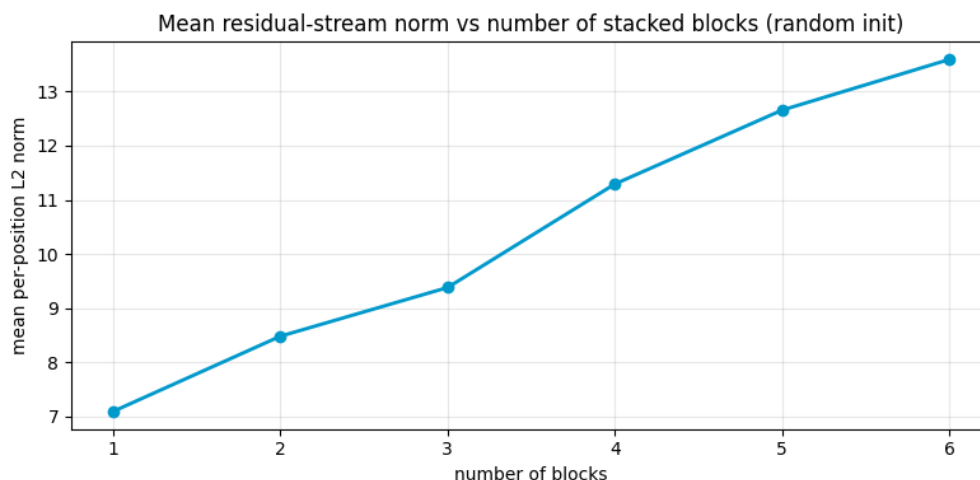
نتایج عددی به‌دست‌آمده به شرح زیر است:

```
1 norms by depth: [7.095, 8.483, 9.382, 11.291, 12.654, 13.59]
```

```
2
```

تحلیل نمودار و مفاهیم کلیدی

شکل ۱۹ روند تغییرات این اعداد را به صورت بصری نشان می‌دهد.



شکل ۱۹: رشد میانگین نُرم L_2 در جریان میان‌بر با افزایش تعداد بلوک‌های ترنسفورمر در حالت مقداردهی اولیه تصادفی.

کالبدشکافی و تحلیل رفتار شبکه:



۱. رشد صعودی و پیوسته: همان‌طور که در نمودار و اعداد مشخص است، نرم بردارها از مقدار حدود 7.1 در خروجی بلوک اول، با یک شیب تقریباً یکنواخت به 13.6 در خروجی بلوک ششم رسیده است. هرچه بلوک‌های بیشتری روی هم انباشته (Stacked) شوند، اعداد بزرگتر و بزرگتر می‌شوند.

۲. خطر ناپایداری محاسباتی: اگر این اعداد بزرگ مستقیماً وارد محاسبات پیچیده‌ای مانند ضرب ماتریسی در لایه توجه (Attention) شوند، واریانس به شدت منفجر شده و خروجی لایه Softmax کاملاً به حالت اشباع می‌رود. در چنین شرایطی، شبکه توانایی یادگیری خود را از دست می‌دهد.

۳. دلیل اثربخشی **Pre-Norm**: این نمودار دقیقاً نشان می‌دهد که چرا ما در بلوک ترنسفورمر، پیش از ورود به لایه‌های Attention و MLP، از LayerNorm استفاده کردیم (معماری Pre-norm). لایه نرمال‌ساز به عنوان یک «سوپاپ اطمینان» عمل می‌کند؛ مهم نیست که جریان میان‌بر با رسیدن به لایه شصتم چقدر بزرگ و متورم شده باشد، LayerNorm پیش از هرگونه پردازش جدید، یک کپی از این داده‌ها می‌گیرد و واریانس آن‌ها را مجدداً به عدد ۰.۱ فشرده می‌کند (همان‌طور که در هیستوگرام بخش قبلی دیدیم).

نتیجه‌گیری: این تحلیل به روشنی نشان می‌دهد که معماری ترنسفورمر چگونه با ترکیب هوشمندانه اتصالات میان‌بر (برای انتقال بی‌نقص گرادیان‌ها و جلوگیری از فراموشی اطلاعات) و لایه‌های نرمال‌ساز (برای مهار رشد تصاعدی اعداد)، توانسته است به شبکه‌هایی با صدها لایه عمق دست یابد بدون آنکه در فرآیند آموزش دچار فروپاشی شود.



۴ کوئرا - ساخت و آموزش MiniGPT

۱.۴ از بلوک‌ها تا مدل کامل: Embedding و Head خروجی

در این بخش، تمام بلوک‌های Transformer که تا اینجا بررسی شدند، در قالب یک مدل کامل مانند GPT کنار هم قرار می‌گیرند. یک مدل زبانی فقط از چند بلوک توجه تشکیل نشده است، بلکه یک ساختار کامل از ورودی تا خروجی دارد.

ورودی مدل: Embedding توکن‌ها

اولین مرحله در یک مدل زبانی، تبدیل توکن‌ها به بردارهای عددی است. این کار توسط Token Embedding انجام می‌شود:

```
nn.Embed(vocab_size, n_embd)
```

این لایه در واقع یک جدول یادگرفتنی است که هر شناسه توکن را به یک بردار با بعد n_{embd} نگاشت می‌کند:

$$\text{id token} \rightarrow \mathbb{R}^{n_{embd}}$$

به این ترتیب، هر کلمه یا کاراکتر به یک نمایش برداری قابل پردازش توسط شبکه تبدیل می‌شود.

Embedding مکانی (Positional Embedding)

از آنجایی که Transformer به تنهایی ترتیب توکن‌ها را نمی‌فهمد، نیاز به اطلاعات موقعیت وجود دارد. در GPT از Embedding یادگرفتنی مکانی استفاده می‌شود:

```
nn.Embed(block_size, n_embd)
```

این لایه برای هر موقعیت در دنباله (مثلاً جایگاه ۰ تا T) یک بردار یادگرفتنی تولید می‌کند.

ترکیب دو Embedding

در ورودی مدل، دو نوع embedding با هم جمع می‌شوند:

$$x = x_{\text{token}} + x_{\text{position}}$$

که در آن:

- x_{token} : embedding معنایی توکن
- x_{position} : embedding موقعیت در دنباله

این جمع باعث می‌شود مدل هم معنا و هم ترتیب را همزمان دریافت کند.



عبور از بلوک‌های Transformer

پس از ساخت ورودی اولیه، داده وارد یک سری از بلوک‌های Transformer می‌شود:

$$x \rightarrow \text{Block}_1 \rightarrow \text{Block}_2 \rightarrow \dots \rightarrow \text{Block}_N$$

تعداد این بلوک‌ها برابر است با:

`n_layer`

هر بلوک شامل:

• Attention Multi-Head

• MLP

• Connections Residual

• LayerNorm

است.

نرمال‌سازی نهایی

پس از عبور از تمام بلوک‌ها، یک LayerNorm نهایی اعمال می‌شود تا توزیع ویژگی‌ها پایدار شود:

$$x = \text{LayerNorm}(x)$$

لایه خروجی (Language Model Head)

در نهایت، مدل باید احتمال کلمه بعدی را پیش‌بینی کند. این کار توسط یک لایه خطی انجام می‌شود:

`nn.Dense(vocab_size)`

این لایه بردارهای داخلی را به فضای واژگان نگاشت می‌کند:

$$\mathbb{R}^{n_{\text{embd}}} \rightarrow \mathbb{R}^{\text{vocab_size}}$$

Logits خروجی

خروجی مدل به صورت logits است:

$$\text{logits} \in \mathbb{R}^{B \times T \times V}$$

که در آن:



• B : اندازه batch

• T : طول دنباله

• V : اندازه vocabulary

در هر موقعیت زمانی t ، مدل یک توزیع روی تمام واژه‌های ممکن تولید می‌کند:

$$\text{logits}(b, t, :)$$

تفسیر خروجی

برای هر موقعیت در دنباله:

- مدل پیش‌بینی می‌کند "کلمه بعدی چه خواهد بود"
- هر مقدار در vector نشان‌دهنده امتیاز یک token است
- softmax روی logits یک توزیع احتمالاتی می‌سازد

جمع‌بندی

یک مدل GPT کامل از سه بخش اصلی تشکیل شده است:

- Embedding توکن + موقعیت
- مجموعه‌ای از بلوک‌های Transformer
- یک head خروجی برای پیش‌بینی واژگان

این ساختار ساده در ظاهر، اما بسیار قدرتمند است و اساس تمام مدل‌های زبانی مدرن را تشکیل می‌دهد.

۲.۴ تابع هزینه Cross-Entropy در پیش‌بینی توکن بعدی

مسئله یادگیری در مدل‌های زبانی مانند GPT در اصل یک مسئله طبقه‌بندی چندکلاسه است. در هر موقعیت زمانی، مدل باید از بین تمام واژگان موجود در واژگان (vocabulary)، توکن بعدی را پیش‌بینی کند. بنابراین طبیعی‌ترین تابع هزینه برای این مسئله، Loss Cross-Entropy است.



صورت‌بندی مسئله

در هر گام زمانی t ، مدل یک بردار logits تولید می‌کند:

$$\text{logits}_t \in \mathbb{R}^V$$

که در آن V اندازه واژگان است.

با اعمال تابع softmax، یک توزیع احتمالاتی به دست می‌آید:

$$p_t(i) = \frac{\exp(\text{logits}_t(i))}{\sum_{j=1}^V \exp(\text{logits}_t(j))}$$

در مقابل، برچسب واقعی یک مقدار صحیح (integer label) است که نشان می‌دهد توکن درست کدام است:

$$y_t \in \{1, 2, \dots, V\}$$

تعریف Cross-Entropy Loss

تابع خطا برای یک نمونه به صورت زیر تعریف می‌شود:

$$\mathcal{L}_t = -\log p_t(y_t)$$

و برای کل دنباله، میانگین گرفته می‌شود:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^T -\log p_t(y_t)$$

که در آن:

- T : طول دنباله
- $p_t(y_t)$: احتمال توکن صحیح در زمان t

تعبیر شهودی Cross-Entropy

Cross-Entropy در واقع اندازه‌گیری فاصله بین:

- توزیع پیش‌بینی شده توسط مدل
- توزیع واقعی (one-hot)

است.

اگر مدل به توکن درست احتمال بالا بدهد، loss کوچک می‌شود. اگر احتمال کم بدهد، loss بزرگ می‌شود.



مقدار اولیه loss در شروع آموزش

در ابتدای آموزش، مدل هیچ دانشی ندارد و تقریباً یک توزیع یکنواخت روی واژگان تولید می‌کند:

$$p(i) \approx \frac{1}{V}$$

در این حالت، loss به صورت زیر خواهد بود:

$$\mathcal{L}_{\text{init}} \approx -\log\left(\frac{1}{V}\right) = \log(V)$$

برای مثال، اگر اندازه واژگان $V = 65$ باشد:

$$\mathcal{L}_{\text{init}} \approx \log(65) \approx 4.17$$

این مقدار یک baseline مهم برای بررسی صحت آموزش است.

نقش numerically stable implementation

در پیاده‌سازی‌های واقعی (مانند optax)، به جای محاسبه مستقیم softmax و سپس log، از نسخه پایدار عددی استفاده می‌شود:

`optax.softmax_cross_entropy_with_integer_labels`

این روش:

- از overflow عددی جلوگیری می‌کند
- محاسبات log-softmax را بهینه انجام می‌دهد
- مستقیماً logits و label را دریافت می‌کند

جمع‌بندی

تابع Cross-Entropy در مدل‌های زبانی، معیاری برای اندازه‌گیری کیفیت پیش‌بینی توکن بعدی است. این تابع باعث می‌شود مدل یاد بگیرد در هر موقعیت زمانی، احتمال توکن صحیح را نسبت به سایر توکن‌ها افزایش دهد. کاهش تدریجی این loss در طول آموزش، نشان‌دهنده یادگیری صحیح مدل و بهبود توانایی آن در پیش‌بینی متن است.

۳.۴ بهینه‌سازی آموزش: AdamW، برش گرادیان و زمان‌بندی نرخ یادگیری

در آموزش مدل‌های بزرگ مانند Transformer، انتخاب روش بهینه‌سازی (optimizer) و نحوه تغییر نرخ یادگیری (learning rate schedule) نقش بسیار مهمی در پایداری و سرعت همگرایی دارد. در این بخش سه جزء کلیدی آموزش بررسی می‌شود: AdamW، برش گرادیان و زمان‌بندی warmup-cosine.



AdamW (بهینه‌ساز استاندارد Transformer)

AdamW نسخه‌ای اصلاح‌شده از الگوریتم Adam است که در آن decay weight به صورت جداگانه از به‌روزرسانی گرادیان اعمال می‌شود.

در Adam، به‌روزرسانی پارامترها به صورت زیر است:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

در AdamW، عبارت decay weight به صورت جداگانه اضافه می‌شود:

$$\theta_t = \theta_t - \eta \left(\frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_t \right)$$

که در آن:

• λ : ضریب decay weight

• η : نرخ یادگیری

مزیت اصلی AdamW این است که regularization به صورت صحیح و مستقل از update adaptive اعمال می‌شود.

برش گرادیان (Gradient Clipping)

در آموزش شبکه‌های عمیق، ممکن است مقدار گرادیان‌ها بسیار بزرگ شود و باعث ناپایداری آموزش گردد. برای جلوگیری از این مشکل از clipping بر اساس norm کل گرادیان استفاده می‌شود. اگر گرادیان کل به صورت زیر باشد:

$$\|g\|_2 = \sqrt{\sum_i g_i^2}$$

در صورتی که این مقدار از یک آستانه c بزرگ‌تر شود، گرادیان نرمال‌سازی می‌شود:

$$g \leftarrow g \cdot \frac{c}{\|g\|_2}$$

این کار باعث می‌شود:

- از انفجار گرادیان جلوگیری شود
- آموزش پایدارتر شود
- مدل در مراحل اولیه خراب نشود



زمان‌بندی نرخ یادگیری (Warmup + Cosine Decay)

نرخ یادگیری ثابت معمولاً برای آموزش مدل‌های بزرگ مناسب نیست. به همین دلیل از یک برنامه زمان‌بندی ترکیبی استفاده می‌شود:

۱. Warmup (افزایش اولیه) در ابتدای آموزش، نرخ یادگیری از صفر شروع شده و به تدریج افزایش می‌یابد:

$$\eta(t) = \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}}$$

این مرحله باعث می‌شود مدل در شروع آموزش دچار ناپایداری نشود.

۲. Decay Cosine (کاهش تدریجی) پس از مرحله warmup، نرخ یادگیری به صورت کسینوسی کاهش می‌یابد:

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\pi \frac{t}{T} \right) \right)$$

که در آن:

- $\eta_{\max}$: بیشترین نرخ یادگیری
- $\eta_{\min}$: کمترین نرخ یادگیری
- T : کل زمان آموزش

ترکیب در Optax

در پیاده‌سازی‌های JAX، این سه بخش معمولاً با استفاده از `optax.chain` ترکیب می‌شوند:

```
optax.chain( optax.clip_by_global_norm(1.0), optax.adamw(learning_rate=schedule) )
```

در این ساختار:

- ابتدا گرادینت‌ها `clip` می‌شوند
- سپس `optimizer` اعمال می‌شود
- نرخ یادگیری توسط یک تابع `schedule` کنترل می‌شود

جمع‌بندی

Transformer: مدل‌های

- AdamW برای بهینه‌سازی پایدار استفاده می‌شود
 - Clipping Gradient از انفجار گرادینت جلوگیری می‌کند
 - Warmup-Cosine باعث بهبود همگرایی در بلندمدت می‌شود
- این سه تکنیک در کنار هم، پایه استاندارد آموزش مدل‌های زبانی بزرگ را تشکیل می‌دهند.

۴.۴ مونتاژ معماری کلان: ماژول MiniGPT

پس از طراحی و پیاده‌سازی اجزای بنیادین مانند مکانیزم توجه چندسر (Multi-Head Attention)، شبکه‌های پیش‌خور (MLP) و بلوک‌های ترنسفورمر، اکنون زمان آن فرا رسیده است که این قطعات را در یک قاب نهایی و یکپارچه مونتاژ کنیم. ماژول MiniGPT پوسته بیرونی مدل ماست که وظیفه دریافت شناسه‌های خام متن (Token IDs)، تزریق اطلاعات مکانی، هدایت آن‌ها از میان لایه‌های عمیق ترنسفورمر و در نهایت تولید پیش‌بینی برای کلمه بعدی را بر عهده دارد.

تفاوت مهم این معماری با کدهای بخش‌های ابتدایی (مانند توابع سینوسی) در این است که GPT از کدگذاری مکانی یادگیری‌شونده (Learned Positional Embedding) استفاده می‌کند؛ به این معنا که مدل خودش بهترین بازنمایی ریاضی برای جایگاه کلمات را در حین آموزش کشف می‌کند.

پیاده‌سازی کد MiniGPT

در ادامه، پیاده‌سازی کامل این ماژول در فریم‌ورک Flax آورده شده است:

```
1 class MiniGPT(nn.Module):
2     vocab_size: int
3     block_size: int
4     n_layer: int
5     n_head: int
6     n_embd: int
7     dropout: float = 0.0
8
9     @nn.compact
10    def __call__(self, idx, deterministic=True):
11        B, T = idx.shape
12
13        # 1. Build token + positional embeddings, add them, then apply dropout.
14        tok = nn.Embed(self.vocab_size, self.n_embd, name="tok_emb")(idx)
15        pos = nn.Embed(self.block_size, self.n_embd, name="pos_emb")(jnp.arange(T))
16        x = tok + pos[None, :, :]
17        x = nn.Dropout(self.dropout)(x, deterministic=deterministic)
18
19        # 2. Apply self.n_layer TransformerBlock layers
20        for i in range(self.n_layer):
21            x = TransformerBlock(
22                self.n_head,
23                self.n_embd,
24                self.dropout,
25                name=f"block_{i}"
26            )(x, deterministic=deterministic)
27
28        # 3. Final normalization and linear head
29        x = nn.LayerNorm()(x)
30        logits = nn.Dense(self.vocab_size, name="head")(x)
```



```
31
32 return logits
33
```

کالبدشکافی و تحلیل دقیق ساختار کد

- دکوراتور `@nn.compact`: همان‌طور که پیش‌تر اشاره شد، این دکوراتور به ما اجازه می‌دهد لایه‌های فرعی (مثل Embed و TransformerBlock) را مستقیماً در تابع فراخوانی (`__call__`) و بدون نیاز به تعریف قبلی در `setup` مقداردهی کنیم.
- تعبیه توکن‌ها (`tok_emb`): ورودی مدل (`idx`) یک ماتریس از اعداد صحیح با ابعاد (B, T) است (B اندازه دسته و T طول جمله). لایه `nn.Embed` اول، یک جدول جستجو (Lookup Table) با ابعاد `vocab_size` ایجاد می‌کند و هر شناسه عدد صحیح را به یک بردار پیوسته به طول `n_embd` نگاشت می‌کند. خروجی این بخش ماتریسی با ابعاد (B, T, C) است.
- تعبیه موقعیتی (`pos_emb`): لایه `nn.Embed` دوم، جدولی به اندازه حداکثر ظرفیت حافظه مدل (`block_size`) می‌سازد. با دستور `jnp.arange(T)`، آرایه‌ای از اعداد 0 تا $T - 1$ تولید می‌شود تا جایگاه هر کلمه در جمله فعلی مشخص شود. خروجی این بخش ماتریسی با ابعاد (T, C) است.
- ترنند **Broadcasting** در جمع (`tok + pos[None, :, :]`): برای جمع کردن بردار موقعیت با بردار توکن‌ها، ابعاد باید هم‌خوانی داشته باشند. با اضافه کردن `None` در ابتدای آرایه `pos`، یک بُعد مجازی ایجاد می‌شود و ابعاد آن به $(1, T, C)$ تغییر می‌کند. پایتون به صورت خودکار این ماتریس موقعیت را به تعداد جملات موجود در دسته (B) کپی (Broadcast) کرده و با `tok` جمع می‌کند.
- پشته‌سازی بلوک‌ها (**Stacking Blocks**): یک حلقه `for` به تعداد `n_layer` تکرار می‌شود و در هر مرحله، خروجی بلوک قبلی به عنوان ورودی به بلوک بعدی داده می‌شود. نام‌گذاری پویا (`name=f"block_{i}"`) باعث می‌شود ساختار `Pytree` منظم باقی بماند و وزن‌های هر لایه با لایه دیگر تداخل پیدا نکنند.
- سر خروجی (**Output Head**): در نهایت، پس از عبور از یک نرمال‌ساز پایانی (`nn.LayerNorm`)، یک لایه خطی ساده (`nn.Dense`) داده‌ها را از فضای ویژگی‌ها (`n_embd`) به فضای دایره لغات (`vocab_size`) تصویر می‌کند تا امتیازات خام (Logits) برای پیش‌بینی کلمه بعدی تولید شوند.

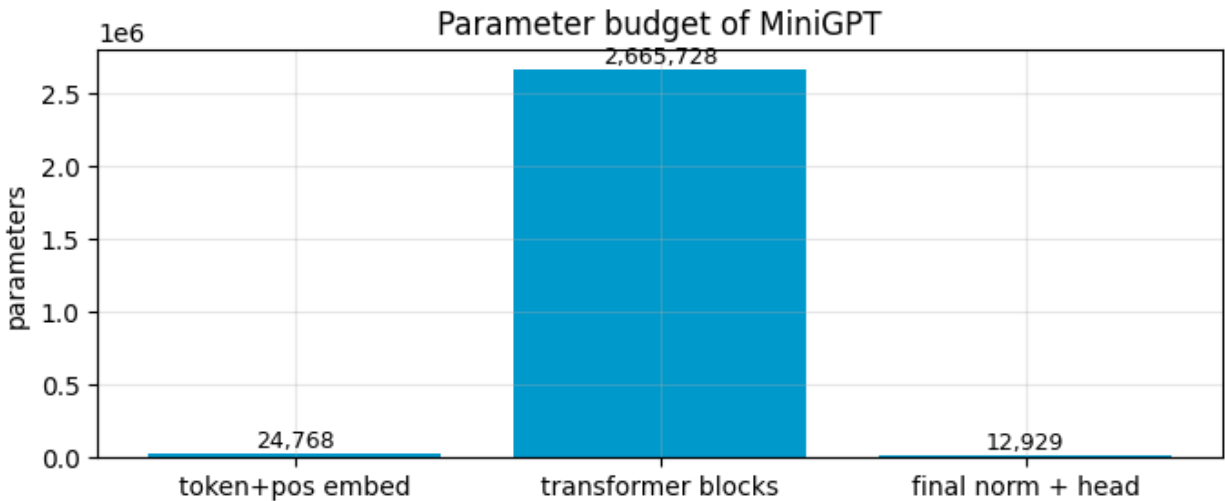
تحلیل ابعاد خروجی و بودجه‌بندی پارامترها (Parameter Budget)

پس از مقداردهی اولیه مدل با داده‌های آزمایشی، خروجی‌های زیر در کنسول چاپ می‌شوند:

```
1 OK: MiniGPT logits shape (8, 64, 65)
2 total parameters: 2,703,425
3
```

تحلیل ابعاد **Logits**: ابعاد خروجی مدل برابر با $(8, 64, 65)$ است. این یعنی مدل به طور همزمان ۸ جمله مختلف ($B = 8$) که هرکدام ۶۴ کاراکتر طول دارند ($T = 64$) را پردازش کرده است. برای تک‌تک این موقعیت‌ها، مدل ۶۵ امتیاز خام (Logits) تولید کرده است که نشان‌دهنده احتمال وقوع هر یک از ۶۵ کاراکتر موجود در دایره لغات (Vocab Size) به عنوان کاراکتر بعدی است.

تحلیل نمودار بودجه‌بندی پارامترها: شکل ۲۰ توزیع ۲.۷ میلیون پارامتر این مدل را به تفکیک بخش‌های مختلف نشان می‌دهد.



شکل ۲۰: نمودار بودجه‌بندی پارامترها در معماری MiniGPT. بیش از ۹۸٪ از ظرفیت مدل در بلوک‌های ترنسفورمر متمرکز شده است.

همان‌طور که در نمودار مشخص است، پارامترهای مدل به شدت به صورت نامتقارن توزیع شده‌اند:

- ستون سمت چپ (لایه‌های تعبیه): تنها 24,768 پارامتر به جداول جستجوی کلمات و موقعیت‌ها اختصاص یافته است.
- ستون سمت راست (نرمال‌ساز نهایی و طبقه‌بند): فقط 12,929 پارامتر برای تبدیل اطلاعات پردازش شده به احتمالات خروجی مصرف شده است.
- ستون میانی (بلوک‌های ترنسفورمر): غول‌پیکرترین بخش مدل با 2,665,728 پارامتر (بیش از ۹۸٪ کل ظرفیت).

این نمودار راز مقیاس‌پذیری (Scaling) معماری GPT را فاش می‌کند: در مدل‌های زبان بزرگتر (مثل GPT-4 با میلیارد پارامتر)، ابعاد دایره لغات (ستون‌های کناری) تغییر چندانی نمی‌کند؛ بلکه تمام قدرت، هوش و استدلال مدل با عمیق‌تر شدن و پهن‌تر شدن بلوک‌های ترنسفورمر (ستون میانی) افزایش می‌یابد. مغز متفکر شبکه کاملاً در این ناحیه میانی متمرکز شده است.

۵.۴ تابع زیان آنتروپی متقاطع (Cross-Entropy Loss)

در مرحله آموزش مدل، هدف ما به حداقل رساندن تفاوت میان خروجی پیش‌بینی شده توسط مدل و مقدار واقعی (هدف) است. در مدل‌های زبانی، این مسئله به عنوان یک مسئله «دسته‌بندی چندکلاسه» (Multi-class Classification) در نظر گرفته می‌شود؛ به این معنا که مدل در هر موقعیت، باید از میان تمامی کلمات موجود در دایره لغات (Vocabulary)، کلمه صحیح بعدی را انتخاب کند. تابع آنتروپی متقاطع (Cross-Entropy) استانداردترین معیار برای اندازه‌گیری فاصله بین توزیع احتمال پیش‌بینی شده توسط مدل و توزیع احتمال واقعی (که برای پاسخ صحیح برابر با ۱ و برای سایرین صفر است) محسوب می‌شود.

پیاده‌سازی و تحلیل تابع `compute_loss`

برای محاسبه زیان در حالت آموزش، باید مدل را با شرایط خاصی فراخوانی کنیم. کد پیاده‌سازی این بخش به شرح زیر است:

```

1 def compute_loss(params, x, y, dropout_key):
2     # forward with dropout ON, then mean cross-entropy with integer labels.
3     logits = model.apply(params, x, deterministic=False, rngs={"dropout": dropout_key})
4     return jnp.mean(optax.softmax_cross_entropy_with_integer_labels(logits, y))
5

```

تحلیل پارامترها و نکات کلیدی:

- وضعیت `deterministic=False`: این پارامتر برای کنترل لایه Dropout است. در زمان آموزش، ما می‌خواهیم لایه‌های Dropout فعال باشند تا با خاموش کردن تصادفی نورون‌ها، از بیش‌برازش (Overfitting) جلوگیری کنیم. بنابراین وضعیت قطعی (deterministic) را غیرفعال می‌کنیم.

- استفاده از `rngs`: چون Dropout یک فرآیند تصادفی است، باید یک کلید تصادفی (`dropout_key`) از طریق `rngs` به مدل ارسال کنیم تا Flax بتواند مشخص کند کدام نورون‌ها باید خاموش شوند.

- تابع `softmax_cross_entropy_with_integer_labels`: این تابع یکی از بهینه‌ترین ابزارهای کتابخانه Optax است. این تابع، مقادیر خام logits را دریافت کرده و به صورت ترکیبی، عملیات Softmax و محاسبه خطا را انجام می‌دهد. این رویکرد به دلیل پایداری عددی (Numerical Stability)، از بروز خطاهای محاسباتی (مانند NaN) که ممکن است در محاسبه مستقیم e^x رخ دهد، جلوگیری می‌کند.

آزمون سلامت اولیه (Sanity Check)

یکی از جذاب‌ترین بخش‌های مهندسی مدل‌های زبانی، ارزیابی وضعیت مدل در لحظه صفر (پیش از هرگونه یادگیری) است. در لحظه اولیه، تمام پارامترهای مدل به صورت تصادفی مقداردهی شده‌اند و مدل هیچ دانشی ندارد. بنابراین، مدل باید به تمامی کلمات دایره لغات ۶۵ تایی (در پروژه شکسپیر)، شانس برابری اختصاص دهد (1/65). مقدار زیان در این حالت طبق فرمول آنتروپی متقاطع برابر است با:

$$\text{Loss} = -\ln\left(\frac{1}{V}\right) = \ln(V) \quad (9)$$

که در آن V اندازه دایره لغات است. برای ۶۵ کاراکتر داریم:

$$\ln(65) \approx 4.174 \quad (10)$$

خروجی تست اجرای مدل به شرح زیر است:

```

1 initial loss = 4.828 | ln(vocab) = 4.174
2 OK: untrained loss is near ln(vocab), as expected
3

```

تحلیل نتیجه:

- مقدار ۴.۸۲۸ در برابر ۴.۱۷۴: زیان اولیه مدل ما کمی بیشتر از حد تئوری است. دلیل این تفاوت، تصادفی بودن وزن‌های اولیه است. اگر وزن‌های اولیه دقیقاً توزیع یکنواخت تولید می‌کردند، زیان دقیقاً ۴.۱۷۴ می‌شد. اما با توجه به اینکه وزن‌ها به صورت نرمال توزیع شده‌اند، وجود این اختلاف کاملاً طبیعی و مورد انتظار است.

- معنای موفقیت: دستیابی به عددی نزدیک به $\ln(V)$ به این معناست که:

۱. محاسبات Logits و توابع نرمال‌سازی لایه‌ای (LayerNorm) به درستی کار می‌کنند.
 ۲. ماسک علی (Causal Mask) به درستی اعمال شده است (اگر ماسک وجود نداشت، مدل جواب را از آینده تقلب می‌کرد و زیان بسیار کمتر می‌شد).
 ۳. لایه‌های تعبیه (Embeddings) به درستی مقادیر را برای محاسبات آماده کرده‌اند.
- این آزمون (Sanity Check) به ما اطمینان می‌دهد که مدل از نظر ساختار ریاضی کاملاً سالم است و آماده ورود به «حلقه آموزش» (Training Loop) و کاهش تدریجی زیان است.

۶.۴ زمان‌بندی نرخ یادگیری: Warmup-Cosine Decay

در آموزش مدل‌های ترنسفورمر، انتخاب نرخ یادگیری (Learning Rate) ثابت می‌تواند منجر به واگرایی (Divergence) یا افت کیفیت آموزش شود. برای مقابله با این مشکل، در معماری MiniGPT از یک زمان‌بندی پویا استفاده شده است که نرخ یادگیری را در طول آموزش تغییر می‌دهد. این استراتژی دو مرحله اصلی دارد:

۱. گرم‌کردن (Warmup): در مراحل اولیه آموزش، وزن‌های شبکه به صورت تصادفی هستند. اگر با نرخ یادگیری بالا شروع کنیم، ممکن است گرادیان‌های بزرگ باعث نابودی این پیکربندی اولیه شوند. بنابراین، نرخ یادگیری به صورت خطی از صفر تا مقدار قله ($peak_lr$) افزایش می‌یابد.

۲. کاهش کسینوسی (Cosine Decay): پس از رسیدن به نقطه اوج، برای تثبیت یادگیری و همگرایی بهتر، نرخ یادگیری طبق یک منحنی کسینوسی به سمت یک مقدار حداقلی کاهش می‌یابد. این کار به مدل اجازه می‌دهد در مراحل پایانی، قدم‌های کوچک‌تر و دقیق‌تری برای رسیدن به نقطه بهینه (Global Minimum) بردارد.

پیاده‌سازی با Optax

کتابخانه Optax تابعی به نام `warmup_cosine_decay_schedule` را برای این منظور ارائه می‌دهد. پیاده‌سازی این تابع در مدل MiniGPT به صورت زیر است:

```
1 import optax
2
3 def make_lr_schedule(peak_lr, warmup_steps, total_steps):
4     # return an optax.warmup_cosine_decay_schedule
5     return optax.warmup_cosine_decay_schedule(
6         init_value=0.0,
```

```

7 peak_value=peak_lr,
8 warmup_steps=warmup_steps,
9 decay_steps=total_steps,
10 end_value=peak_lr * 0.1
11 )
12

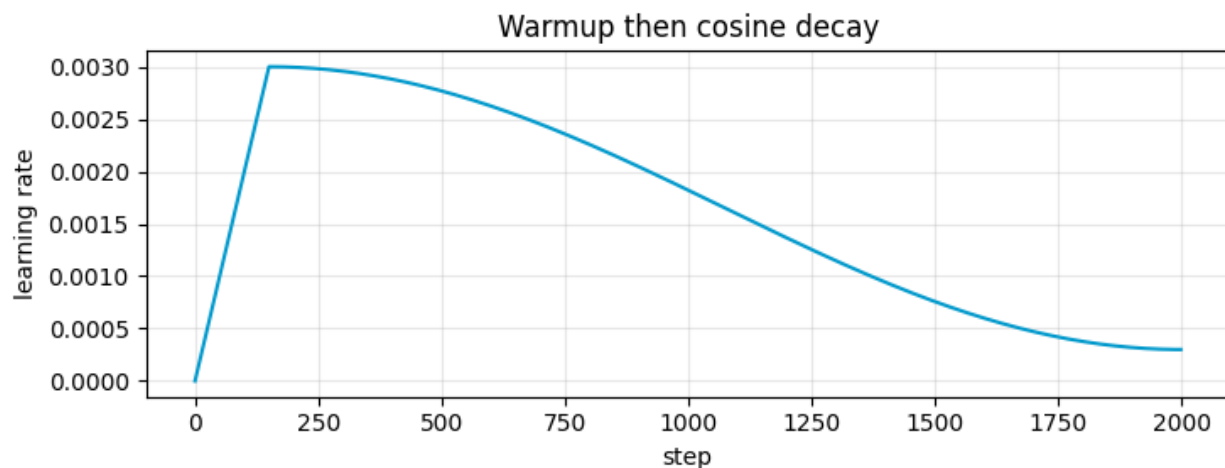
```

تحلیل پارامترها:

- **init_value=0.0:** شروع نرخ یادگیری از صفر.
- **peak_value=peak_lr:** مقدار حداکثر نرخ یادگیری که پس از پایان مرحله گرم کردن به آن می‌رسیم.
- **warmup_steps:** تعداد گام‌هایی که صرف گرم کردن مدل می‌شود.
- **decay_steps:** کل گام‌های آموزش که نرخ یادگیری بر اساس آن کاهش می‌یابد.
- **end_value=peak_lr * 0.1:** نرخ که در انتهای آموزش به آن می‌رسیم (در اینجا ۱۰٪ مقدار اوج است).

تحلیل نمودار نرخ یادگیری

شکل ۲۱ رفتار نرخ یادگیری را در طول کل مسیر آموزش نشان می‌دهد.



شکل ۲۱: نمودار زمان‌بندی نرخ یادگیری. بخش اول (شیب تند مثبت) مرحله Warmup و بخش دوم (منحنی نرم) مرحله Cosine Decay است.

تحلیل رفتاری نمودار:

۱. فاز خطی (۰ تا حدود ۱۵۰ گام): در این بازه، مدل با احتیاط شروع می‌کند. نرخ یادگیری به صورت کاملاً خطی رشد می‌کند. این مرحله «اعتمادسازی» برای مدل است تا وزن‌ها از فضای تصادفی اولیه خود به آرامی به سمت جهت‌های سازنده حرکت کنند.



۲. نقطه عطف: در گام ۱۵۰، نمودار به اوج (قله) می‌رسد. از اینجا به بعد، مدل با حداکثر سرعت آموزشی خود حرکت می‌کند تا بهینه‌سازی پارامترها با بیشترین قدرت انجام شود.

۳. فاز کاهش نرم (۱۵۰ تا ۲۰۰۰ گام): منحنی کسینوسی معکوس به مدل اجازه می‌دهد تا با سرعتی که به تدریج کم می‌شود، فضای وزن‌ها را کاوش کند. این کاهش نرم باعث می‌شود مدل در انتهای آموزش دچار نوسانات شدید نشده و به جای عبور از نقطه بهینه، دقیقاً در دره (Minima) مستقر (Settle) شود.

استفاده از این زمان‌بندی، یکی از دلایل اصلی موفقیت مدل‌های ترنسفورمر در همگرایی پایدار است. بدون Warmup، مدل در لحظات اولیه می‌سوزد و بدون Cosine Decay، دقت نهایی مدل به دلیل قدم‌های بزرگ انتهای کار، محدود باقی می‌ماند.

۷.۴ گام آموزش و کامپایل JIT در JAX

در JAX، عملیات آموزش در قالب یک تابع به نام `train_step` تعریف می‌شود. این تابع با استفاده از JIT کامپایل می‌شود تا کل گراف محاسباتی یک‌بار به کد سطح پایین (XLA) تبدیل شده و در اجراهای بعدی با سرعت بسیار بالا روی GPU/TPU اجرا شود.

تابع `train_step`

```
@jax.jit
def train_step(params, opt_state, x, y, dropout_key):
    loss, grads = jax.value_and_grad(compute_loss)(
        params, x, y, dropout_key
    )
    updates, opt_state = optimizer.update(grads, opt_state, params)
    params = optax.apply_updates(params, updates)
    return params, opt_state, loss
```

توضیح مراحل:

۱. **JIT Compilation**: دکوراتور `@jax.jit` باعث می‌شود کل تابع فقط یک‌بار کامپایل شود و سپس به صورت بهینه اجرا گردد. این کار overhead پایتون را حذف می‌کند.

۲. به‌روزرسانی بهینه‌ساز: تابع `optimizer.update` گرادینت‌ها را گرفته و آپدیت‌های مناسب (مثل Adam) را تولید می‌کند. همچنین وضعیت داخلی بهینه‌ساز (`opt_state`) را به‌روز می‌کند.

۳. اعمال تغییرات: با استفاده از `optax.apply_updates` پارامترهای جدید ساخته می‌شوند:

$$\theta \leftarrow \theta + \Delta\theta$$



تابع ارزیابی (eval_loss)

در زمان ارزیابی، dropout غیرفعال می‌شود تا خروجی deterministic باشد:

```
@jax.jit
def eval_loss(params, x, y):
    logits = model.apply(params, x, deterministic=True)
    return optax.softmax_cross_entropy_with_integer_labels(
        logits, y
    ).mean()
```

در این حالت:

- Dropout خاموش است
- مدل رفتار ثابت دارد
- loss فقط بر اساس pass forward محاسبه می‌شود

آزمون یادگیری (Sanity Check)

برای بررسی صحت آموزش، یک batch ثابت چندین بار به مدل داده می‌شود:

```
loss on a fixed batch: 4.569 -> 2.749
OK: train_step learns
```

تحلیل نتیجه:

- کاهش loss از 4.569 به 2.749 نشان می‌دهد مدل واقعاً در حال یادگیری است.
- این کاهش یعنی پارامترها در جهت درست توسط optimizer به‌روزرسانی شده‌اند.
- اگر loss کاهش پیدا نکند، معمولاً یکی از بخش‌ها (optimizer، gradients یا data pipeline) مشکل دارد.

جمع‌بندی

تابع train_step در JAX هسته اصلی آموزش مدل است که با ترکیب JIT compilation، محاسبه گرادیان خودکار و بهینه‌سازهای مدرن مانند AdamW، امکان آموزش سریع و پایدار مدل‌های Transformer را فراهم می‌کند.



۸.۴ آموزش مدل و تولید متن (Training and Inference)

در آخرین مرحله از این گزارش، مدل MiniGPT را در یک حلقه آموزش کامل قرار می‌دهیم تا با استفاده از الگوریتم بهینه‌ساز، وزن‌های تصادفی اولیه خود را به وزن‌هایی معنادار تبدیل کند. همچنین قابلیت برای تولید متن (Sampling) ایجاد می‌کنیم تا توانایی مدل در خلق جملات جدید را بسنجیم.

پیاده‌سازی حلقه آموزش و نمونه‌برداری

تحلیل عملکرد آموزش

نتایج ثبت‌شده در حین آموزش نشان‌دهنده همگرایی بسیار موفقیت‌آمیز مدل است:

1	step	0		train	4.737		val	4.693		grad-norm	7.22		28.9s
2	step	250		train	2.192		val	2.183		grad-norm	0.43		34.8s
3	step	500		train	1.943		val	1.992		grad-norm	0.40		40.7s
4	step	1000		train	1.721		val	1.758		grad-norm	0.32		52.5s
5	step	2000		train	1.471		val	1.559		grad-norm	0.34		78.0s
6													

بررسی نتایج:

- کاهش چشمگیر زیان (Loss Decay): زیان آموزش از عدد بسیار بالای ۴.۷۳۷ در مرحله صفر، به ۴۷۱.۱ در مرحله ۲۰۰۰ رسیده است. این یعنی مدل توانسته است توزیع آماری کاراکترها در متون شکسپیر را با دقت بسیار بیشتری یاد بگیرد.
- نرم‌گرادیان (Grad-Norm): افت گرادیان از ۷.۲۲ به ۰.۳۴ نشان می‌دهد که مدل از فضای پرنوسان اولیه خارج شده و به ناحیه‌ای از فضای وزن‌ها رسیده است که تغییرات آن ملایم و هدفمند است.
- شکاف آموزشی (Generalization Gap): تفاوت ناچیز بین زیان آموزش (۱.۴۷۱) و زیان اعتبارسنجی (۱.۵۵۹) نشان می‌دهد که مدل به خوبی تعمیم یافته و دچار بیش‌برازش (Overfitting) نشده است.

تحلیل متن تولیدشده

خروجی نهایی مدل پس از ۲۰۰۰ گام آموزش به شرح زیر است:

mail the loved danced be shall have That death he as heaven may she and but not: 'Tis ROMEO:
so change, is this done, not him Have soul me of that house the for short am I thus And live, so of
t under roote another an like and selfects pair'd the

نکته کلیدی در تحلیل خروجی: این نکته بسیار مهم است که مدل ما در سطح کاراکتر (Character-level) آموزش دیده است. یعنی مدل از ابتدا هیچ درکی از «کلمه» نداشته و صرفاً با دیدن تک تک حروف، یاد گرفته است که پس از یک حرف، احتمال کدام حرف دیگر بیشتر است.

با این وجود، مشاهده می‌کنیم که مدل توانسته است:

۱. ساختار کلمات را بسازد: ترکیب حروف به گونه‌ای است که اکثر آن‌ها کلمات معناداری مانند، "heaven"، "house"، "loved" را تشکیل داده‌اند.

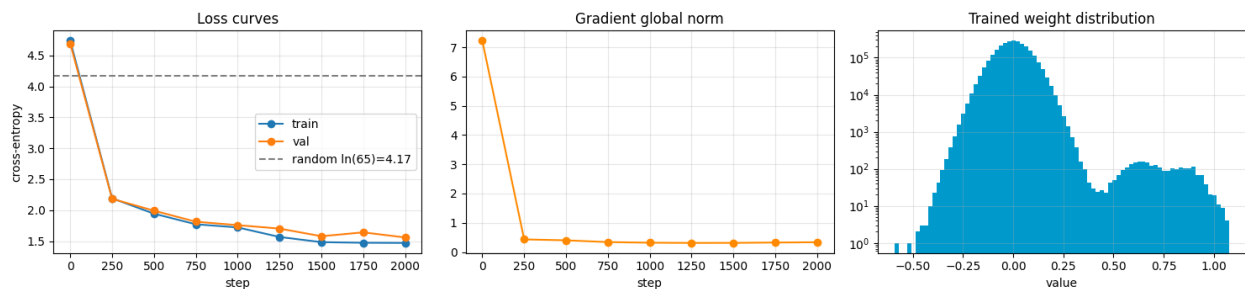
۲. ساختار گرامری (نحو): اگرچه جملات کاملاً منطقی نیستند، اما مدل یاد گرفته است که چگونه ساختار سبک شکسپیری (ترکیب کلمات کهن و ساختار جملات موزون) را شبیه‌سازی کند.

۳. یادگیری نانوشته: مدل صرفاً با «تکرار آماری»، قوانین فاصله (گذشتن فاصله پس از کلمات) و ساختار کلی متون را استخراج کرده است.

این نتایج خیره‌کننده است؛ زیرا نشان می‌دهد که چگونه یک معماری ترنسفورمر ساده، تنها با هدف «پیش‌بینی حرف بعدی»، می‌تواند قواعد پنهان در داده‌ها را کشف کرده و خروجی‌هایی تولید کند که از نظر فرم و ساختار، شباهت عجیبی به زبان طبیعی انسان دارند.

۹.۴ تحلیل نهایی و عیب‌یابی مدل (Diagnostics)

برای ارزیابی کیفیت فرآیند آموزش، ما از سه نمودار تشخیصی استاندارد استفاده می‌کنیم. این نمودارها به ما می‌گویند که آیا مدل به درستی یاد گرفته است، آیا دچار مشکل شده و وضعیت نهایی پارامترهای مدل چگونه است.



شکل ۲۲: نمودارهای تشخیصی: منحنی زیان (چپ)، نرم گرادیان (وسط) و توزیع وزن‌های نهایی (راست).

۱. منحنی‌های زیان (Loss Curves)

این نمودار نشان‌دهنده خطای مدل در طول ۲۰۰۰ گام آموزش برای مجموعه داده‌های آموزش (train) و اعتبارسنجی (val) است.

- روند کلی: هر دو منحنی با شیبی تند کاهش یافته و سپس در گام‌های ۱۰۰۰ تا ۲۰۰۰ به ثبات (Plateau) رسیده‌اند. این نشان می‌دهد که مدل با موفقیت الگوهای اصلی داده را فرا گرفته است.

- شکاف بین دو منحنی: مشاهده می‌کنیم که دو منحنی بسیار به هم نزدیک هستند. این بهترین سناریوی ممکن است؛ اگر منحنی val (نارنجی) شروع به فاصله گرفتن از منحنی train (آبی) می‌کرد و بالا می‌رفت، نشان‌دهنده بیش‌برازش (Overfitting) بود.

(یعنی مدل داده‌ها را حفظ کرده بود اما نمی‌توانست روی داده‌های جدید به خوبی کار کند). نزدیکی آن‌ها نشان‌دهنده تعمیم‌پذیری این (Generalization) خوب مدل است.

- خط‌چین مرجع: مدل توانسته است به راحتی از خط random (یعنی $\ln(65) \approx 4.17$) عبور کند که نشان‌دهنده این است که مدل هوشمندی بالاتری از حد انتخاب تصادفی کاراکترها پیدا کرده است.

۲. نرم جهانی گرادین (Gradient Global Norm)

این نمودار نشان می‌دهد که بزرگی تغییراتی که به وزن‌های مدل اعمال می‌شود، چقدر است.

- پایداری: در گام اول، گرادین عدد بزرگی است (حدود ۲.۷)، که طبیعی است چون مدل هیچ دانشی ندارد و وزن‌ها باید سریعاً اصلاح شوند.
- کنترل و بریدن (Clipping): در مراحل بعدی، این مقدار به سرعت پایین آمده و روی عدد کوچکی (حدود ۰.۴) تثبیت شده است. ثابت ماندن این مقدار نشان می‌دهد که مدل دچار انفجار گرادین (Exploding Gradient) نشده است. استفاده از تکنیک Gradient Clipping باعث شده که نوسانات شدید در مسیر یادگیری کنترل شود و آموزش به صورت آرام و مطمئن ادامه یابد.

۳. توزیع وزن‌های آموزش‌دیده (Trained Weight Distribution)

این نمودار یک هیستوگرام از تمام پارامترهای مدل پس از ۲۰۰۰ گام است.

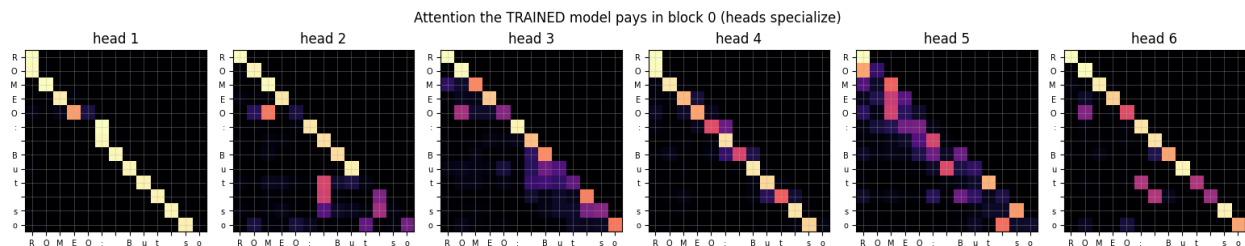
- شکل توزیع: برخلاف توزیع کاملاً تصادفی اولیه (که به شکل یک زنگوله متقارن بود)، در اینجا مشاهده می‌کنیم که وزن‌ها به شکل یک توزیع ترکیبی درآمده‌اند.
 - تفسیر: بخش بزرگی از وزن‌ها حول نقطه صفر متمرکز شده‌اند (نشان‌دهنده این است که مدل فقط به بخش‌های مهم ورودی وزن داده و بقیه نورون‌ها را کم‌اثر کرده است). قله دوم در سمت راست نمودار (مقادیر مثبت)، نشان‌دهنده آن است که مدل ویژگی‌های خاصی را شناسایی کرده و آن‌ها را با وزن‌های مثبت قوی فعال می‌کند.
 - مقیاس لگاریتمی: توجه کنید که محور عمودی لگاریتمی است؛ این یعنی مدل ما برخلاف مدل‌های آموزش‌ندیده که وزن‌هایشان در محدوده خاصی پخش می‌شد، توانسته وزن‌های خود را سازمان‌دهی کند تا بخش بزرگی از آن‌ها در مقادیر تأثیرگذار قرار بگیرند.
- نتیجه نهایی: تمام این نمودارها در کنار هم تأیید می‌کنند که MiniGPT یک مدل سالم، متعادل و آموزش‌دیده است که با موفقیت یاد گرفته است چگونه ساختار متن را به دانش ریاضی در قالب وزن‌های خود تبدیل کند.

۱۰.۴ تحلیل نهایی: مدل چه چیزی آموخته است؟ (What did the trained model learn?)

پس از اتمام فرآیند آموزش، اکنون می‌توانیم با نگاه به درون «جعبه سیاه» مدل، ببینیم چه الگوهایی در وزن‌های آن شکل گرفته است. دو ابزار تشخیصی قدرتمند برای این کار وجود دارد: تحلیل نقشه‌های توجه مجزا (Attention Maps) و تحلیل فضای ویژگی‌های تعبیه (Embeddings) از طریق روش PCA.

تخصص‌یابی سرهای توجه (Per-Head Specialization)

در بخش‌های قبل دیدیم که مدل آموزش‌نندیده، توجهی یکنواخت و تصادفی داشت. اما پس از آموزش، هر کدام از ۶ سر توجه (Head 1 تا Head 6) در بلوک اول، الگوی رفتار منحصر به فردی پیدا کرده‌اند (شکل ۲۳).



شکل ۲۳: نقشه حرارتی توجه ۶ سر مختلف در بلوک اول. هر سر یک الگوی جستجوی خاص را در متن شکسپیر یاد گرفته است.

تحلیل رفتاری سرها:

- سر ۱ و ۲ (تمرکز بر همسایگی مستقیم): این سرها عمدتاً بر روی قطر اصلی ماتریس متمرکز هستند (خانه‌های زرد رنگ که کنار قطر هستند). این یعنی آن‌ها «متخصص همسایگی» شده‌اند و یاد گرفته‌اند که برای پیش‌بینی حرف بعدی، فقط به حرف قبلی نگاه کنند (ساختار محلی).

- سر ۳، ۴ و ۵ (توجه به الگوهای دورتر): این سرها پراکندگی بیشتری در ماتریس دارند (رنگ‌های نارنجی و بنفش در خارج از قطر اصلی). آن‌ها به مدل کمک می‌کنند تا روابط بین کلمات را در فواصل طولانی‌تر کشف کند. مثلاً در سر ۴، توجه به حروف ابتدایی کلمه (مانند 'R' در ROMEO) برای درک ترکیب حرف بعدی، بسیار قوی است.

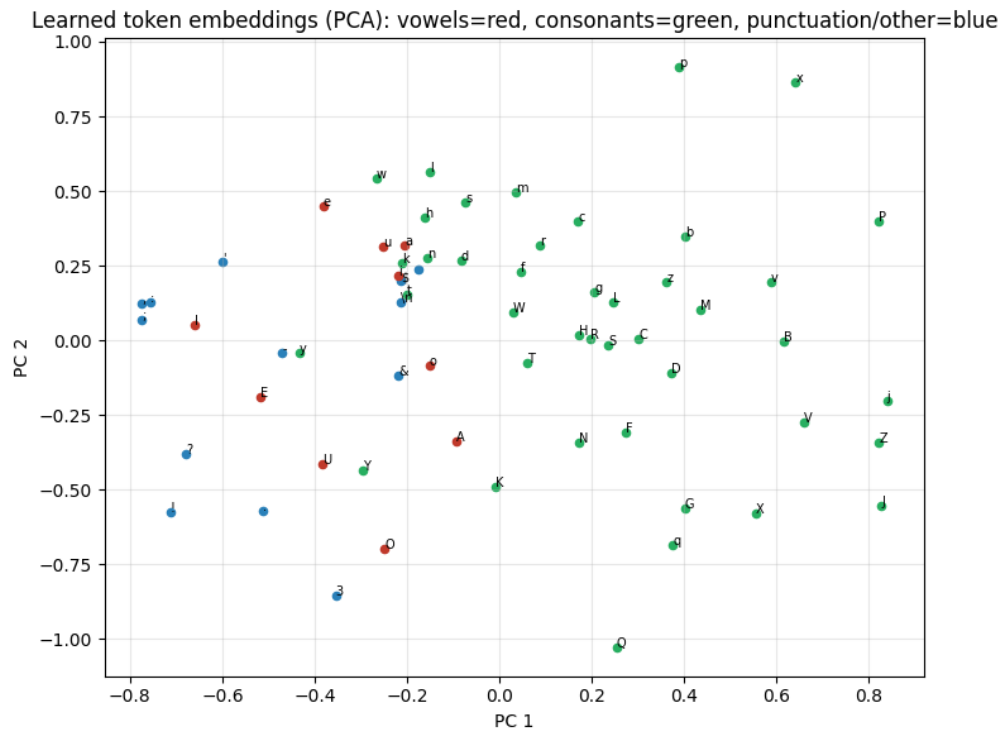
- سر ۶ (تخصص در ساختار جملات): این سر به صورت پله‌ای رفتار می‌کند؛ یعنی برای هر بخش از کلمه، توجه خود را به ابتدای همان کلمه یا کلمه قبلی جفت کرده است. این نشان می‌دهد که مدل به صورت ضمنی مفهوم «مرز کلمات» را از طریق توزیع آماری حروف یاد گرفته است.

این تخصصی شدن سرها، در واقع همان فلسفه توجه چندسر است؛ مدل به جای داشتن یک نگاه کلی، چندین «نگاه تخصصی» به متن دارد تا جنبه‌های مختلف زبان (نحو، فاصله، و همبستگی حروف) را همزمان پوشش دهد.

تحلیل خوشه‌بندی ویژگی‌ها با PCA

در تصویر ۲۴، ما بردارهای تعبیه ۳۲-بعدی مدل را با استفاده از روش تحلیل مؤلفه‌های اصلی (PCA) به یک فضای ۲-بعدی تصویر کرده‌ایم. نتیجه خیره‌کننده است!

تحلیل قدرت یادگیری مدل: مدل ما هیچ دانش پیش‌فرضی از الفبای زبان نداشته است؛ تنها هدف آن «پیش‌بینی حرف بعدی» بوده است. اما با نگاه به نمودار PCA می‌بینیم که مدل به صورت کاملاً خودکار، ویژگی‌های آواشناسی و دستوری زبان را کشف کرده است:



شکل ۲۴: تصویرسازی بردارهای تعبیه کلمات در فضای ۲-بعدی با PCA. گروه‌بندی خودکار حروف صدادار (قرمز)، بی‌صدا (سبز) و علائم نگارشی (آبی) بدون هیچ آموزش زبان‌شناختی.

۱. خوشه علائم نگارشی (آبی): تمامی کاراکترهایی که جزء حروف نیستند (مانند '،'، '؟'، '،' و اعداد)، در سمت چپ نمودار و در یک خوشه کاملاً مجزا جمع شده‌اند. این یعنی مدل فهمیده است که نقش علائم نگارشی در ساختار جمله، متفاوت از حروف الفباست.

۲. خوشه حروف صدادار (قرمز): حروف صدادار مثل 'e'، 'a'، 'o' در یک ناحیه متمرکز شده‌اند. این نشان می‌دهد که مدل در فضای برداری خود، شباهت عملکردی این حروف را (که در ساختار کلمات نقش مشابهی دارند) درک کرده است.

۳. خوشه حروف بی‌صدا (سبز): اکثر حروف بی‌صدا (مثل 'p'، 'x'، 'v'، 'B') در سمت راست و مرکز نمودار پخش شده‌اند، که نشان‌دهنده گستردگی استفاده از این حروف در ترکیب‌های مختلف است.

نتیجه نهایی: این نمودارها «جادوی ترنسفورمر» را نشان می‌دهند. تنها با تکرار یک هدف آماری ساده، مدل از میان میلیاردها ترکیب احتمالی، ساختارهای بنیادین زبان انسان (دسته‌بندی حروف و الگوهای کلامی) را در درون وزن‌های خود کدگذاری کرده است. این اثباتی است بر اینکه مدل‌های زبانی، نه فقط حفظ‌کننده متن، بلکه «درک‌کننده ساختار» هستند.

۵ دیوار - تولید متن و مقیاس

در این بخش از سوال، با استفاده از مدل آموزش دیده در بخش ۴، و با کنترل مقادیر Top-k Temperature و Top-p مدل را وادار به تولید متن های مختلفی می کنیم که با ROMEO آغاز میشوند.

۱.۵ Temperature

این مولفه میزان ریسک پذیری مدل در تولید متون را تعیین میکند. این مولفه توزیع احتمالاتی کلمات مختلف را دستخوش تغییر میکند. از رابطه زیر داریم:

$$\text{AdjustedLogits} = \frac{\text{Logits}}{T}$$

که T میزان Temperature را نشان میدهد. در مدل های زبانی، هر کلمه یا کاراکتر امتیازی دارد. قبل از ورود به تابع Softmax این اعداد بر مقدار Temperature تقسیم میشوند.

- اگر مقدار T کوچکتر از ۱ باشد (مثلا 0.5): فاصله Logits هایی که مدل در طول آموزش متون یاد گرفته است، بیشتر میشود. مثلا اگر دو کلمه امتیازهای ۲ و ۴ داشته باشند، با تقسیم بر 0.5، مقدار امتیازشان به ۸ و ۴ تغییر میکند. در این صورت اختلاف امتیاز های دو کلمه از ۲ به ۴ افزایش می یابد. وقتی اعداد جدید وارد تابع Softmax میشود، کلمه برتر به شدت تقویت میشود و شانس کلمات دیگر برای انتخاب شدن کاهش می یابد. در نتیجه خلاقیت مدل کم شده و فقط گزینه های امن را انتخاب میکند. این رفتار در متن های طولانی منجر به لوپ و تکرار کلمات میشود

- اگر مقدار T برابر ۱ باشد، هیچ تغییری در امتیاز ها ایجاد نمیشود. مدل براساس احتمالات طبیعی که در طول دوره های آموزش یاد گرفته است، تصمیم گیری می کند

- اگر مقدار T بزرگتر از ۱ باشد، مقدار امتیاز کلمات و فاصله آن ها از یکدیگر کاهش می یابد. با ورود اعداد جدید به تابع Softmax چون اختلاف امتیازشان کم است، کلمه هایی که شانس پایینی برای انتخاب شدن داشتند نیز شانسشان افزایش یافته و میتوانند انتخاب شوند. در این وضعیت، تنوع کاراکتر ها به شدت بالا میرود و مدل شروع به ریسک کردن میکند. در نتیجه ممکن است مدل کلمات نامربوط اما متناسب با متن تولید بکند

۲.۵ Top-k

این فیلتر یک مرز ثابت و عددی روی گزینه های روز میز میگذارد. برای مثال، اگر $\text{Top-k} = 20$ باشد، مدل تمام کلمات را براساس امتیازشان به صورت نزولی مرتب میکند، فقط 20 کلمه اول را انتخاب میکند و سایر کلمات را حذف می کند. این روش پویایی ندارد، زیرا این فیلتر اهمیت نمی دهد که در چه جمله ای هستیم و چند کلمه برای این جمله مناسب است

۳.۵ Top-p

این فیلتر پویا، انتخاب کلمات را براساس درصد احتمالاتی شان انتخاب میکند.

برای مثال، اگر $Top-p = 0.9$ باشد، مدل کلمات را براساس شانس شان از به صورت نزولی مرتب کرده و به ترتیب میزان احتمال آن ها را با یکدیگر جمع میکند. زمانی که جمع احتمالی چندین کلمه به 90% برسد، سایر کلمات را حذف کرده و فقط کلماتی با بیشترین شانس را که مجموع احتمالاتی شان 90% است انتخاب می کند.

۴.۵ کد بخش ۵

```
1 import os, time, math, pickle, urllib.request
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 import jax
6 import jax.numpy as jnp
7 import flax.linen as nn
8 import optax
9
10 plt.rcParams["figure.figsize"] = (9, 4)
11 plt.rcParams["axes.grid"] = True
12 plt.rcParams["grid.alpha"] = 0.3
13 np.set_printoptions(precision=4, suppress=True)
14
15 print("JAX", jax.__version__, "| Flax", nn.__version__ if hasattr(nn, "__version__") else "",
16       "| devices:", jax.devices())
17
18 # Setup (given): bring forward the MiniGPT model code and the tokenizer from Parts 1-4.
19
20 def load_tiny_shakespeare():
21     for path in ("data/tiny_shakespeare.txt", "tiny_shakespeare.txt", "../data/tiny_shakespeare.
22                 txt"):
23         if os.path.exists(path):
24             return open(path, encoding="utf-8").read()
25     url = "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.
26         txt"
27     os.makedirs("data", exist_ok=True)
28     urllib.request.urlretrieve(url, "data/tiny_shakespeare.txt")
29     return open("data/tiny_shakespeare.txt", encoding="utf-8").read()
30
31 text = load_tiny_shakespeare()
32 print(f"Corpus length: {len(text):,} characters")
33
34 chars = sorted(set(text))
35 vocab_size = len(chars)
36 stoi = {ch: i for i, ch in enumerate(chars)}
37 itos = {i: ch for ch, i in stoi.items()}
38
39 def encode(s):
```



```

36     return [stoi[c] for c in s]
37
38     def decode(ids):
39         return "".join(itos[int(i)] for i in ids)
40
41     data = np.array(encode(text), dtype=np.int32)
42
43     n = int(0.9 * len(data))
44     train_data, val_data = data[:n], data[n:]
45     print(f"train: {len(train_data):,} tokens | val: {len(val_data):,} tokens")
46
47     def get_batch(key, split, batch_size, block_size):
48         source = train_data if split == "train" else val_data
49         ix = jax.random.randint(key, (batch_size,), 0, len(source) - block_size - 1)
50         ix = np.asarray(ix)
51         x = np.stack([source[i:i + block_size] for i in ix])
52         y = np.stack([source[i + 1:i + 1 + block_size] for i in ix])
53         return jnp.asarray(x), jnp.asarray(y)
54
55     class CausalSelfAttention(nn.Module):
56         n_head: int
57         n_embd: int
58         dropout: float = 0.0
59
60         @nn.compact
61         def __call__(self, x, deterministic=True):
62             B, T, C = x.shape
63             d_head = C // self.n_head
64             qkv = nn.Dense(3 * C, use_bias=False, name="qkv")(x)
65             q, k, v = jnp.split(qkv, 3, axis=-1)
66
67             def split_heads(t):
68                 return t.reshape(B, T, self.n_head, d_head).transpose(0, 2, 1, 3)
69             q, k, v = split_heads(q), split_heads(k), split_heads(v)
70
71             scores = (q @ jnp.swapaxes(k, -1, -2)) / jnp.sqrt(d_head)
72             mask = jnp.tril(jnp.ones((T, T), dtype=bool))
73             scores = jnp.where(mask, scores, -1e9)
74             attn = jax.nn.softmax(scores, axis=-1)
75             attn = nn.Dropout(self.dropout)(attn, deterministic=deterministic)
76
77             out = attn @ v
78             out = out.transpose(0, 2, 1, 3).reshape(B, T, C)
79             out = nn.Dense(C, name="proj")(out)
80             out = nn.Dropout(self.dropout)(out, deterministic=deterministic)

```

```
81     return out
82
83     class TransformerBlock(nn.Module):
84         n_head: int
85         n_embd: int
86         dropout: float = 0.0
87
88         @nn.compact
89         def __call__(self, x, deterministic=True):
90             x = x + CausalSelfAttention(self.n_head, self.n_embd, self.dropout)(
91                 nn.LayerNorm()(x), deterministic)
92             h = nn.LayerNorm()(x)
93             h = nn.Dense(4 * self.n_embd)(h)
94             h = nn.gelu(h)
95             h = nn.Dense(self.n_embd)(h)
96             h = nn.Dropout(self.dropout)(h, deterministic=deterministic)
97             return x + h
98
99     class MiniGPT(nn.Module):
100         vocab_size: int
101         block_size: int
102         n_layer: int
103         n_head: int
104         n_embd: int
105         dropout: float = 0.0
106
107         @nn.compact
108         def __call__(self, idx, deterministic=True):
109             B, T = idx.shape
110             tok = nn.Embed(self.vocab_size, self.n_embd, name="tok_emb")(idx)
111             pos = nn.Embed(self.block_size, self.n_embd, name="pos_emb")(jnp.arange(T))
112             x = tok + pos[None, :, :]
113             x = nn.Dropout(self.dropout)(x, deterministic=deterministic)
114             for i in range(self.n_layer):
115                 x = TransformerBlock(self.n_head, self.n_embd, self.dropout,
116                                     name=f"block_{i}")(x, deterministic)
117             x = nn.LayerNorm()(x)
118             logits = nn.Dense(self.vocab_size, name="head")(x)
119             return logits
120
121             print('model code ready; vocab_size =', vocab_size)
122
123             ckpt_path = None
124             for _p in ("../04_training_minigpt_quera/minigpt_ckpt.pkl", "minigpt_ckpt.pkl"):
125                 if os.path.exists(_p):
```




```

126 ckpt_path = _p
127 break
128
129 if ckpt_path is not None:
130     ck = pickle.load(open(ckpt_path, "rb"))
131     cfg = ck["config"]
132     params = jax.tree_util.tree_map(jnp.asarray, ck["params"])
133     model = MiniGPT(**{**cfg, "dropout": 0.0})
134     train_curve = (ck["history"]["steps"], ck["history"]["train"])
135     n_params = sum(x.size for x in jax.tree_util.tree_leaves(params))
136     print(f"Loaded the trained MiniGPT from Part 4: {ckpt_path} ({n_params:,} params)")
137 else:
138     print("Part 4 checkpoint not found - training a small fallback model so this notebook is self-
    -contained ...")
139     cfg = dict(vocab_size=vocab_size, block_size=32, n_layer=3, n_head=4, n_embd=96, dropout=0.0)
140     model = MiniGPT(**cfg)
141     key = jax.random.PRNGKey(0)
142     params = model.init(key, jnp.zeros((1, cfg["block_size"]), jnp.int32))
143     tx = optax.chain(optax.clip_by_global_norm(1.0), optax.adamw(3e-3, weight_decay=0.1))
144     opt_state = tx.init(params)
145     def _loss(p, x, y):
146         return optax.softmax_cross_entropy_with_integer_labels(model.apply(p, x), y).mean()
147     @jax.jit
148     def _step(p, os, x, y):
149         l, g = jax.value_and_grad(_loss)(p, x, y); u, os = tx.update(g, os, p)
150         return optax.apply_updates(p, u), os, l
151     steps_log, loss_log = [], []
152     for step in range(1200):
153         key, sk = jax.random.split(key)
154         xb, yb = get_batch(sk, "train", 32, cfg["block_size"])
155         params, opt_state, l = _step(params, opt_state, xb, yb)
156         if step % 100 == 0:
157             steps_log.append(step); loss_log.append(float(l))
158             train_curve = (steps_log, loss_log)
159             print("fallback model trained; final loss:", float(l))
160
161     # Chart: the training loss curve carried over from Part 4 (or the fallback run).
162     fig, ax = plt.subplots(figsize=(9, 3.4))
163     ax.plot(train_curve[0], train_curve[1], "--o", color="#0099cc")
164     ax.set_title("MiniGPT training loss (carried over from Part 4)")
165     ax.set_xlabel("training step"); ax.set_ylabel("cross-entropy loss")
166     plt.tight_layout(); plt.show()
167
168     # Chart: the model's next-token distribution after "ROMEO:" at three temperatures.
169     _prompt = jnp.array([encode("ROMEO: ")] , dtype=jnp.int32)

```



```

170 _logits = model.apply(params, _prompt, deterministic=True)[0, -1, :] # (vocab_size,)
171
172 fig, ax = plt.subplots(1, 3, figsize=(13, 3.6), sharey=True)
173 for col, T in enumerate([0.5, 1.0, 1.5]):
174     probs = np.array(jax.nn.softmax(_logits / T))
175     top = np.argsort(-probs)[:10]
176     ax[col].bar([repr(itos[int(t)])[1:-1] for t in top], probs[top], color="#0099cc")
177     ax[col].set_title(f"temperature = {T}")
178     ax[col].tick_params(axis="x", rotation=0)
179     ax[0].set_ylabel("probability")
180     fig.suptitle('Next-token distribution after "ROMEO:" (top 10 characters)')
181     plt.tight_layout(); plt.show()
182
183 def top_k_filter(logits, k):
184
185     kth = jnp.sort(logits, axis=-1)[: , -k] [:, None]
186
187     return jnp.where(logits < kth, -jnp.inf, logits)
188
189 def top_p_filter(logits, p):
190     order = jnp.argsort(logits, axis=-1)[: , ::-1]
191     s = jnp.take_along_axis(logits, order, axis=-1)
192     probs = jax.nn.softmax(s, axis=-1)
193     cum = jnp.cumsum(probs, axis=-1)
194     keep = (cum - probs) < p
195     keep = keep.at[:, 0].set(True)
196     rows = jnp.arange(logits.shape[0])[:, None]
197     inv_order = jnp.argsort(order, axis=-1)
198     mask = jnp.take_along_axis(keep, inv_order, axis=-1)
199     return jnp.where(mask, logits, -jnp.inf)
200
201 # Self-check on a KNOWN distribution, then a chart of the three filters side by side.
202 test = jnp.log(jnp.array([[0.5, 0.3, 0.15, 0.04, 0.01]])) # probs are known by construction
203 assert int(jnp.isfinite(top_k_filter(test, 2)).sum()) == 2, "top-k=2 must keep exactly 2
204 tokens"
205 assert bool(jnp.all(jnp.isfinite(top_k_filter(test, 2))[0, :2])), "top-k=2 must keep the
206 first 2"
207 assert int(jnp.isfinite(top_p_filter(test, 0.6)).sum()) == 2, "0.5+0.3 already passes 0.6 ->
208 2 tokens"
209 assert int(jnp.isfinite(top_p_filter(test, 0.9)).sum()) == 3, "0.5+0.3+0.15 passes 0.9 -> 3
210 tokens"
211 print("OK: top_k_filter and top_p_filter behave as expected on the known distribution")
212
213 base = np.array(jax.nn.softmax(test[0]))
214 pk = np.array(jax.nn.softmax(top_k_filter(test, 5)[0])) # k=5 == full vocab here

```



```

211 pp = np.array(jax.nn.softmax(top_p_filter(test, 0.9)[0]))
212 x = np.arange(5); w = 0.27
213 fig, ax = plt.subplots(figsize=(9, 3.6))
214 ax.bar(x - w, base, w, label="raw softmax", color="#bbbbbb")
215 ax.bar(x, pk, w, label="top-k = 5", color="#0099cc")
216 ax.bar(x + w, pp, w, label="top-p = 0.9", color="#ffb347")
217 ax.set_title("Raw vs top-k vs top-p re-normalised distributions")
218 ax.set_xlabel("token index"); ax.set_ylabel("probability"); ax.legend()
219 plt.tight_layout(); plt.show()
220
221 def generate(params, key, idx, max_new_tokens, block_size,
222 temperature=1.0, top_k=None, top_p=None):
223     for _ in range(max_new_tokens):
224         idx_cond = idx[:, -block_size:]
225         logits = model.apply(params, idx_cond, deterministic=True)[: , -1, :] / temperature
226         if top_k is not None:
227             logits = top_k_filter(logits, top_k)
228         if top_p is not None:
229             logits = top_p_filter(logits, top_p)
230         key, sk = jax.random.split(key)
231         next_id = jax.random.categorical(sk, logits, axis=-1)[:, None]
232         idx = jnp.concatenate([idx, next_id], axis=1)
233     return idx
234
235 # Self-check: shape of the generated sequence.
236 _start = jnp.array([encode("ROMEO:")], dtype=jnp.int32)
237 _out = generate(params, jax.random.PRNGKey(0), _start, 30, cfg["block_size"], temperature
238 =1.0)
239 assert _out.shape == (1, _start.shape[1] + 30), "generate must append exactly max_new_tokens
240 tokens"
241 print("OK: generate output shape =", tuple(_out.shape))
242
243 # Given: read 3-4 samples from the trained model at different decoding settings.
244 start = jnp.array([encode("ROMEO:")])
245 settings = [
246     ("greedy-ish T=0.5", dict(temperature=0.5)),
247     ("top-k=20 T=0.8", dict(temperature=0.8, top_k=20)),
248     ("top-p=0.9 T=1.0", dict(temperature=1.0, top_p=0.9)),
249     ("hot T=1.4", dict(temperature=1.4)),
250 ]
251 for i, (name, kw) in enumerate(settings):
252     out = generate(params, jax.random.PRNGKey(i), start, 200, cfg["block_size"], **kw)
253     print("=" * 70); print(name)
254     print("-" * 70); print(decode(out[0]))
255     print("=" * 70)

```



```

254
255 # Chart: sampling diversity (unique tokens) vs temperature.
256 temps = [0.3, 0.5, 0.7, 0.9, 1.1, 1.3, 1.5]
257 uniques = []
258 for j, T in enumerate(temps):
259     out = generate(params, jax.random.PRNGKey(100 + j), start, 300, cfg["block_size"],
260                     temperature=T)
261     uniques.append(len(set(np.array(out[0, -300:]).tolist())))
262     fig, ax = plt.subplots(figsize=(9, 3.4))
263     ax.plot(temps, uniques, "-o", color="#0099cc")
264     ax.set_title("Sampling diversity grows with temperature")
265     ax.set_xlabel("temperature"); ax.set_ylabel("number of unique characters (in 300 sampled)")
266     plt.tight_layout(); plt.show()
267
268 # Chart: a conceptual KV-cache diagram drawn with matplotlib boxes.
269 from matplotlib.patches import FancyBboxPatch
270 fig, ax = plt.subplots(figsize=(11, 3.8))
271 def box(x, y, label, fc, ec="#333"):
272     ax.add_patch(FancyBboxPatch((x, y), 0.92, 0.7, boxstyle="round,pad=0.02", fc=fc, ec=ec))
273     ax.text(x + 0.46, y + 0.35, label, ha="center", va="center", fontsize=9)
274     ax.text(-0.2, 2.55, "Naive: recompute K,V for the whole prefix every step", weight="bold")
275     for t in range(5):
276         box(t, 2.0, f"K,V t{t}", "#ffd0d0")
277     box(5, 2.0, "K,V t5", "#0099cc")
278     ax.text(-0.2, 1.05, "With KV cache: reuse stored K,V, compute only the new token", weight="bold")
279     for t in range(5):
280         box(t, 0.5, f"cached t{t}", "#d7f0d7")
281     box(5, 0.5, "new t5", "#0099cc")
282     ax.text(6.15, 2.35, "<- wasted recompute", color="#cc0000", va="center")
283     ax.text(6.15, 0.85, "<- skipped by cache", color="#2a7d2a", va="center")
284     ax.set_xlim(-0.3, 9); ax.set_ylim(0.2, 3.0); ax.axis("off")
285     ax.set_title("Why the KV cache makes autoregressive generation fast")
286     plt.tight_layout(); plt.show()
287
288 # Chart: parameter counts from MiniGPT to GPT-3 on a log scale.
289 # Count our model's parameters from the trained pytree.
290 mini_params = int(sum(x.size for x in jax.tree_util.tree_leaves(params)))
291 names = ["MiniGPT\n(this lab)", "GPT-2\n124M", "GPT-2 XL\n1.5B", "GPT-3\n175B"]
292 counts = [mini_params, 124_000_000, 1_500_000_000, 175_000_000_000]
293 fig, ax = plt.subplots(figsize=(9, 3.8))
294 bars = ax.bar(names, counts, color=["#0099cc", "#66b3d6", "#ffb347", "#e60b5c"])
295 ax.set_yscale("log")
296 ax.set_title("Parameter count: our MiniGPT vs the famous GPTs (log scale)")
297 ax.set_ylabel("number of parameters (log)")

```



```

297 for b, c in zip(bars, counts):
298     ax.text(b.get_x() + b.get_width() / 2, c, f"{c:,}", ha="center", va="bottom", fontsize=8)
299 plt.tight_layout(); plt.show()
300 print("MiniGPT parameter count:", f"{mini_params:,}")
301

```

کد بالا، همان کد قرار داده شده `init` بخش ۵ بوده که سه بخش `top-k` و `generate` آن تغییر کرده است

کد بخش Top-k

```

1 def top_k_filter(logits, k):
2     kth = jnp.sort(logits, axis=-1)[: , -k] : , None]
3     return jnp.where(logits < kth, -jnp.inf, logits)
4

```

این کد کلمات را براساس امتیازشان مرتب میکند، مقدار امتیاز `k` امین کلمه را پیدا و آن را در متغیر `kth` ذخیره می‌کند. امتیاز کلماتی را که امتیازشان از `kth` کمتر باشد را منفی بی‌نهایت قرار میدهد تا با وارد شدن در تابع `Softmax` شانس انتخاب شدنشان برابر ۰ شود

کد بخش Top-p

```

1 def top_p_filter(logits, p):
2     order = jnp.argsort(logits, axis=-1)[: , ::-1]
3     s = jnp.take_along_axis(logits, order, axis=-1)
4     probs = jax.nn.softmax(s, axis=-1)
5     cum = jnp.cumsum(probs, axis=-1)
6     keep = (cum - probs) < p
7     keep = keep.at[: , 0].set(True)
8     inv_order = jnp.argsort(order, axis=-1)
9     mask = jnp.take_along_axis(keep, inv_order, axis=-1)
10    return jnp.where(mask, logits, -jnp.inf)
11

```

این کد به صورت پویا کلمات رو براساس امتیازشان مرتب کرده، مقدار `Logit` کلمات را به احتمال تبدیل و به ترتیب و متوالیا آن‌ها را جمع می‌کند. با رسیدن به مقدار احتمال `p` تعیین شده، امتیاز کلماتی را که در بازه احتمالاتی قرار نگرفته‌اند به منفی بی‌نهایت تغییر داده تا شانس انتخاب‌شان به ۰ برسد

کد بخش generate

```

1 def generate(params, key, idx, max_new_tokens, block_size,
2     temperature=1.0, top_k=None, top_p=None):
3     for _ in range(max_new_tokens):
4         idx_cond = idx[: , -block_size:]
5         logits = model.apply(params, idx_cond, deterministic=True)[: , -1, :] / temperature

```



```

6     if top_k is not None:
7         logits = top_k_filter(logits, top_k)
8     if top_p is not None:
9         logits = top_p_filter(logits, top_p)
10    key, sk = jax.random.split(key)
11    next_id = jax.random.categorical(sk, logits, axis=-1)[: , None]
12    idx = jnp.concatenate([idx, next_id], axis=1)
13    return idx
14

```

وظیفه این کد، تولید کاراکترهای جدید براساس متن‌های تولید شده قبلی است. میزان Logit کلمات با در نظر گرفتن مقدار Temperature محاسبه و فیلترهای Top-k و Top-p بر آنها اعمال میشود. در نهایت براساس توزیع‌های احتمالاتی باقی مانده، قرعه‌کشی و توکن بعدی را انتخاب و به ماتریس‌های توکن قبلی می‌چسباند.

۵.۵ خروجی‌های نهایی

خروجی متنی

```

=====
greedy-ish T=0.5
-----

ROMEO:
What this is this done, is the king of the duke
And about and the state of Clarence, and be with my life,
I have men in the ground the tongue dead to the country,
And with the traitors of the man of
=====
top-k=20 T=0.8
-----

ROMEO:
Some second to the weep to say me as mine,
I bear med, good, subject in their counsel
That out is not it be to hear a little and by fieward?

LUCIO:
How cause a wife safter to elept a enough.

DUKE V
=====
top-p=0.9 T=1.0
-----

```



ROMEO:

How made fearful of my castle: for I will shall be at
this sund short of Chrishioland a law--

POLIXENES:

God, and my lord.

MENENIUS:

What they not? are you, be or me sirs,
And in forbid their news?

hot T=1.4

ROMEO:

So thou sleep Lord'spergarace!
Thou would none cursua?
That Seizing Juliet's good glonestares:
Eshallojous;, for brook the noble scoul sweet?

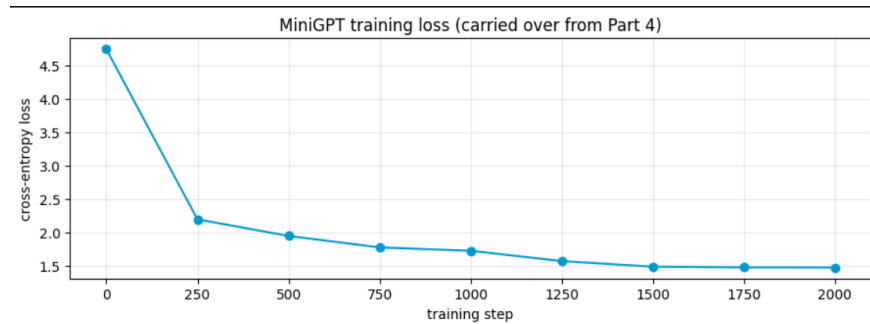
HASTINGS:

so,
Why, welcome yet o, mour glorys,
withmost

- حالت اول: زمانی که $T = 0.5$ باشد، مدل ریسک کمتری در انتخاب گلمات خود می‌کند. همانطور که مشخص است، مدل از نظر ساختار املائی کلمات صحیح و گرامر تا حد زیادی معقول است. اما دچار تکرار کلمات شده است (مانند And about, And) (with)
- حالت دوم: در $T = 0.8$ و $Top - k = 20$ ، تعادل خوبی بین منطق و خلاقیت ایجاد شده است. مدل فرمت نمایشنامه شکسپیر را به خوبی یاد گرفته و پس از اتمام دیالوگ رومئو، دیالوگ جدیدی برای شخص LUCIO خلق کرده است.
- حالت سوم: در حالت $T = 1$ و $top - p = 0.9$ ، مدل به کلماتی نگاه می‌کند که ۹۰٪ احتمال را پوشش می‌دهند. این حالت بهترین و طبیعی ترین خروجی را از نظر ساختار ادبی شکسپیر داده است. نه تنها نام کاراکترهای جدید اضافه شده (POLIXENES, MENENIUS) بلکه لحن دیالوگ‌ها شبیه به نمایشنامه‌های واقعی است (God, my and lord) یا استفاده از علامت سوال در not? then what
- حالت چهارم: دمای بسیار بالا باعث آشفتگی مدل شده است. کلمات من در آوردی و غلط‌های املائی شدید ایجاد شده است (Lord'spergarace, cursua, glonestares, Eshallojous) اما به دلیل یادگیری قوی ترنسفورمر، مدل همچنان تلاش کرده تا قالب ظاهری نمایشنامه را

مانند نوشتن نام کاراکتر با حروف بزرگ و گذاشتن دو نقطه را حفظ کند، هر چند محتوای دیالوگ ها بی معنی شده است

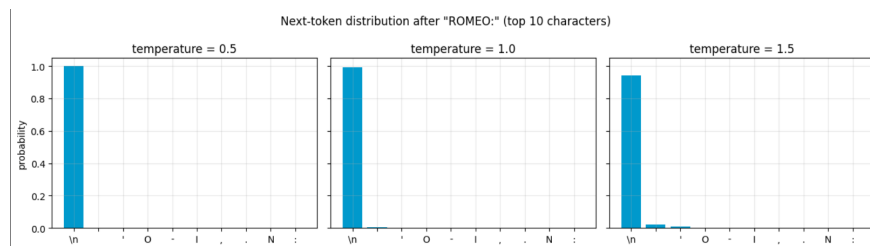
میزان Loss آموزش



شکل ۲۵: خطای آموزش minigpt

همانطور که در صورت سوال گفته شده بود، خطای آموزش از حدود 4.5 تا حدود 1.5 کاهش یافته است

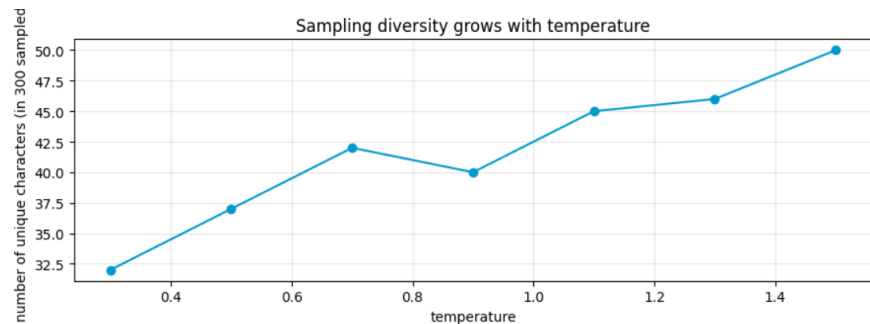
توزیع احتمالاتی توکن های بعد از ROMEO:



شکل ۲۶: نمودار احتمالاتی توکن های بعد از کاراکتر ROMEO:

همانطور که دیده میشود، احتمال اینکه بعد از کاراکتر ROMEO: متن به خط بعدی برود با اختلاف فاحشی بیشتر از سایر کاراکتر هاست. این نشان میدهد که مدل به خوبی ساختار نمایشنامه های شکسپیر را یاد گرفته است. در عین حال، مشاهده میشود که با افزایش دما احتمال، (T) رفتن به سطر بعدی کمی کمتر میشود و عباراتی مانند خط فاصله یا quote کمی شانس برای قرار گرفتن بعد از ROMEO: پیدا می کنند. این نمودار تاثیر پارامتر Temperature را در احتمال انتخاب کاراکتر های بعدی نشان می دهد

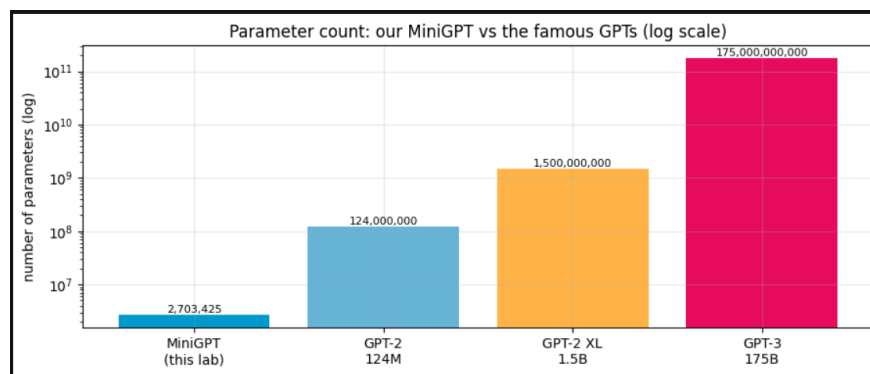
بررسی پارامتر Temperature در میزان تنوع کاراکترها



شکل ۲۷: نمودار تعداد کاراکترهای متمایز استفاده شده در متن براساس پارامتر Temperature

این نمودار، میزان تعداد کاراکترهای متمایز استفاده شده در ۳۰۰ کاراکتر نمونه برداری شده نشان می‌دهد. همانطور که انتظار می‌رود، نمودار صعودی است. با افزایش پارامتر Temperature ریسک پذیری مدل و تنوع در کلمه‌های استفاده شده بیشتر می‌شود. به طوری که در $T = 0.3$ کل نمونه ما با تقریباً ۳۲ کاراکتر متمایز ساخته شده در حالی که در $T = 1.5$ نمونه با ۵۰ کاراکتر متمایز ساخته شده است. اعدادی که نشان دهنده افزایش تنوع در کلمات متن با افزایش پارامتر Temperature می‌باشد.

میزان پارامترهای minigpt

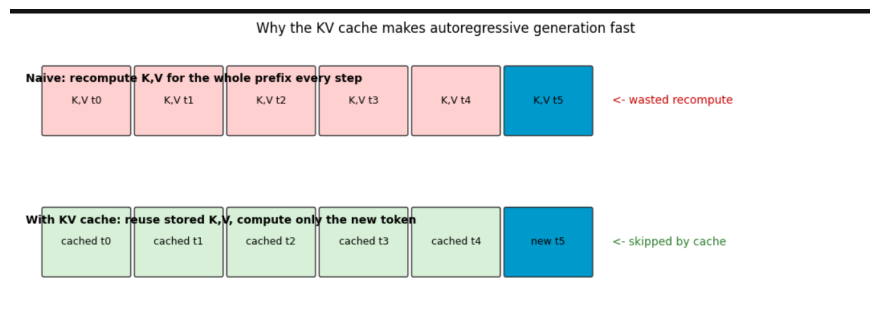


شکل ۲۸: مقایسه تعداد پارامترهای minigpt با سایر مدل‌های مشهور

این نمودار صرفاً به مقایسه تعداد پارامترهای minigpt ساخته شده با مدل‌های مشهور مانند GPT-3 می‌پردازد. تعداد پارامترهای minigpt برابر ۲۷۰۳۴۲۵ بوده و بیشترین تعداد پارامتر در مدل‌های مشهور برای GPT-3 برابر ۱۷۵ میلیارد است.

بررسی مکانیزم حافظه پنهان کلید-مقدار (KV-Cache)

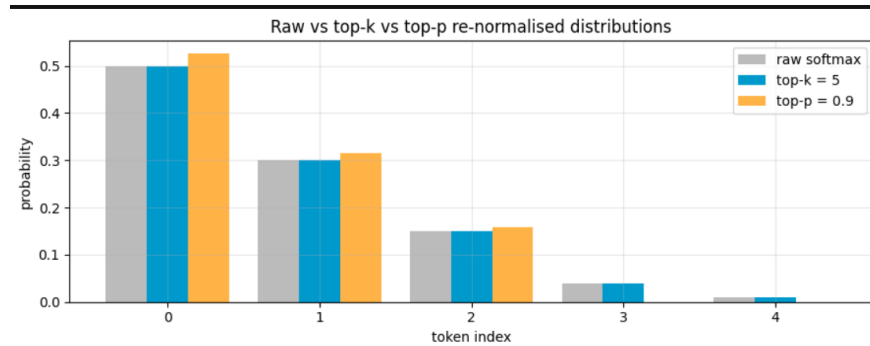
یکی از چالش‌های اصلی در تولید متن خودبازگشتی (Autoregressive) افزایش زمان محاسبات با طولانی شدن طول کانتکست است. این نمودار به مقایسه دو رویکرد ساده و رویکرد بهینه با استفاده از مکانیزم KV-Cache می‌پردازد.



شکل ۲۹: مقایسه دو رویکرد ساده و بهینه برای کاهش محاسبات تکراری

همان‌طور که در نمودار فوق مشاهده می‌شود، در روش ساده (Naive)، مدل در هر گام برای تولید توکن جدید مجبور است ماتریس‌های Key و Value تمام توکن‌های گذشته را از ابتدا بازمحاسبه کند که این امر منجر به اتلاف شدید منابع محاسباتی (wasted recompute) می‌گردد. در مقابل، با فعال‌سازی مکانیزم KV Cache، ماتریس‌های مربوط به توکن‌های قبلی در حافظه ذخیره شده (cached) و در گام‌های بعدی بازاستفاده می‌شوند. در این حالت، محاسبات تنها برای توکن جدید انجام شده و پیچیدگی زمان استنتاج هر توکن از حالت وابستگی به طول متن، به یک هزینه ثابت $O(1)$ بهبود می‌یابد که این امر سرعت تولید متن در مدل MiniGPT را به شدت افزایش می‌دهد.

مقایسه فیلترهای Top-k، Top-p و Softmax معمولی



شکل ۳۰: نمودار بررسی عملکرد فیلترهای Top-k، Top-p و حالت بدون فیلتر

نمودار بالا توزیع احتمالاتی ۵ توکن را روی دایره لغات فرضی نشان می‌دهد. نمودار به ما نشان می‌دهد که توزیع احتمالاتی ۵ توکن فرضی به ترتیب در حال کاهش است. نمودار میله‌ای طوسی رنگ، توزیع احتمالاتی توکن‌ها را با تابع Softmax معمولی و بدون فیلتر نشان می‌دهد. همان‌طور که مشخص است، توکن‌های ۴ و ۵ با اینکه احتمال کمی دارند، اما احتمال‌شان صفر نیست. در متن‌های طولانی این احتمال کم می‌تواند باعث شود مدل کلمه‌های بی‌ربطی را در بخش‌هایی از متن استفاده کند و در نتیجه رشته کلام از دست برود. نمودار آبی، وضعیت احتمالاتی توکن‌ها را با اعمال فیلتر $Top-k = 5$ نشان می‌دهد. چون کل لغات ما ۵ توکن است و این فیلتر ۵ تا از بهترین‌ها را نگه می‌دارد، عملاً هیچ توکنی حذف نشده و نمودار آن هیچ تفاوتی با نمودار طوسی ندارد.

فیلتر $Top-p = 0.9$ (نمودار زرد) پس از اعمال بر مدل، باعث حذف توکن‌های ۳ و ۴ از دایره انتخاب‌ها شده است. این فیلتر رفتار پویایی از خود نشان می‌دهد و وابسته به تعداد کلمات نیست. وظیفه این فیلتر انتخاب توکن‌هایی با بیشترین احتمال است به طوری که مجموع احتمال آنها به سقف ۹۰٪ برسد. به همین دلیل، توکن‌های ۳ و ۴ را حذف و شانس ۳ توکن اول را برای انتخاب بیشتر کرده



است